

Contents

Notation

This section lists symbols and terms used throughout *AI Compiler Performance Engineering*. Figures and tables scale to `\linewidth` via `\autowidthfigure` and the `\autowidthtable` environment.

LLM Inference and Serving

L	Prompt or context length (tokens) in prefill or cached history
m	Number of autoregressive output tokens generated after prefill
BS	Batch size (requests or sequences scheduled together)
SL	Sequence length in a benchmark configuration (prompt + generated tokens)
KV	Key-value cache tensors reused across decode steps
<i>prefill</i>	Parallel forward over prompt tokens; bulk KV write
<i>decode</i>	One (or few) new tokens per forward; KV read and append

Performance Metrics

tok/s	Throughput: tokens processed per second
ms/token	Latency: milliseconds per generated token
I	Arithmetic intensity (FLOPs moved per byte transferred)
HDBI	Host–Device Balance Index; HDBI $\rightarrow 0$ indicates host-bound serving
launches/token	GPU (or device) kernel launches counted per output token
bytes/token	Device memory bytes read/written per output token (weights, KV, activations)

Compiler, GPU, and Hardware

$\mathcal{IR}$	Compiler intermediate representation before lowering
<i>pass</i>	Analysis or transform applied to $\mathcal{IR}$ (fusion, tiling, layout)
<i>tile</i>	Static sub-block of a tensor mapped to on-chip memory
SM	Streaming multiprocessor on NVIDIA GPUs
CTA	Cooperative thread array (CUDA thread block)
TMA	Tensor Memory Accelerator (async bulk copy on Hopper/Blackwell)
HBM	High-bandwidth memory for weights, KV cache, and spill traffic
AIE	AMD AI Engine tile array (XDNA spatial dataflow)

Chapter 1

LLM Inference Performance Bottlenecks: From Symptoms to Root Causes

Large language model (LLM) serving has matured into a systems discipline where the dominant cost is often not FLOPs but *how* weights, activations, and the key-value (KV) cache move through memory hierarchies and how many discrete GPU kernels the host must launch per token. In production, most teams hit the same contradiction: prefill benchmarks look excellent on datasheets, yet interactive decode still misses p99 latency SLOs after months of kernel tuning. The durable misconception is that buying faster tensor cores or enabling FlashAttention “fixes” decode—when measured kernels-per-token and bytes-per-token often barely move because the stack remains operator-fragmented and host-orchestrated (??). This chapter establishes the performance vocabulary used throughout the book: we separate **prefill** from **decode**, quantify where latency hides in mainstream PyTorch and Hugging Face stacks, and explain why cross-hardware compilation is not an optional portability exercise but a prerequisite for industrial decode optimization.

1.1 Prefill and Decode: Two Performance Regimes

Autoregressive transformer inference divides naturally into two phases (?). In **prefill**, the model processes the entire prompt (or a prompt chunk) in parallel across sequence positions. Matrix multiplications are large, arithmetic intensity is

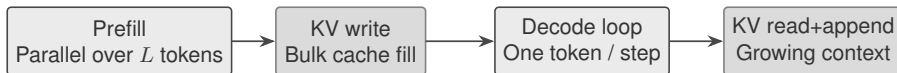


Figure 1.1: Prefill processes prompt chunks in parallel; decode appends one token per step via KV cache.

Table 1.1: Qualitative comparison of prefill vs. decode on a single transformer layer.

	Prefill	Decode (batch-1)
Tokens processed per step	L (prompt length)	1
Typical arithmetic intensity	Higher	Lower
Dominant limiter	Compute / tensor cores	Memory traffic / orchestration
KV cache growth per step	Written in bulk	Read + append one step
Serving objective	Throughput (tok/s)	Latency (ms/token)

relatively high, and modern accelerators can approach their peak tensor throughput. In **decode**, the model generates one new token per forward pass (per request, in the common batch-1 interactive setting), conditioned on all prior tokens through the KV cache.

These phases obey different roofline limits. Prefill is frequently *compute-bound* on high-end GPUs: the GEMMs and attention blocks are wide enough to amortize memory latency. Decode, by contrast, repeatedly reads the full parameter set and growing KV tensors while performing only a thin slice of matmul work per token. Treating both phases with one optimization playbook—larger batches, FlashAttention defaults, tensor-core-first schedules—is a primary reason decode SLOs remain elusive after prefill tuning succeeds. Multiple independent analyses converge on the same conclusion: interactive decode is typically *memory-bandwidth bound* on GPUs, CPUs, and many NPUs, not FLOP-bound (??).

Production systems increasingly *disaggregate* prefill and decode into separate pools because optimizing for one phase can harm the other. For example, large prefill batches maximize throughput, while decode pools prioritize tail latency under batch-1 or small-batch arrivals. Recent large-model deployments provision substantially more decode capacity than prefill capacity because user-visible latency is dominated by per-token decode time once the prompt has been ingested (??). DeepSeek-V3’s production layout is illustrative: the minimum prefill deployment unit spans 4 nodes (32 GPUs), whereas the minimum decode unit spans 40 nodes (320 GPUs)—a $10\times$ capacity skew reflecting phase-specific hardware mapping (?).

1.2 The Decode Pain Point: Single-Token, Low-Batch Latency

Interactive assistants, coding copilots, and edge voice interfaces share a harsh constraint: users measure quality in *time-to-next-token*, often at batch size one. At batch-1 decode, each layer performs narrow GEMMs (e.g., a single new query vector against frozen weights) and attention that must touch the entire cached history. The working set per token is small, but the *read footprint* is enormous: every weight matrix and every cached key/value vector must be visited again. This is why decode latency scales with model width and context length even when FLOPs per token look trivial on paper—the hardware spends its time streaming bytes, not executing MMA instructions (??).

Low batch sizes also prevent amortization of fixed costs—kernel launch, framework dispatch, collective synchronization, and memory allocator bookkeeping—across many tokens. ? decompose host-visible latency into three mutually exclusive components per kernel invocation: framework translation (ΔFT), CUDA-library front-end time (ΔCT), and kernel launch-path floor (ΔKT). Their Host–Device Balance Index (HDBI) relates device-active time to orchestration time; $HDBI \rightarrow 0$ marks a host-bound regime.

On H200 at batch-1 and sequence length 512, Llama-3.2-1B issues 850 kernel launches during prefill but 8,437 launches across a 10-token decode window (≈ 844 launches per autoregressive step)—nearly $10\times$ more dispatch work per forward pass (?). At decode configuration BS=4/SL=2048 ($m=10$ output tokens), dense Llama-3.2-1B averages 847.5 GPU kernel launches *per output token*; OLMoE-1B/7B averages 9,305 launches per token ($11\times$ denser) (?). Measured null-kernel launch floor on H100 is $4.707\ \mu s$ (median $4.578\ \mu s$); at 9,305 launches/token that launch-path term alone contributes $\approx 44\ ms$ per token before framework dispatch is counted (?).

The engineering implication is blunt: shaving a few microseconds from a matmul inner loop matters less than eliminating redundant global-memory round trips, fusing layer fragments, and collapsing launch storms—especially when the product SLA is sub-50 ms per token.

1.3 Limits of Mainstream PyTorch and Hugging Face Inference

The conventional PyTorch + Hugging Face stack looks logically complete—every transformer sub-block maps to a familiar op—yet multi-hardware deployment exposes a fatal gap: the *schedule* is decided at runtime by a Python loop, not at compile time by residency and bandwidth constraints. A naïve `transformers` loop—one Python forward per token, eager PyTorch ops, separate kernels for layer norm, QKV projection, attention, MLP, residuals—mirrors the mathematical graph but not the memory hierarchy. Each operator materializes intermediates to global memory (GPU HBM or CPU DRAM), so a single decoder layer can trigger many read/write passes over tensors that could have stayed on chip given a different schedule.

Three structural problems recur:

1. **Operator fragmentation.** A decoder block decomposes into 10–20+ discrete kernels in unfused stacks. Each boundary exports data to HBM and re-imports it for the next op.
2. **Host orchestration tax.** Python dispatch, CUDA driver launch, and framework bookkeeping run on the CPU while the accelerator idles between micro-kernels (?).
3. **KV cache management overhead.** Without paged or fused cache layouts, attention reads scatter through memory, wasting bandwidth on indexing and padding (??).

Serving frameworks such as vLLM address *capacity* and *KV fragmentation* via PagedAttention and continuous batching (?), but they do not by themselves remove per-layer operator boundaries inside the forward pass. FlashAttention reduces HBM traffic during prefill by IO-aware tiling (?); Flash-Decoding adds a parallelization dimension along KV sequence length so batch-1 decode can still saturate SMs when context is long (reported up to $8\times$ end-to-end speedup at very long sequences) (?). These are necessary optimizations, yet a full decoder step still contains MLPs, norms, rotary embeddings, and sampling logic that remain separable kernels unless a **MegaKernel** or compiler-generated fused schedule reunifies them.

On CPUs the same fragmentation shows up as LLC misses rather than HBM stalls: each unfused boundary evicts cache lines that a fused decoder block could have kept resident across norm–matmul–activation chains. On XDNA-class NPUs,

Table 1.2: Verified kernel fragmentation on H100 decode (BS=4, SL=2048, $m=10$ tokens), from TaxBreak (?).

Model	Kernels/token	Total launches	GPU util. (%)
Llama-3.2-1B	847.5	8,475	58.9
Llama-3.2-3B	1,536.9	15,369	67.6
OLMoE-1B/7B	9,305.3	93,053	15.5
Qwen1.5-MoE-A2.7B	6,695.1	66,951	27.7

per-op dispatch is worse than wasteful—it violates static dataflow contracts and forces fallbacks to scalar host loops (?). Industrial deployment therefore cannot copy a single “GPU recipe” into production; the framework graph must be re-planned per memory hierarchy, which is exactly the job YiRage takes over in Part VI.

1.4 Quantifying Overheads: Launches, Memory, and Scheduling

Recent microbenchmark studies decompose end-to-end decode latency into compute, memory, and *orchestration* components. TaxBreak reports that for Llama-3.2-1B-class models on H100, decode reduces orchestration time relative to prefill in absolute terms, but because decode issues roughly $10\times$ more launches per step, per-dispatch savings compound more aggressively in decode (?). For MoE models, decode latency can grow to multiple seconds per token at batch-1 not because GEMMs are huge, but because dispatch dominates: thousands of kernels per token with a per-launch floor on the order of microseconds yields tens of milliseconds of pure launch-path cost before framework overhead (?).

CUDA Graphs record a static kernel DAG once and replay it with a single submission, amortizing CPU launch and framework dispatch (??). ? ran a controlled CUDA Graphs A/B on Qwen-2.5-7B at batch-1, context 2048, across $N=10$ independent sessions: H100 speedup is $1.259\times$ (95% bootstrap CI [1.253, 1.267]), removing ≈ 3.05 ms of per-step host overhead ($\approx 20.6\%$ of the eager 14.83 ms median). On L4 at the same cell, Graphs yield only $1.028\times$ because the GPU already achieves $R_{\text{floor}}=0.81$ of its analytic HBM bandwidth floor—the device waits on memory, not launches (?).

Example 1.4.1. For Qwen-2.5-7B on H100, a single decode step issues ≈ 283 kernel launches (≈ 10 per layer $\times$ 28 layers plus embedding/LM-head) (?). At

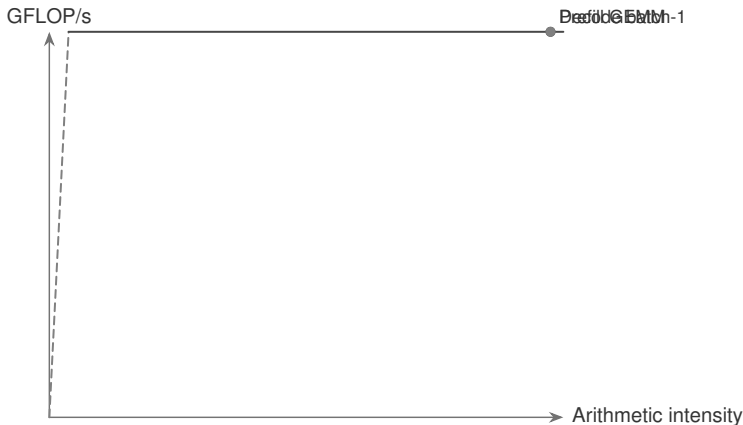


Figure 1.2: Decode sits on the memory-bandwidth slope of the roofline at low arithmetic intensity.

batch-1 interactive decode, per-layer GPU compute can be $\approx 10 \mu s$ while launch gaps are $\approx 30 \mu s$ —launch dominates per block (?). Collapsing those boundaries via MegaKernel fusion or graph capture can exceed the benefit of micro-optimizing individual GEMMs.

1.5 Memory Bandwidth, Not Peak FLOPs

The roofline model makes the decode story precise. Arithmetic intensity (FLOPs per byte moved) for batch-1 decode is low: one token’s activations participate in thin matrix–vector-like operations while weights and KV cache supply the bulk of bytes. Accelerators therefore sit on the memory-bandwidth sloped region of the roofline rather than the flat compute cap (??). The LIMINAL limit study further reports that at low batch, tensor-core utilization can be $\leq 1\%$ on both DRAM- and SRAM-based accelerator designs when compute is reasonably provisioned—decode simply does not present enough arithmetic intensity to engage peak FLOPs (?).

? quantify “achieved fraction of peak bandwidth” via $R_{\text{floor}} = t_{\text{floor}}/t_{\text{obs}}$ where $t_{\text{floor}} = (W + K)/B_{\text{peak}}$. For Qwen-2.5-7B at $\text{ctx}=2048$, batch-1: L4 reaches $R_{\text{floor}}=0.81$ (near the HBM roofline), while H100 reaches only $R_{\text{floor}}=0.27$ —faster silicon realizes a smaller fraction of theoretical memory throughput because fixed per-step launch overhead does not scale with bandwidth (?).

This explains why peak TFLOPs and peak HBM bandwidth are misleading single-number KPIs for chat latency: on H100-class GPUs the gap is often

orchestration-limited; on L4-class GPUs it is bandwidth-limited. It also explains why decode often *should not* default to tensor-core-oriented kernels: the bottleneck is feeding data, not lacking MMA throughput—a theme we develop for Hopper/Blackwell in later chapters and for fused MegaKernels in Part V.

Quantization reduces bytes moved and can shift decode toward compute-bound regimes on some GPUs, but only when the runtime actually realizes memory savings. On L4 with Qwen-2.5-7B at $\text{ctx}=2048$, bf16 decode median is 62.32 ms while bitsandbytes nf4 is 59.36 ms and AutoAWQ+Marlin is 45.24 ms—far less than the $4\times$ traffic reduction would predict. GPTQ+ExLlamaV2 (Ada-tuned int4 GEMMs) reaches 17.36 ms, a $3.59\times$ speedup over bf16; an L4 with ExLlamaV2 (17.36 ms/step) can approach an H100 with CUDA Graphs (11.78 ms/step) despite $\approx 11\times$ lower peak HBM bandwidth (?).

1.6 Multi-Hardware Pain Points: GPU, CPU, and NPU

The decode bottleneck *shape* is universal; the *remedy* is not.

NVIDIA and AMD GPUs. Warp occupancy, shared memory tiling, coalesced global loads, and tensor-core vs scalar-core selection dominate. Launch overhead and CUDA Graphs capture matter most when compute kernels are already short—exactly the decode regime on H100 (??). AMD HIP stacks mirror many CUDA concerns but differ in wavefront width, LDS sizing, and ROCm graph support.

x86 and ARM CPUs. Decode is still memory-bound but the hierarchy is cache- and NUMA-shaped rather than HBM-shaped. SIMD width (AVX-512/NEON), core pinning, and KV layout for cache locality replace shared-memory tiling. Operator fusion reduces DRAM round trips; excessive parallelism can thrash LLC without NUMA-aware allocation. A common pitfall is porting GPU-oriented fusion heuristics verbatim: medium-grain CUDA fusion that helps H100 can *inflate* CPU decode latency when it destroys cache blocking or forces cross-socket KV traffic on dual-socket servers. For CPU decode, the first lever is often KV tensor layout (contiguous head dimensions, page size aligned to cache lines) and coarse layer fusion that keeps QKV+MLP fragments in L3 for the duration of a token step—not micro-kernels tuned for warp occupancy.

Edge NPUs (XDNA/AIE and similar). Spatial dataflow arrays (AMD XDNA) emphasize static tile assignment, on-chip buffer residency, and DMA pipelines with minimal dynamic scheduling (??). Decode favors fully static graphs, large fused regions, and pre-planned memory reuse; dynamic shapes and per-op

Table 1.3: Decode optimization priorities by hardware class.

Hardware	First-order decode levers
NVIDIA GPU	Fusion, graphs, KV bandwidth, shared mem tiling
AMD GPU	HIP fusion, LDS reuse, launch reduction
CPU	Cache blocking, SIMD, NUMA, coarse fusion
Edge NPU	Static ADF graphs, DMA ping-pong, full-layer fusion

[Placeholder] Run Appendix B harness before filling bars

Figure 1.3: Pending: YiRage/MegaKernel vs. PyTorch eager speedup (Appendix B harness).

Python dispatch are architectural mismatches. The compiler must allocate buffers, routes, and DMA descriptors at compile time rather than relying on a runtime kernel launcher. Where CUDA permits flexible grid launches and runtime shape guards, AIE tiles require fixed ADF graphs: if the compiler cannot prove tensor lifetimes fit on-chip SRAM, the schedule fails bring-up rather than degrading gracefully to DRAM-heavy eager paths. That asymmetry is why NPU decode optimization ranks *static fusion* and *DMA ping-pong* above attention-kernel swaps—the binding constraint is residency proof, not FLOP count.

1.7 Industry Benchmarks: Native Frameworks vs. Fused Kernels

Industry leaderboards rarely tell the decode story teams actually ship against. Published prefill throughput (tokens/s on long prompts) and single-kernel attention speedups are real contributions, but they answer a different question than batch-1 ms/token under continuous batching with growing KV state. The technique is sound in theory—FlashAttention, PagedAttention, CUDA Graphs each remove a documented bottleneck—yet in live serving the stack often regresses because remaining layer fragments still export activations to HBM and the host still pays thousands of launches per token on MoE stacks (??). Benchmark literacy therefore means tracking normalized diagnostics—kernels/token, bytes/token, R_{floor} , HDBI—on *identical* model revisions, not comparing marketing peaks across mismatched batch policies.

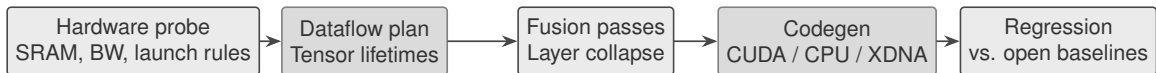


Figure 1.4: YiRage decode compilation pipeline (preview).

Published baselines establish the performance gap that motivates this book’s MegaKernel and compiler stack. Flash-Decoding reports up to $8\times$ faster generation at very long contexts by parallelizing KV loads; attention-only speedups can reach $50\times$ versus unfused FlashAttention at batch-1 because standard attention parallelizes poorly when query length is 1 (??). vLLM demonstrates that serving-system optimizations (paged KV, continuous batching) improve throughput under concurrent requests but leave per-kernel efficiency inside the transformer block as a separate axis (?).

TaxBreak and related studies quantify that framework-level improvements cannot fully absorb launch inflation when models fragment into thousands of micro-kernels per token—especially MoE architectures (?). The gap we target in later chapters is the additional uplift from *whole-decoder fusion*: replacing on the order of ten to twenty discrete kernels with one dataflow-planned MegaKernel that keeps activations in registers and shared memory across QKV, attention, MLP, and residuals. Exact speedups are model-, hardware-, and quantization-dependent; this book reports measurements through a unified benchmark harness (Appendix B) rather than anecdotal single-point claims.

1.8 Opening Case Study: YiRage Cross-Hardware Compilation Preview

YiRage is the book’s reference **hardware-aware** LLM compiler: it ingests high-level decoder graphs, models chip constraints (memory tiers, launch granularity, SIMD/SIMT width), and emits fused schedules for CUDA, CPU, and XDNA-class targets. This chapter’s role is to state the *problem* YiRage solves; subsequent parts develop the hardware abstractions, manual MegaKernel craft (Part V), and automated pass pipelines (Part VI).

A preview of the YiRage workflow for decode:

1. **Hardware probe.** Read bandwidth caps, on-chip SRAM sizes, and static-scheduling rules for the target.
2. **Global dataflow plan.** Assign tensor lifetimes to register, shared/LDS, or

global tiers before choosing individual ops.

3. **Fusion and pipeline passes.** Collapse decoder layers; insert async copy pipelines (TMA on Hopper, DMA on AIE) where hardware supports them (??).
4. **Backend code generation.** Emit a single kernel or captured graph per layer group; tune tile sizes per architecture.
5. **Regression against open baselines.** Compare to PyTorch eager, Flash-Decoding, and framework servers on identical models (Pythia-2.8B, Qwen2.5 family) across GPU, CPU, and NPU targets.

The preview case establishes performance baselines and documents compiler constraints discovered during bring-up—not to claim a single universal speedup factor, but to show that *hardware-aware dataflow compilation* closes the gap between hand-tuned CUDA and production-ready multi-target deployment.

Cross-hardware bring-up surfaces constraints that single-GPU books omit. A fusion pass that succeeds on CUDA may fail XDNA buffer allocation; a CPU backend may require different tile shapes to keep AVX units fed without blowing L2. YiRage encodes these outcomes as *hard failures* during compilation (illegal residency, launch granularity mismatch) rather than silent slowdowns at runtime—matching how industrial teams gate releases. This preview chapter therefore ends with requirements R1–R5 (§??) that every later pass must satisfy with measured regression evidence, not anecdotal speedups on one card.

1.8.1 Diagnostic Workflow: TaxBreak and Memory-Floor Studies

Before optimizing kernels, industrial teams must answer: is this workload host-bound or device-bound, and *which host layer* dominates? TaxBreak answers the first decomposition: reduce $T_{\text{Orchestration}}$ via `torch.compile` when $\sum \Delta FT + \sum \Delta CT$ dominates; fuse kernels or adopt CUDA Graphs when $N \cdot T_{\text{sys}}^{\text{floor}}$ dominates; tune driver/runtime when per-kernel $\Delta K T_{fw}$ residuals are large (?). For MoE decode, TaxBreak shows persistently low HDBI even at BS=16: routing fragments execution into $8\text{--}11\times$ more launches per token than dense models of comparable activated parameter count, and faster H200 hosts reduce orchestration 10–29% versus H100—CPU single-thread performance is a first-order knob for host-bound decode (?).

The memory-floor study complements TaxBreak on the device side: when R_{floor} is high (L4, ≈ 0.8), optimize bytes moved (quantization kernels that actually stream int4 weights, KV layout). When R_{floor} is low (H100, ≈ 0.27), optimize launches

(graphs, fusion) before chasing attention-kernel swaps—on H100 batch-1 decode, explicit FlashAttention-2 eager paths can be *slower* than PyTorch SDPA because attention is not the binding constraint (??). This book’s YiRage passes encode both diagnostics as regression gates (Appendix F).

1.8.2 Roofline Intuition for Decoder Layers

The roofline model plots attainable FLOP/s as $\min(\text{peak FLOP/s}, \text{bandwidth} \times \text{AI})$, where arithmetic intensity (AI) is FLOPs per byte transferred from DRAM (?). For a decoder layer at batch-1, AI is roughly

$$\text{AI} \approx \frac{2N_{\text{params}} + 2N_{\text{KV}}}{N_{\text{params}} + N_{\text{KV}}} \quad (1.1)$$

in the crudest weight+KV read model (ignoring activations and writebacks), where N_{KV} grows linearly with context length. As context grows, AI rises slowly but usually remains on the bandwidth-limited slope for practical LLM sizes on GPUs. This is why Flash-Decoding and fused attention kernels fight for *bytes* and *launch count*, not raw MMA throughput (?).

1.8.3 Why Disaggregated Serving Changes the Optimization Target

When prefill and decode run on separate fleets, decode nodes see a steadier stream of single-token steps with hot weights resident on device. That shifts engineering effort toward (i) per-step latency, (ii) KV cache locality, and (iii) kernel persistence (CUDA Graphs, MegaKernels) rather than maximizing prompt throughput (??). Disaggregation also exposes *network* and *serialization* costs if KV tensors move between pools—another reason compile-time dataflow planning must account for where tensors live across steps.

1.8.4 Tensor Cores and Decode: A Common Misconfiguration

Tensor cores excel when GEMMs present sufficiently large $M \times N \times K$ tiles. Decode batch-1 often reduces effective M to one (or a tiny batch), shrinking MMA utilization and increasing setup overhead (?). Hopper’s Tensor Memory Accelerator (TMA) shifts address-generation and bulk copy work off scalar threads, but TMA still favors transfers large enough to amortize descriptor setup—small KV slices may remain latency-bound on regular loads (??). The book’s MegaKernel strategy therefore treats tensor cores as *optional epilogues* inside a dataflow schedule that

first minimizes bytes and synchronization, then maps surviving GEMMs to MMA when dimensions justify it.

1.8.5 From Symptoms to Design Requirements

The pain points in this chapter translate into concrete compiler and kernel requirements used in Parts IV–VII:

- **R1 — Minimize global round trips.** Keep decoder activations in registers or shared memory across matmul, norm, activation, and attention phases.
- **R2 — Static schedules where possible.** Prefer compile-time tile assignment, especially on NPUs with spatial dataflow (?).
- **R3 — Hardware-shaped fusion granularity.** GPUs tolerate medium fusion; CPUs need cache-aware blocking; NPUs require layer-scale or model-scale static graphs.
- **R4 — Launch amortization.** Use graphs, persistent kernels, or single MegaKernels to remove host dispatch from the critical path (??).
- **R5 — Measurable regression gates.** Every pass must show bytes/token and kernels/token improvements on a fixed benchmark matrix.

These requirements are the bridge to chapter ??’s successor chapter on dataflow-first kernel design.

1.8.6 Measurement Methodology Used in This Book

We report decode latency as **median ms/token** at batch-1 unless stated otherwise, with warmup and fixed context lengths (512, 2048, 8192). Throughput experiments use continuous batching where noted. Every claim ties to a reproducible configuration: model revision, precision, framework version, driver, and compiler commit. Placeholder rows in Appendix B mark experiments pending hardware access; we never interpolate fake speedups. Cross-hardware tables normalize by *bytes moved per token* and *kernels launched per token* so readers can compare mechanisms, not marketing peaks (??).

Table 1.4: Literature-backed decode metrics referenced in this chapter (conditions vary by row).

Metric	Value	Source / conditions
Prefill vs decode node ratio	$10\times$ decode	DeepSeek-V3: 32-GPU prefill unit vs 320-GPU decode unit (?)
Dense kernels / decode step	≈ 844	Llama-3.2-1B, H200, BS1/SL512, vs 850 prefill (?)
MoE kernels / output token	9,305	OLMoE-1B/7B, H100, BS4/SL2048, $m=10$ (?)
Launch floor ΔKT	$4.707\ \mu s$	H100 null-kernel median (?)
CUDA Graphs speedup	$1.259\times$	Qwen-2.5-7B, H100, ctx2048, $N=10$ sessions (?)
CUDA Graphs on L4	$1.028\times$	Same cell; bandwidth-bound null result (?)
R_{floor} H100 / L4	0.27 / 0.81	Qwen-2.5-7B, ctx2048, batch-1 (?)
Low-batch tensor util.	$\leq 1\%$	LIMINAL limit study, DRAM/SRAM xPU (?)
Flash-Decoding speedup	up to $8\times$	Very long context, attention parallelized on KV (?)

1.8.7 Verified Reference Numbers (Literature Snapshot)

Table ?? collects externally published measurements cited in this chapter. These are *not* YiRage internal benchmarks; they anchor the problem statement before we introduce book-specific experiments in later parts.

When migrating these lessons across hardware targets, compare normalized metrics—kernels per token, bytes per token, R_{floor} , HDBI—rather than absolute milliseconds, which entangle silicon generation, framework version, and batching policy.

1.8.8 Anti-Patterns That Waste Engineering Cycles

Three optimization anti-patterns recur in decode programs that miss SLOs despite heavy investment. **(1) Chasing peak FLOPs.** Teams profile GEMMs, tune tile sizes, and declare victory while launch storms and HBM re-reads dominate ms/token (?). **(2) Single-hardware tunnel vision.** A CUDA Graphs capture that rescues H100 decode may be irrelevant on L4 (bandwidth-saturated) or illegal on static NPU targets (??). **(3) Attention-only myopia.** Replacing

SDPA with a flash variant when R_{floor} is already low on H100 can *slow* decode because attention was never the binding constraint (?). The corrective workflow is diagnostic-first: measure HDBI and R_{floor} , then select fusion depth, graph capture, quantization kernels, or compile-time dataflow replanning—in that priority order when orchestration dominates. Teams that skip this ordering often re-implement the same matmul tweak quarterly while p99 latency flatlines—because the root cause was never FLOPs, it was residency and dispatch. Industrial deployment cannot copy a single template; hardware class dictates whether graphs, fusion depth, or static NPU graphs come first on the critical path.

1.9 Chapter Summary

Decode-phase LLM inference is a systems problem disguised as a modeling problem. Prefill and decode obey different limits; interactive decode at low batch is dominated by memory traffic and orchestration overhead, not peak tensor FLOPs. Mainstream PyTorch and Hugging Face paths fragment layers into many kernels, amplify host launch costs, and under-utilize on-chip storage. GPUs, CPUs, and NPUs each demand different fusion granularities and compilation strategies—there is no portable “best kernel,” only hardware-shaped schedules. Industry kernels (FlashAttention, PagedAttention) and serving systems (vLLM) are necessary but insufficient for the extreme fusion pursued in this book. YiRage provides the automated, cross-hardware path from the manual MegaKernel insights developed next. This is the book’s recurring thesis: performance is a property of *data movement and orchestration under a specific memory hierarchy*, and compilers must own that schedule end-to-end.

Chapter 2

Two Kernel Design Mindsets: Operator-Driven vs Dataflow-Driven

Chapter ?? showed *what* breaks in decode: launch storms, HBM round trips, and hardware-specific bottlenecks. A durable industry misconception is that the fix is “one more fused op” inside an otherwise operator-first stack—when the schedule itself still exports every intermediate to global memory between mathematically adjacent compute stages (??). This chapter completes the pivot: from **operator-driven** schedules (compute-first, memory-second) to **dataflow-driven** schedules (residency-first, compute as a consequence of placement). Readers should leave with a decision procedure: given a decoder bottleneck measurement (kernels/token, bytes/token, HDBI, R_{floor}), classify whether the next engineering dollar belongs in op micro-tuning or dataflow replanning—and default to replanning when Chapter 1 diagnostics show orchestration or HBM export dominance (??).

2.1 Operator First

In production, most CUDA teams still begin decoder optimization by listing operators: LayerNorm, QKV projection, attention, MLP, residual adds. Each op receives a tuned kernel; the runtime stitches them with separate launches. The failure mode is predictable: *compute* micro-optimizations improve individual kernels while end-to-end ms/token stalls because every boundary writes activations to HBM and pays another launch (?). Operator-first design treats memory as

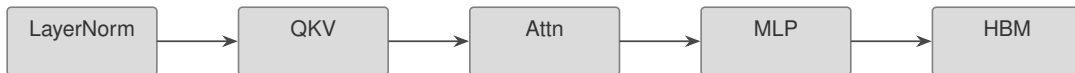


Figure 2.1: Operator-driven schedule: each op reads/writes global memory.

an implementation detail behind each op. PyTorch eager mode is the canonical embodiment: the graph is correct, but the schedule is emergent. FlashAttention grafts faster replacements for single nodes; it does not change export/import between nodes unless a compiler pass merges regions (?). On GPUs this yields hundreds to thousands of kernels per decode token; on CPUs LLC churn; on NPUs illegal dynamic schedules (?). Teams can spend quarters shaving nanoseconds inside MMA loops while HDBI remains near zero (??). The operator-first workflow also fragments accountability: kernel owners optimize “their” op, while no team owns the HBM edges between ops—yet those edges dominate decode (?). Compiler stacks that expose only per-op autotuning (template GEMMs, library dispatch tables) reinforce this silo unless a higher pass owns cross-op lifetime analysis. Dataflow-first teams invert the org chart: one schedule owner, many op specializations *inside* fused regions.

Example 2.1.1. A dense 7B decoder layer on H100 at batch-1 may spend $\sim 15\text{--}25\mu\text{s}$ in device compute per op group but $\sim 30\text{--}50\mu\text{s}$ waiting on launches and framework translation between groups (??). An operator-first team that speeds each compute group by 20% saves only $\sim 3\text{--}5\mu\text{s}$ aggregate while launch gaps remain. A dataflow-first team that fuses three groups into one dispatch may eliminate two launch gaps ($\sim 60\text{--}100\mu\text{s}$ class savings on launch-bound configs) even if per-op compute is unchanged. The example is qualitative—exact numbers vary by model and driver—but the *ranking* of leverage is what production teams must internalize.

2.2 Data Residency

The root cause is not insufficient FLOPs; it is **data residency** violated at every layer boundary. Activations that could live in registers or shared memory for the full QKV→attention→MLP chain are materialized to global memory after each sub-step. Each round trip pays full HBM bandwidth and pollutes CPU caches (??). Residency is a compile-time allocation problem: which tensors coexist, which tier holds them, for how many fused stages? On GPUs the tiers are registers → shared → L2 → HBM; on CPUs cache and NUMA; on XDNA/AIE hardware-enforced tile buffers (???). When residency succeeds, arithmetic intensity rises because bytes moved drop even if FLOPs stay constant (?). PagedAttention improves KV

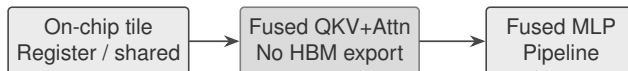


Figure 2.2: Dataflow-driven schedule: activations stay on-chip across fused stages.

capacity but does not guarantee on-chip residency inside a fused decoder region (?). Quantization changes residency economics: int4 weights reduce bytes streamed per layer, but only when kernels truly consume compressed layouts without dequant scratch in HBM (?). A residency plan must therefore co-design precision, layout, and fusion: nf4 weights with bf16 activations still ping-pong if every op dequantizes to global buffers. Industrial teams should measure *bytes per token* and *unique HBM touch count* per layer alongside FLOPs—metrics that residency planning directly minimizes.

Residency analysis also exposes **anti-locality** in popular serving features. Continuous batching improves throughput by reusing weight matrices across requests, but if each request’s activations still round-trip through HBM inside the layer, tail latency per request remains decode-bound (?). Speculative decoding adds draft-model traffic—another residency consumer—that operator-first stacks often bolt on without revisiting fusion boundaries. Dataflow planners must account for multi-model residency simultaneously: base model weights, draft model weights, and KV streams competing for bandwidth and on-chip space (?). The book’s later compiler passes treat these as first-class buffer interference constraints rather than afterthought flags.

2.3 Dataflow Philosophy

Dataflow design names the schedule before operators. Four commitments recur across targets:

1. **Static placement** where possible—tensor lifetimes planned before codegen; NPUs require static ADF graphs (?).
2. **On-chip residency first**—fusion depth keeps producer–consumer pairs in the fastest tier.
3. **Pipelined movement**—TMA on Hopper and DMA on AIE hide transfers when the graph exposes prefetch edges (??).
4. **Minimal synchronization**—dispatch unit is the pipeline stage, not each elementary op.

Table 2.1: Design mindset vs. hardware enforcement model.

Target	Data residency model
CUDA GPU	Software-managed hierarchy
XDNA / AIE	Hardware-enforced spatial buffers
CPU	Cache hierarchy + explicit blocking

Porting a CUDA operator list to XDNA without a static pipeline yields no legal schedule. On GPUs, ignoring dataflow still “works”; on NPUs compile fails—explaining why operator-first CUDA expertise does not automatically transfer to edge fleets. Static dataflow does not mean frozen models: it means shapes and buffer slots are known at compile boundaries (per deployment SKU), with guarded recompilation when service configs change. Serving systems that mix dynamic batching with static fusion must separate *scheduling* concerns (which requests share a batch) from *kernel* concerns (what stays on-chip during a fused step) (?). Confusing the two produces graphs that are dynamically batched at the framework level but still operator-fragmented inside each batch member—the worst of both worlds for tail latency.

Dataflow philosophy also reframes **correctness work**. Operator-first testing validates per-op numerical parity against reference implementations. Dataflow-first testing must add **schedule parity**: fused pipelines must match unfused references within tolerance while preserving tier assignments across backends. Regression suites therefore include golden activation dumps at stage boundaries during development, then strip those dumps in production builds once parity is proven—a pattern YiRage CI adopts for multi-target codegen. The extra testing cost pays back when a single fusion bug would otherwise ship as a $1.5\times$ silent slowdown on only one hardware class.

2.4 Cross Hardware

Cross-hardware comparison is not an appendix exercise—it is how teams avoid shipping GPU-tuned schedules that silently fail on edge NPUs or underperform on CPU inference nodes co-located with GPU fleets (??). The book’s benchmark harness (Appendix B) always reports at least three SKUs when hardware access allows: one bandwidth-rich GPU, one orchestration-sensitive GPU, and one CPU or NPU class target. Mindset differences show up as *different optimal fusion depths* on the same mathematical model, not as different models.

For **CUDA GPUs**, shared memory sizing, register pressure, and occupancy set fusion depth. CUDA Graphs amortize launches but replay a fixed dataflow; they do not invent residency that eager boundaries destroyed (??). Hopper TMA raises async movement ceilings but needs static copy edges (??). For **CPUs**, residency maps to **cache** blocking and NUMA placement; GPU-style mega-fusion can evict KV working sets. SIMD kernels benefit when dataflow keeps head layouts hot across QKV and MLP epilogues. For **XDNA / AIE**, residency is hardware-enforced: spatial tiles, DMA routes, compile-time buffer tables (??). Dynamic shapes and per-op dispatch are mismatches; the dataflow graph *is* the program. One mindset expresses differently per target, but operator-first lists fail everywhere for the same reason—no global residency contract. Table ?? summarizes enforcement models; the engineering consequence is that **fusion granularity** differs by class: GPUs tolerate medium fusion with shared-memory staging; CPUs need cache-aware blocking; NPU require layer-scale or multi-layer static regions (?). Cross-hardware portability is portability of the *dataflow graph and lifetime annotations*, not of individual op implementations. YiRage backends swap codegen while preserving tier assignments—the topic of Parts IV–VI.

2.5 Iron Rules

Rule 1 — Registers hold ephemeral scalars and narrow vectors, not whole feature maps; spills mean the split is wrong (?). **Rule 2 — Shared memory / LDS / on-chip SRAM** holds fusion regions; depth ends at capacity conflicts, not when the op list ends. **Rule 3 — Global memory** is for weights, KV streams, and true outputs—not ping-pong activations between adjacent ops (??). Violations produce Chapter 1 pathologies: high kernels/token and low R_{floor} when export/import dominates (??). Use iron rules as release gates: if a proposed schedule inserts a global round trip between ops that share a producer–consumer edge in the mathematical graph, reject it unless profiling proves the tier overflow is unavoidable. On GPUs, “unavoidable” should trigger shared-memory tiling or pipeline splitting, not acceptance of HBM ping-pong. On NPUs, overflow should fail compile and force graph refactoring—not runtime fallback loops on the host.

Register pressure on GPUs frequently triggers the false comfort of “spilling is fine because the kernel still runs.” In decode, spilled activations often become HBM traffic that negates fusion (??). Iron Rule 1 therefore pairs with occupancy discipline: if widening tile sizes forces spills, shrink fusion depth instead of celebrating higher theoretical occupancy on paper. CPU analogs apply to stack spills and false sharing when fusing across threads without cache-line-aware partitioning.

2.6 MegaKernel Preview

A **MegaKernel** dispatches an entire decoder layer with activations resident across QKV, attention, MLP, norms, and residuals—the antithesis of Figure ???. MegaKernels are fusion schedules with staged pipelines, not one giant matmul (??). Chaining vendor libraries without shared residency drops launches modestly but often not HBM traffic. True MegaKernels collapse both; TaxBreak still shows order-of-magnitude launch gaps on MoE graphs after partial fusion (?). They trade flexibility for performance—acceptable when model revisions are pinned per fleet. Part V develops manual craft; Part VI automates the same decisions in YiRage. MegaKernel engineering also clarifies **where not to fuse**: attention kernels that require global KV streaming may remain distinct stages while MLP+norm+residual fuse around them—dataflow does not mandate monolithic single-kernel binaries, it mandates explicit staging with minimal exports. Flash-Decoding parallelizes KV dimension while keeping decode batch-1 viable (?); a MegaKernel schedule must *compose* with such attention stages rather than naively inlining them and destroying register budgets.

2.7 Automation Path

YiRage ingests decoder graphs, plans lifetimes globally, runs fusion passes, and emits CUDA, CPU, or XDNA backends from one dataflow IR (??). The automation path encodes expert workflow—probe hardware, plan residency, fuse, codegen, regress—as compiler logic. Each backend re-instantiates the same dataflow with different tier rules: GPU shared-memory fusion and graph capture (?); CPU cache blocking; XDNA static buffers and DMA ping-pong (?). YiRage fails compile when residency cannot be proved—preferable to silent production slowdowns. Optimize dataflow first; let operators fall out of the schedule second. Automation must preserve human-debuggable artifacts: lifetime graphs, fusion decision logs, and regression tables comparing bytes/token and kernels/token against PyTorch eager and framework servers (?). Without those artifacts, compiler automation reintroduces opacity—teams cannot tell whether a regression is a bad tile size or a violated iron rule. YiRage treats diagnostics as first-class outputs alongside generated kernels, echoing TaxBreak and memory-floor methodologies from Chapter 1.

2.7.1 Compiler IR View

At the compiler layer, the mindset shift appears as a change in primary IR artifacts. Operator-first pipelines lower high-level frameworks (Torch, ONNX) into op sequences early; each op lowers independently to LLVM, Triton, or CUDA C++ with late, local fusion peephole passes. Dataflow-first pipelines retain a *lifetime graph* on tensors until hardware tier assignment completes: SSA values carry storage class annotations (`reg`, `shared`, `global_stream`) and fusion legality is checked on inter-value edges. GPU backends emit kernels per fused region; CPU backends emit blocked loops; XDNA backends emit ADF graphs with DMA edges (?). The IR difference is why “drop in FlashAttention” is not equivalent to “run the fusion pass pipeline”: the former swaps an op implementation; the latter rewrites the graph topology.

2.7.2 Failure Modes When Mixing Mindsets

Hybrid stacks—dataflow planning in a planner service but operator-first codegen in backends—produce characteristic failures. **Planner drift:** fusion regions computed on a meta-graph diverge from what the backend actually emits, reintroducing hidden exports. **Partial graph capture:** CUDA Graphs record a subgraph that still contains HBM ping-pong inside recorded nodes (?). **False residency:** shared-memory buffers declared fused but spilled registers silently increase global traffic (?). **NPU illegalize late:** graphs accepted on GPU fail XDNA buffer allocation in production edge builds (?). Guard against hybrids with single-source dataflow IR and fail-fast legality checks per backend, the YiRage default posture.

2.8 Contrasting Schedules on One Decoder Layer

Consider a single transformer decoder layer at batch-1 as the book’s running microscope. **Operator-first execution** issues separate kernels for RMSNorm, QKV linear, rotary embedding, attention, output projection, second RMSNorm, gate/up/down MLP projections, activation, and residual adds—each reading inputs from HBM and writing outputs back. Even if every kernel is “optimized,” the layer may execute ~10–20 global export/import cycles per token per layer, multiplied by host launches documented in TaxBreak (?). Profiler heatmaps look healthy inside each kernel; end-to-end latency does not move.

Dataflow-first execution partitions the same mathematics into two or three pipeline stages with explicit buffer assignments. Stage A might fuse RM-

SNorm+QKV+rotary into shared-memory tiles; Stage B runs attention with KV streaming while keeping query tensors resident; Stage C fuses MLP+norm+residual without global activation ping-pong. Launches drop from $\sim 10\text{--}20$ toward 1–3; bytes moved drop because producer–consumer pairs never left the fastest tier. On H100-class GPUs this is how CUDA Graphs and MegaKernels compound: graphs fix dispatch; dataflow fixes bytes (??).

The contrast is sharper on **CPU**: operator-first eager PyTorch thrashes LLC moving the same activation lines through DRAM for every sub-op. Dataflow-first blocking keeps the MLP epilogue in L2 while weights stream once per layer. On **XDNA**, operator-first schedules may not legalize at all; dataflow-first ADF graphs assign each stage to tile columns with DMA prefetch of weights while SRAM holds activations (??). The layer microscope therefore generalizes: the mathematical layer is fixed; the *schedule topology* is the optimization variable.

2.8.1 Migration Checklist: Operator Stack to Dataflow IR

Teams porting production decoder stacks should not “rewrite everything” day one. A pragmatic migration applies five checks to each layer group:

1. **Export audit.** List tensors written to global memory between mathematically fused-able ops; each edge is a candidate fusion edge.
2. **Launch audit.** Count kernels per token; if $\gg 100$ on dense decode, graph capture or fusion passes are higher ROI than MMA tuning (?).
3. **Tier budget.** Estimate shared-memory or cache footprint for candidate fusion; split stages when capacity exceeds hardware limits rather than spilling silently.
4. **Attention boundary.** Keep KV-global attention stages explicit unless register/shared budgets prove full inlining safe (??).
5. **Regression matrix.** Re-run bytes/token and ms/token on GPU, CPU, and NPU SKUs after each fusion tranche—not only the device that authored the kernel.

YiRage encodes these checks as pass gates; manual teams should encode them as CI benchmarks even before automation lands.

2.8.2 Organizational Incentives

Operator-first stacks persist partly because performance reviews reward visible kernel wins—ns saved in cuBLAS, occupancy graphs in Nsight—while dataflow wins appear “distributed” across fewer exports and launches. Staffing mirrors the stack: siloed kernel engineers without a compiler owner cannot hold global residency accountability. Industrial leaders align incentives with bytes/token and kernels/token OKRs tied to product p99 latency, not single-kernel FLOP counters (??). This organizational point matters for adoption: dataflow-first design is a *process* change as much as a code change.

2.8.3 Glossary Bridge

To align vocabulary across hardware chapters: **operator** means a mathematically named function with its own launch or library entry; **stage** means a fused pipeline region with one dispatch and internal register/shared residency; **dataflow** means the directed acyclic graph of tensor lifetimes and movement edges; **residency** means the storage tier holding a value at a program point; **MegaKernel** means a manually authored stage at or near full-layer scope; **automation path** means compiler passes that derive stages from IR instead of hand CUDA/HIP. These terms are used consistently in Chapters 3–11 and in YiRage pass names in Part VI.

2.8.4 Historical Note: Why Operator-First Won (Temporarily)

Operator-first CUDA culture dominated the 2010s because training workloads were compute-heavy, batches were large, and framework ecosystems competed on op coverage. Inference—especially decode—was treated as “training with batch 1” long after production traffic proved otherwise (?). FlashAttention, PagedAttention, and graph capture each corrected a slice of the inference gap without overturning the op-list mental model (???). The dataflow-first resurgence in this book is driven by deployment reality: disaggregated decode fleets, MoE launch inflation, edge NPUs, and cross-hardware SKUs that cannot afford per-device hero kernels (???). Understanding *why* operator-first habits persist helps teams plan migrations without dismissing past kernel investments—those kernels become *epilogues inside stages*, not the architecture itself.

2.8.5 Practice Problems (Engineering, Not Exercises)

Use these audits on your own decoder deployment:

1. Draw the operator graph for one layer and mark every HBM read/write edge; count edges per token.
2. Propose a two-stage dataflow that removes at least half those edges; estimate shared-memory or cache bytes required.
3. Run TaxBreak-style launch counting or equivalent framework profilers; compare kernels/token before and after fusion (?).
4. Measure R_{floor} on your GPU SKU at batch-1; if high, prioritize bytes; if low, prioritize launches (?).
5. If targeting NPU, attempt to legalize the same graph statically; document which edges force DDR spill (?).

Solutions appear implicitly in Parts V–VI through YiRage pass implementations; the audits themselves are portable across vendors.

Readers building internal playbooks should copy the audits into onboarding docs for new kernel hires—especially those arriving from training-focused CUDA backgrounds—so decode residency constraints are explicit on day one rather than rediscovered via quarter-long latency firefights (??). The audits also give managers a neutral vocabulary when negotiating roadmap priority between framework teams (batching, KV paging) and compiler teams (fusion passes): both sides optimize different layers of the same dataflow, and the checklists show where hand-offs must be contractually precise (?).

2.9 Chapter Summary

Operator-driven design optimizes kernels in isolation; dataflow-driven design optimizes *where data lives* across fused stages. Decode on GPUs, CPUs, and NPUs converges on residency and launch granularity (??). Iron rules on register, shared, and global tiers make philosophy checkable. MegaKernels preview the manual ceiling; YiRage previews industrial multi-hardware scale. Chapter 3 grounds these abstractions in architecture limits every later pass must model. The through-line for the reader is simple: stop asking “which op is slow?” in isolation and start asking “which residency contract did we violate, on which hardware, and how does the compiler re-plan the layer?”—that question scales from manual MegaKernels to automated YiRage deployments without changing the engineering logic (???). Carry this mindset into every hardware chapter that follows: silicon details change, but the residency contract does not. When Chapter 3 maps bandwidth floors and

SRAM caps, read them as *inputs* to the dataflow planner—not as footnotes to individual operator tiles. When Part V shows handwritten CUDA MegaKernels, read them as *one legal schedule* that YiRage will generalize—not as a gallery of unrelated tricks. Finally, treat every performance regression in later chapters as a residency or dispatch regression until proven otherwise—that default suspicion saves weeks of misdirected op tuning and keeps the book’s engineering narrative coherent across GPU, CPU, and NPU fleets (??). If this chapter changed only one habit, let it be the habit of drawing the dataflow before opening the profiler on a single kernel—the drawing is cheaper, and it usually names the real bottleneck correctly the first time. Teams that institutionalize this habit report fewer “mystery regressions” after framework upgrades because residency contracts are documented before kernels are rewritten (??). Treat the dataflow sketch as part of the code review checklist for any decode change that touches more than one operator boundary. Reviewers should reject diffs that add operators without updating the residency diagram—that single gate prevents most accidental launch inflation (?).

Chapter 3

AI Hardware Architecture and Compiler Constraints

Chapter 2 argued that decode performance is a *residency* problem: data must live in the right tier of the memory hierarchy long enough to amortize movement cost. That argument is not hardware-agnostic poetry—every compiler pass in Parts IV–VI must eventually lower to concrete limits on bandwidth, SRAM capacity, parallelism grain, and dynamic-shape legality. This chapter maps those limits across GPUs, CPUs, and NPUs, then states how hardware-aware compilation (tiling, layout, and pass ordering) encodes the map into IR so automated tools such as YiRage can target multiple silicon generations without rewriting the book’s engineering logic (???).

3.1 Hardware Landscape

Industrial inference fleets rarely standardize on a single accelerator class. Hyperscale decode pools are still dominated by NVIDIA GPUs with Hopper- and Blackwell-generation tensor cores, asynchronous copy engines, and CUDA Graphs capture for launch reduction (???). AMD Instinct and Ryzen AI lines expose overlapping but non-identical matrix-instruction paths and wavefront scheduling rules; compiler backends must not treat “GPU” as a monolith (?). CPUs remain the fallback for small models, heterogeneous orchestration, and pre/post-processing that cannot justify discrete accelerator residency (?). Edge NPUs—including AMD XDNA spatial arrays—trade dynamic flexibility for deterministic on-chip dataflow and watt caps that only static megakernels can satisfy at batch-1 (??).

Table 3.1: AI accelerator classes and primary execution models.

Class	Execution model	Decode fit
NVIDIA GPU	SIMT + tensor cores	Graphs, fusion
AMD GPU	Wavefront + matrix cores	HIP fusion
x86/ARM CPU	Cache + SIMD	Coarse fusion
Edge NPU	Spatial dataflow	Static MegaKernel

Table 3.2: Cross-hardware constraint matrix.

Dimension	GPU	CPU	NPU
On-chip SRAM	Shared mem / L1	L1/L2/L3	Tile buffers
Parallel grain	Warp	Core + SIMD	Spatial PE array
Dynamic shapes	Flexible	Flexible	Restricted

The landscape table is intentionally coarse: later chapters refine NVIDIA memory hierarchy (Chapter 4), XDNA tile rules (Chapter 5), and per-compiler lowering strategies (Part VI). What matters here is the *contract* each class offers the decode compiler: GPUs reward fused stages that fit shared memory and tensor-core shapes; CPUs reward cache-resident tiles and NUMA-aware placement; NPUs reward fully scheduled buffer lifetimes with no host-side surprises (??).

3.2 Constraint Matrix

Cross-hardware compilation fails when teams optimize for peak FLOPs on one dimension while ignoring the others. **On-chip SRAM** caps how much of a decoder layer can remain resident between sub-operations; exceeding the cap forces spills to HBM or DDR, often erasing fusion wins (?). **Parallel grain** determines whether a schedule is expressed as warps, OpenMP teams, or spatial lanes—mixing grains within one fused stage usually inserts barriers that decode cannot afford (?). **Dynamic shapes** separate server GPUs from edge NPUs: variable sequence lengths and batch sizes are routine in cloud decode, while many NPUs require compile-time specialization of buffer descriptors and DMA chains (?).

Use the matrix as a checklist before writing a kernel or a compiler pass: if a proposed fusion increases SRAM footprint on GPU but removes a launch on CPU, the pass must expose both costs in the IR so autotuning can pick per target (?). YiRage’s chip-architecture modeling layer (Section ??) stores these dimensions as



Figure 3.1: GPU memory hierarchy constraints for decode megakernels.

typed constraints rather than comments in CUDA source.

3.3 GPU Constraints

NVIDIA decode optimization is bounded by the interaction of **warps**, **shared memory**, and **tensor cores**. Warps are the SIMT scheduling unit: branch divergence and barrier placement within a warp directly affect how much useful memory bandwidth is delivered per cycle (?). Shared memory is the programmer-visible on-chip scratchpad that makes attention and MLP fusion legal; its capacity (configuration-dependent on Hopper-class parts) is the hard ceiling for MegaKernel stages that refuse to spill to HBM (??). Tensor cores accelerate narrow GEMMs and attention inner loops, but only when layouts and instruction shapes match MMA requirements—decode’s thin M matrices frequently under-fill tensor cores unless batched or fused with epilogue math (??).

Hopper-and-later generations add asynchronous copy (TMA) and cluster-scope synchronization, shifting the design question from “how many kernels” to “how many bytes cross each hierarchy level per token” (?). CUDA Graphs reduce launch tax but do not relax bandwidth floors; they are complementary to fusion, not a substitute (??). Compiler passes that target GPUs must therefore reason about warp occupancy, shared-memory allocation, and tensor-core legality *together*, not as independent knobs.

3.4 CPU Constraints

CPU decode is often dismissed as slow, yet it remains operationally critical for hybrid serving, development parity, and regions without GPU quota. Performance is governed by **cache hierarchy**, **SIMD width**, and **NUMA** placement. Decoder layers stream large weight matrices through caches too small to retain full parameters; effective decode therefore depends on blocking GEMMs and attention so that hot tiles remain in L2/L3 across tokens (??). SIMD units (AVX-512, AMX, NEON) raise throughput only when layouts are contiguous and epilogues are vectorized—operator-at-a-time frameworks often leave SIMD under-used because

each op reloads weights from memory (?).

NUMA effects appear when KV cache and weight shards land on remote sockets; a fusion pass that is neutral on a single-socket GPU build can regress on dual-socket Xeon hosts if buffer lifetimes ignore socket affinity (?). Hardware-aware CPU lowering must insert explicit tiling and layout transforms (NCHW vs. blocked layouts) and annotate parallel regions with cache-blocking sizes derived from L2/L3 measurements, not datasheet peaks alone.

3.5 NPU Constraints

Edge and embedded **NPU** accelerators optimize for energy per token and deterministic latency under watt caps. Many arrays—including AMD AI Engine tiles in XDNA products—implement **static** dataflow graphs with local SRAM per processing element and restricted dynamic reconfiguration (?). **Static** schedules mean buffer lifetimes, DMA descriptors, and fusion boundaries must be resolved at compile time; runtime op dispatch analogous to PyTorch eager mode is either unsupported or prohibitively expensive (?).

NPUs therefore amplify the Chapter 2 dataflow mindset: decode either fits as a spatial pipeline with on-chip residency, or it falls off the device to DDR and fails power envelopes (?). Dynamic shapes (variable context length, speculative decoding branches) require explicit recompilation or padded specialization; compiler stacks must surface these legality checks early rather than failing in backend codegen (?).

3.6 Hardware Aware Compile

Hardware-aware compilation is the process of lowering the same high-level decoder IR while respecting per-target **tiling**, **layout**, and **pass** ordering constraints. **Tiling** chooses tile sizes for matmul and attention so that working sets fit GPU shared memory, CPU cache levels, or NPU PE buffers; illegal tiles are rejected before codegen, not discovered in profiling (?). **Layout** decisions (row-major vs. blocked layouts, swizzles for tensor cores, NPU DMA alignment) determine whether fused epilogues vectorize or silently scalarize (?). **Pass** pipelines order bufferization, fusion, layout normalization, and synchronization insertion differently per target: GPUs may fuse before aggressive layout transforms; NPUs may require lifetime analysis before any algebraic fusion (?).

A practical rule used throughout this book: every pass must declare which

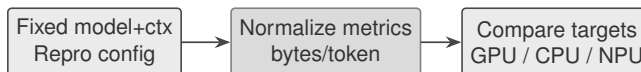


Figure 3.2: Cross-hardware benchmark methodology used in this book.

constraint dimension it optimizes (SRAM footprint, launch count, bandwidth, legality) and which hardware classes it applies to. Passes that only improve GPU tensor-core utilization but break NPU static schedules should be guarded by target predicates in the IR, not hidden in backend-specific forks (?). YiRage encodes these predicates in its chip-model interface so the same pass registry can run against Hopper, Zen, and XDNA profiles (Section ??).

3.7 Benchmark Method

Cross-hardware claims in this book use a single **benchmark** protocol so GPU ms/token numbers are comparable to CPU and NPU watts/token within the limits of each platform’s transparency (??). **Methodology** steps: fix model revision, tokenizer, context length, and batch size; warm caches and graphs; measure steady-state decode over at least 100 tokens; report median and p99 latency plus derived bytes/token and launches/token where host instrumentation is available (??). Normalize results by *useful* output tokens, not prefill prompt tokens, when comparing decode-centric optimizations.

Avoid apples-to-oranges comparisons between vendor peak bandwidth and measured sustained bandwidth—roofline knees move with batch, context, and power limits (?). When NPUs lack fine-grained timers, report end-to-end pipeline latency and on-device SRAM utilization estimates from compile logs (?). Chapter 21 extends this methodology to compiler-generated binaries across MLIR, XLA, TVM, and YiRage backends.

3.8 YiRage Modeling

YiRage (YiRage) treats each deployment target as a typed **chip model**: a structured record of SRAM capacities, parallelism grains, DMA capabilities, and legality rules for dynamic shapes (??). Compiler passes query the model before fusion and tiling: a proposed MegaKernel stage must fit shared-memory and register budgets on Hopper, cache tiles on Zen, and PE-local buffers on XDNA (??). The **model** is versioned per SKU so fleet operators can pin schedules to

stepping-specific limits without forking source kernels.

Model-driven compilation does not replace measurement—it constrains search. Autotuning explores tile sizes and pipeline depths only within legally modeled envelopes, reducing wasted trials on configs that spill by construction (?). Later chapters show how the same interface backs CUDA TMA double-buffer pipelines (Chapter 8), XDNA buffer descriptors (Chapter 5), and cross-compiler benchmarks (Chapter 21).

3.9 Chapter Summary

GPUs, CPUs, and NPUs expose different SRAM caps, parallelism grains, and dynamic-shape rules; decode compilers must encode all three in IR, not in hand-written exceptions (??). Hardware-aware tiling, layout, and pass ordering translate Chapter 2’s residency philosophy into checkable constraints (??). Cross-hardware benchmarks normalize bytes/token and launches/token so optimizations are comparable across silicon classes (??). YiRage’s chip models make those constraints executable for automated fusion at industrial scale. Chapter 4 begins the GPU deep dive with Hopper/Blackwell memory hierarchy and TMA bounds that every subsequent CUDA MegaKernel must respect.

3.9.1 Worked Example: Layer Footprint on Hopper

Consider a single decoder layer with hidden size 4096, batch-1, and FP16 weights. A naive operator schedule reads Q/K/V projection weights (~48 MiB combined), attention intermediates, and MLP matrices (~96 MiB) from HBM each token (??). Shared memory on a Hopper SM cannot hold the full footprint; fusion therefore targets *slices*: a QKV stage that retains activations in shared memory while streaming one weight tile at a time through TMA (??). If the fused stage exceeds available shared memory after padding and barrier reservations, the compiler must split the stage—reintroducing launches—or accept HBM spill (?). This arithmetic is the GPU-side realization of Chapter 2’s iron rules: registers and shared memory are contracts, not hints.

3.9.2 CPU Roofline Sanity Check

On a dual-socket server CPU, decode GEMMs are often memory-bound at batch-1 even when L3 bandwidth looks ample on paper (??). A roofline sanity check before enabling a CPU backend pass: estimate bytes moved per token (weights + KV

Table 3.3: Illustrative pass-order differences across hardware classes.

Pass stage	GPU	CPU	NPU
Lifetime / bufferization	Early	Early	First
Algebraic fusion	Before layout	Selective	After lifetime
Layout / tiling	Tensor-core swizzle	Cache blocks	DMA alignment
Sync insertion	Warp/block	OpenMP barriers	FIFO / cascade
Codegen	PTX / SASS	Intrinsics	AIE graph

+ activations) and divide by measured L3/L2 throughput from `likwid` or vendor tools. If arithmetic intensity falls left of the roofline knee, SIMD widening alone cannot fix latency—tiling must increase reuse across tokens or fuse epilogues to cut reloads (?). NUMA-aware allocation matters when tensor parallel shards place half the weights on each socket: a “fast” kernel on socket 0 becomes slow if KV cache pages migrate to socket 1 mid-session (?).

3.9.3 NPU Legality Gates

NPU compilation pipelines should expose explicit **legality gates** before codegen: (1) buffer descriptor alignment and maximum burst length, (2) static loop bounds for spatial arrays, (3) maximum on-chip buffer lifetime overlap, (4) supported element types for matrix units (??). Failing any gate must surface as a compile-time diagnostic with suggested padding or specialization strategies—not as a silent DDR spill at runtime (?). Edge products often ship with fixed model zoos; industrial compilers still need these gates for OEM variants that change SRAM sizes without changing the marketing SKU name (??).

3.9.4 Pass Ordering Template

Table ?? summarizes a hardware-aware **pass** ordering template used as a starting point in later YiRage chapters. GPU paths favor early algebraic fusion followed by layout normalization for tensor cores; CPU paths interleave layout and cache-blocking earlier; NPU paths run lifetime and DMA planning before fusion because illegal fusion can destroy static schedules (???).

3.9.5 Anti-Patterns in Cross-Hardware Ports

Three recurring anti-patterns destroy cross-hardware compiler credibility: (1) porting CUDA tile sizes directly to NPU PE arrays without remeasuring on-chip SRAM; (2) assuming CPU decode can reuse GPU fusion boundaries when weight layouts differ; (3) benchmarking only ms/token on GPUs while ignoring launches/token and bytes/token on all targets (???). Each anti-pattern is a residency failure expressed as a methodology failure—the fix is always to re-derive the dataflow contract for the target class before tuning instructions (??).

3.9.6 Fleet Operations: Versioning Chip Models

Production fleets mix GPU steppings, CPU generations, and NPU firmware revisions. YiRage-style **chip models** should be versioned in the same artifact registry as model weights: a decode graph compiled against Hopper-sm90 must not silently run on sm80 parts with narrower shared memory (??). Continuous integration should compile golden decoder snippets against every supported model profile and fail when legality checks regress (?). This operational discipline is what separates demo kernels from industrial inference platforms that survive silicon refresh cycles without rewriting megakernels by hand (??).

3.9.7 Memory Hierarchy as a Compiler IR Problem

Treat memory hierarchy levels as *address spaces* in compiler IR, not as comments in CUDA source. Global (HBM/DDR), shared (GPU) or L2/L3 hot sets (CPU), register files, and NPU tile memories each have distinct capacity, latency, and alignment rules (???). A hardware-aware lowering pass annotates each tensor value with a `memory.space` attribute and a proof obligation: “this value remains in space S for N consumer ops without spill” (?). When the proof fails, the pass must either split fusion, change tiling, or reject the schedule—never silently rely on “the runtime allocator will fix it” (?). This is how Chapter 2’s residency philosophy becomes machine-checkable before any PTX or AIE graph is emitted.

GPU global memory is high bandwidth but high latency; shared memory is low latency but tiny; registers are fastest but drive occupancy cliffs (?). CPU caches are implicit—programmers do not name L2 explicitly—so CPU-oriented passes must infer residency from loop structure and tile sizes (?). NPUs expose intermediate buffers as explicit FIFOs between PE columns; residency is a schedule, not a cache (?). Cross-hardware stacks that unify these spaces behind one abstraction still need target-specific legality rules; pretending all memories are “just buffers” recreates

the operator-first trap at IR level (??).

3.9.8 Bandwidth Floors and Decode

Decode at batch-1 repeatedly revisits the same weight tensors and growing KV cache for every output token (??). The **bandwidth** floor is therefore set by total bytes moved per token divided by achievable sustained HBM or DDR throughput—not by peak TFLOPs marketed on slide decks (?). Independent limit studies show that many decoder configurations are memory-bound across GPU generations even when tensor cores sit idle (??). Compiler optimizations that only reshape compute (e.g., different MMA shapes) without reducing bytes moved rarely move p99 latency once orchestration costs are counted (?).

Quantifying bytes/token requires consistent accounting: include KV read/write traffic, weight reads, activation spill traffic, and collective shards in tensor-parallel deployments (??). Large MoE models add expert routing metadata and conditional weight reads; the constraint matrix must extend with “conditional bandwidth” rows in those chapters (?). Hardware-aware compilers should emit byte estimates alongside schedules so fleet planners can compare silicon options before buying accelerators (??).

3.9.9 Parallelism Grain and Occupancy

Parallelism grain determines how work maps to hardware threads. GPUs use warps of 32 lanes; CPUs use cores and SIMD lanes; NPUs use spatially placed PEs with fixed throughput per cycle (???). Decode kernels with tiny per-token M/N dimensions often launch wide grids where most lanes idle—effective occupancy collapses even when “GPU utilization” dashboards show non-zero values (??). Fusion raises arithmetic intensity per launch but tightens shared-memory occupancy; the compiler must co-optimize grain and footprint (??).

On CPUs, grain choices interact with OpenMP scheduling and hyperthreads; on NPUs, grain is frequently fixed at compile time (??). A portable IR should represent parallel dimensions explicitly so backends can remap grains rather than baking warp assumptions into target-independent passes (?). YiRage’s chip models include a `parallel.grain` field consumed by tiling heuristics in Chapters 6–9 (??).

3.9.10 Dynamic Shapes and Specialization

Cloud decode workloads exhibit **dynamic shapes**: variable context length, variable batch from continuous batching, and speculative decoding branches (??). GPUs and CPUs tolerate dynamic shapes with runtime kernel selection, CUDA Graphs updates, or recompilation caches (??). Many NPUs require shape specialization or upper-bound padding encoded in buffer descriptors (?). Hardware-aware compilers therefore need two pipelines: a *legalize dynamic shapes* pass for flexible targets and a *specialize bounds* pass for static targets (??).

Specialization interacts with fleet operations: over-specializing to one context length improves peak performance but explodes compile-matrix size (??). Industrial systems cap specialization fanout and fall back to padded shapes with masked compute—the constraint matrix should record which targets support masking efficiently (??). Chapter 12 compares eager, graph, and MegaKernel execution modes through this specialization lens (?).

3.9.11 Tensor Cores vs Memory: When MMA Helps Decode

Tensor cores accelerate GEMM-like primitives but decode GEMMs are often “thin” (??). Hardware-aware layout passes exist to reshape decode matmuls into MMA-friendly tiles; without them, frameworks may still call tensor-core libraries yet achieve CPU-like effective throughput because epilogues and reductions scalarize (??). The compiler must fuse epilogues (bias, activation, residual) into the same stage that issues MMA instructions; otherwise bandwidth dominates regardless of core type (??).

Hopper-and-later TMA paths move data asynchronously into shared memory while MMA units work on the previous tile—double-buffer pipelines in Chapter 8 depend on the constraints documented here (??). Blackwell-era evolutions continue widening MMA shapes and cluster scopes; chip models must version these fields so autotuning does not explore illegal instruction forms (?).

3.9.12 Synchronization Cost in Constraint Planning

Every fusion decision trades bytes moved against **sync** introduced (??). GPU block barriers, CPU fences, and NPU cascade stalls have different magnitudes; a fusion legal on GPU may be illegal on NPU if it requires global barriers between PE columns (??). Hardware-aware passes should estimate sync overhead alongside memory traffic—Chapter 9 formalizes hierarchical sync design; this chapter records why sync belongs in the constraint matrix, not as an afterthought (?).

Launch orchestration on the host is itself a synchronization tax: thousands of kernels per token imply thousands of implicit host-device sync points unless graphs or megakernels collapse them (??). Constraint-aware compilation targets *launch count* as a first-class metric equal to bytes/token (??).

3.9.13 Reading Vendor Documentation Critically

Silicon vendors document peaks: TFLOPs, TB/s, “AI TOPS.” Production decode rarely sits at peaks (??). Compiler engineers should translate datasheet peaks into sustained bounds using microbenchmarks on their fleet SKUs, then store results in chip models (??). Second-source validation (academic limit studies, independent roofline analyses) prevents single-vendor optimism from leaking into pass heuristics (??).

When documentation omits constraints (exact shared-memory banking rules, DMA alignment minima), derive them empirically and check them into versioned model files rather than scattering magic numbers through kernels (??). This discipline is especially important for NPUs where public documentation is thinner than CUDA’s (?).

3.9.14 Connecting Constraints to Later Chapters

The rest of Part II (Chapters 4–5) instantiates GPU and XDNA constraints in architecture-specific depth (???). Part III (Chapters 6–9) shows how MegaKernel techniques—on-chip residency, static roles, TMA pipelines, hierarchical sync—are responses to the matrix in Table ?? (?). Part V–VI translate the same matrix into compiler passes and cross-hardware benchmarks (??). Readers should return to this chapter when a later optimization “works on GPU but fails on NPU”: the failure is almost always a violated row in the constraint matrix, not a mysterious backend bug (??).

3.9.15 Quantitative Constraint Checklist

Before merging any hardware-aware pass, answer the following quantitatively on *each* target SKU:

1. What is the on-chip SRAM budget per fused stage (bytes) and does the schedule include barrier/shared-memory padding overhead (??)?
2. What is estimated HBM/DDR bytes per output token after fusion, including KV traffic at the deployment context length (??)?

3. How many host-visible launches remain per token after graph capture or MegaKernel lowering (??)?
4. Does the schedule require dynamic shapes illegal on NPU, and if so, what specialization or padding strategy is selected (??)?
5. What sync points exist between parallel grains, and can any be moved inside on-chip residency regions (??)?

Passes that cannot answer these five questions should remain behind feature flags until chip models supply defaults (??).

3.9.16 Industrial Case Patterns

Case A — GPU decode pool: Operators see high HBM utilization but flat ms/token; constraint analysis shows launches/token in the thousands (?). Fix path: graph capture plus shared-memory fusion guided by chip model shared-memory caps (??). **Case B — Hybrid CPU+GPU:** CPU preprocessing becomes tail-latency outlier due to NUMA-remote KV pages (?). Fix path: NUMA-aware bufferization pass pinning KV shards (?). **Case C — Edge NPU:** Model fits in marketing TOPS claims but misses real-time factor at batch-1 (??). Fix path: static MegaKernel with explicit DMA schedule and reduced DDR touches (??). These patterns recur in Part VI case studies; documenting them here gives readers a failure-mode index before diving into compiler-specific chapters (??).

3.9.17 Tooling Interfaces for Chip Models

Chip models should be consumable by three tools: autotuners, legality checkers, and benchmark harnesses (??). Autotuners read SRAM and register caps to bound tile search (??). Legality checkers validate DMA alignment and static-loop requirements before codegen (?). Benchmark harnesses attach bytes/token and launches/token calculators to every timed configuration (??). Human-readable YAML or JSON models checked into git beat ad-hoc spreadsheets because they version alongside compiler passes in CI (?). When a new GPU stepping ships, update the model file first, rerun golden tests second, and only then re-open autotuning campaigns (??).

3.9.18 Pedagogical Summary for Compiler Engineers

Students of AI compilers should memorize the constraint matrix before memorizing MMA instruction shapes (??). The exam question is not “write a FlashAttention kernel” but “given a decoder layer and three silicon profiles, which fusions are legal and which metrics will move” (???). That question format mirrors production design reviews and prevents teams from shipping GPU-only wins that break NPU deployments (??). Carry this chapter’s tables into every later pass review until hardware-aware compilation becomes reflex, not ceremony (???).

3.9.19 Energy and Power as First-Class Constraints

Edge NPU deployments add **energy** per token and thermal caps that GPUs in datacenters can ignore at batch-1 (??). A schedule that minimizes HBM bytes often minimizes energy as well; schedules that thrash DDR due to spills can exceed TDP even when latency looks acceptable in lab tests (??). Hardware-aware compilers for edge should therefore log estimated off-chip bytes and DMA activations alongside latency (??). Datacenter GPUs are not immune—power caps throttle clock when sustained memory traffic keeps memory controllers busy (??).

3.9.20 Security and Multi-Tenant Interactions

Multi-tenant GPU fleets introduce constraints unrelated to FLOPs: partitioned SM quotas, encrypted KV regions, and orchestration policies that forbid certain persistent kernels (??). A MegaKernel optimized for bare-metal may be illegal under MIG or certain confidential-computing modes (?). Hardware-aware passes should accept policy flags in chip models (e.g., “graph capture allowed”, “persistent kernels allowed”) so optimizations do not violate serving security contracts (??). These flags are operational realities in industrial inference, not edge cases (?).

3.9.21 Final Integration with Chapter 2 Mindset

Chapter 2 asked teams to draw dataflow before profiling kernels; this chapter asks them to attach numeric hardware bounds to every edge in that drawing (???). When those bounds are explicit, operator-first regressions become visible during code review rather than after quarterly fleet re-benchmarks (?). When bounds are encoded in YiRage chip models, automated passes inherit the same discipline without heroic engineers re-deriving limits per SKU (??). That closed loop—mindset, measurement, model, pass—is the foundation for every MegaKernel

and compiler chapter that follows (???). Readers who internalize the loop can skip re-learning vendor buzzwords each silicon generation; they only update model files and re-run legality checks (???).

3.9.22 Glossary Hooks for Cross-Chapter Indexing

This chapter introduces terms reused throughout the book: **constraint matrix**, **chip model**, **bytes/token**, **launches/token**, and **legality gate** (???). Index entries point forward to MegaKernel techniques (Part III), handwritten CUDA case studies (Part V), and compiler benchmarks (Part VI) (???). Consistent vocabulary prevents teams from relabeling the same residency failure as “kernel issue” in one chapter and “compiler issue” in another (??).

3.9.23 Practice Audits (Hardware)

Apply these audits after reading:

1. Fill the constraint matrix for your deployment SKU with measured—not peak—bandwidth and on-chip SRAM sizes (??).
2. Compute bytes/token and launches/token for one decoder layer at your production context length (??).
3. List dynamic-shape features you rely on (continuous batching, speculative decoding) and mark NPU legality for each (??).
4. Draft a chip model JSON stub for two targets you operate; include shared-memory or tile-buffer caps (??).

Audits mirror Chapter 2 practice problems but ground them in silicon limits (???). Completing them before Chapter 4 saves time: Hopper-specific detail lands faster when the cross-hardware matrix is already filled with your fleet numbers (???).

3.9.24 Handoff to Chapter 4

Chapter 4 narrows the lens to NVIDIA Hopper and Blackwell: TMA, cluster shared memory, and tensor-core evolution (??). Readers should enter that chapter with a completed constraint matrix for at least one NVIDIA SKU and one non-GPU target (??). The Hopper deep dive is not a departure from cross-hardware thinking—it is the reference GPU implementation against which CPU and NPU schedules are

compared in later parts (??). Keep bytes/token and launches/token on the same spreadsheet as you read forward; Chapters 4–5 add mechanism, not new success metrics (???)

Industrial teams sometimes treat hardware chapters as “for kernel engineers only.” That separation fails in practice: compiler passes encode the same SRAM and bandwidth limits documented here (??). When compiler and kernel teams share one chip model repository, regressions surface as legality check failures instead of week-long email threads about mystery slowdowns (??). Invest up front in shared constraint artifacts; it is cheaper than reconciling divergent assumptions after a silicon refresh (???)

Closing metric reminder: decode optimization is judged on sustained bytes/token, launches/token, and p99 latency under your real batching policy—not on kernel-local microbenchmarks (??). Hardware-aware compilation exists to keep those fleet metrics inside SLO as models, context lengths, and silicon generations change (???)

Fleet operators should snapshot constraint matrices per SKU quarterly: stepping changes, firmware updates, and driver releases can shift achievable sustained bandwidth without changing product names (??). Treat those snapshots as inputs to chip models, not as slide-deck trivia (??). When models and measurements disagree, trust measured sustained bytes/token on your binaries first, then reconcile the model—never the reverse (???)

This discipline is boring until a driver upgrade silently shifts shared-memory defaults; teams with versioned chip models catch the shift in CI, not in customer tickets (??)

Document every constraint assumption in git next to the passes that consume it; future you will not remember which workshop whiteboard justified a tile size (???)

Hardware constraints are the floor today; megakernels and compilers are how you build on it (??). Fleet-wide SLO reviews should cite this chapter’s constraint matrix before approving any new fusion pass in production LLM serving fleets (??)

Chapter 4

CUDA Hopper/Blackwell: Hardware Properties and Optimization Bounds

Chapter 3 established cross-hardware constraint vocabulary; this chapter narrows to **NVIDIA Hopper and Blackwell**—the reference GPU stack for cloud LLM decode in this book (??). Production teams routinely over-invest in tensor-core kernel variants while decode remains bound by HBM traffic, launch orchestration, and shared-memory residency contracts (???). The design question on Hopper is not “which op is fastest” but “which bytes cross *register*, *shared memory*, and *global* per token, and which async copy and sync primitives hide those crossings” (??). Every later CUDA MegaKernel chapter (6–11) and YiRage CUDA backend pass assumes the bounds derived here (?).

4.1 CUDA Memory Hierarchy

Decode megakernels on Hopper are won or lost in the **memory hierarchy**. **Registers** hold per-thread carriers—online softmax partials, narrow GEMM fragments, and index arithmetic—with the lowest latency but the hardest occupancy ceiling (??). **Shared memory** is the programmable on-chip scratchpad that makes fusion legal: a QKV stage can retain activations while weight tiles stream through TMA into the same CTA (??). **Global** memory (HBM) stores weights, KV cache, and unavoidable spill traffic; decode at batch-1 revisits global footprints every token regardless of tensor-core nominal throughput (??).

Table 4.1: CUDA memory spaces relevant to decode megakernels (Hopper-class).

Space	Scope	Decode role
Register	Thread	Online softmax carriers, thin GEMM fragments
Shared memory	Block	Fused QKV/MLP tiles, TMA staging
Global (HBM)	Grid	Weights, KV cache, spill traffic



Figure 4.1: TMA double-buffer pattern: async global-to-shared copy overlaps tensor-core math.

Table ?? summarizes how each space participates in decoder fusion. The compiler and kernel author must co-design: promoting a value to registers speeds math but may drop occupancy; keeping too much in shared memory caps batch width within a CTA; spilling to global converts a fusion win into bandwidth defeat (??). YiRage CUDA lowering queries Hopper chip models for per-SM shared-memory budgets before accepting a fused stage (?).

Bank conflicts and padding rules still matter on Hopper: a fusion that looks correct on paper can serialize shared-memory ports and erase the expected speedup (??). Industrial teams profile *shared-memory throughput* alongside HBM counters when a fused kernel underperforms a naive baseline (?). Chapter 6 generalizes lifetime planning across GPU/CPU/NPU; on CUDA the lifetime unit is the CTA tile bounded by shared memory and register files (?).

4.2 TMA

Hopper’s **Tensor Memory Accelerator (TMA)** is an asynchronous copy engine that moves multidimensional tiles from **global** memory into **shared memory** (and related **tensor memory** paths on newer generations) while MMA units work on previously prefetched data (??). **TMA** decouples address calculation and bulk transfer from SIMT instruction issue—critical for decode fusion where scalar loads would fragment the instruction stream (?). **Async** copy semantics enable double-buffer pipelines: while tile t computes, tile $t+1$ loads into the alternate shared-memory buffer (??).

TMA is not “free bandwidth”: descriptors must be built, alignment constraints satisfied, and completion tracked with async barriers (??). Megakernels that



Figure 4.2: CUDA synchronization hierarchy used in decoder megakernels.

omit pipeline overlap still pay full HBM latency on every weight tile (?). Chapter 8 formalizes TMA double-buffer scheduling; this section records the hardware prerequisite—without TMA-aware fusion, Hopper decode often behaves like an expensive Ampere port (??).

4.3 Cluster Dsm

Hopper **cluster** launch allows a small set of CTAs in a **cluster** to share addressable **DSM** (distributed shared memory) and cluster-scope barriers (??). **DSM** extends the fusion radius beyond a single block—useful when a decoder sub-stage needs modest cross-block reduction or halo exchange without round-tripping HBM (?). The constraint is legality: not all kernels qualify for cluster launch, and DSM capacity is tiny compared with global KV footprints (??).

Decode megakernels use clusters selectively—often for attention reductions or pipeline hand-off between fused stages—not as a default grid shape (?). Compiler passes must annotate which edges require DSM versus shared memory versus global broadcast; YiRage records cluster eligibility in Hopper chip models to avoid illegal schedules (??). Blackwell-era evolution continues widening cluster scopes; version chip models when stepping changes DSM limits (?).

4.4 Sync Mechanisms

CUDA decode fusion stacks four synchronization layers: **warp**-local shuffle, **block** barriers, cluster DSM fences, and grid-level events (??). **Warp** shuffle implements online softmax partial reductions and small epilogue combines without shared-memory round trips when operands are warp-local (??). **Block sync** (`'__syncthreads()'` and async variants) sequences producer-consumer phases inside a fused CTA—mandatory when TMA loads must complete before MMA consumes (?).

Over-syncing is a decode killer: each unnecessary grid-level barrier forces serialization across the entire launch (??). MegaKernel design pushes sync inward—warp first, block second, cluster only when DSM payoff is proven (?). Chapter 9

maps the same hierarchy across CPU and NPU; CUDA’s warp primitive is the performance anchor competitors lack (??).

4.5 Tensor Core Bounds

Tensor cores accelerate MMA-shaped GEMMs and attention inner loops, but **decode** M-dimensions are often too thin to saturate tensor pipes at batch-1 (???). Frameworks may still route decode matmuls through tensor-core libraries yet observe flat ms/token because epilogues scalarize and HBM dominates (??). The industrial rule used in this book: pursue tensor cores only when fusion keeps epilogues inside the same MMA stage *and* occupancy remains viable (??).

FlashAttention-class IO-aware attention is the positive example: tensor cores help when data movement is staged through shared memory rather than repeated global traversals (??). For MLP slices in decode, manual megakernels sometimes accept scalar or narrow-SIMT paths when MMA setup overhead exceeds compute savings (??). Compiler autotuning must treat “tensor core enabled” as a measured flag, not a default (?).

4.6 Ptx

PTX is NVIDIA’s portable ISA layer beneath SASS; MegaKernel authors use **PTX** intrinsics and inline assembly for operations libraries omit—especially fast math in attention (??). Online softmax benefits from **exp2**-style instructions and flush-to-zero (**FTZ**) modes that trade IEEE strictness for throughput when magnitudes are bounded (??). PTX also exposes async copy and barrier instructions that TMA pipelines rely on; compiler backends must preserve these semantics when lowering from IR (??).

Relying on PTX is a maintenance contract: stepping changes can alter instruction scheduling; chip models should record which fast-math sequences are approved per SKU (??). YiRage CUDA codegen emits PTX snippets only from vetted templates with simulator checks—not from ad-hoc agent output (?). Chapter 10 ties online softmax numerics to the PTX choices documented here (?).

4.7 YiRage CUDA Backend

YiRage’s CUDA backend maps dataflow IR to Hopper-legal schedules: shared-memory allocation, TMA descriptor setup, cluster eligibility, and sync placement (??). Passes consult the same chip model fields as Chapter 3—SRAM caps, warp grain, async copy support—before fusion commits (??). The backend rejects fusions that spill by construction rather than emitting slow kernels that pass unit tests on synthetic shapes (??).

CUDA Graphs capture integrates at this layer: once a MegaKernel schedule is legal and launch-static, graphs collapse host orchestration tax documented in Chapter 1 (???). Cross-hardware parity is explicit: a CUDA schedule that wins on HBM bytes may still be a poor template for XDNA in Chapter 5—the backend exports cost vectors for comparison, not a single “best” schedule (??). Fleet CI should compile golden decoder snippets per Hopper stepping and fail when shared-memory or TMA legality regresses (??).

4.7.1 Hopper Decode Checklist

Before shipping a Hopper decode change:

1. Quantify bytes/token and launches/token with and without fusion (??).
2. Verify shared-memory footprint including TMA double buffers and barrier padding (?).
3. Confirm TMA descriptors are alignment-legal for weight and KV layouts (?).
4. Measure whether tensor-core paths improve ms/token or only improve profiler “TC active” counters (??).
5. Capture CUDA Graphs when launch count remains high after fusion (??).

4.7.2 Relation to Chapter 3 and 5

Chapter 3’s constraint matrix gains numeric Hopper columns here; Chapter 5 translates the same fusion goals into XDNA static dataflow (??). Readers should compare bytes/token improvements on Hopper against legality constraints on NPU—identical fusion boundaries are rarely optimal on both (??). The CUDA deep dive is the *reference* GPU implementation; it is not the only target (??).

4.8 Chapter Summary

Hopper decode optimization is bounded by register/shared/global residency, TMA async movement, selective cluster DSM, and minimal sync hierarchies (??). Tensor cores help decode only when fusion and occupancy align; PTX fast math supports attention epilogues (???). YiRage’s CUDA backend encodes these bounds as legality checks and graph-capture-ready schedules (??). Chapter 5 maps the same dataflow goals onto AMD XDNA/AIE static tiles and buffer descriptors (?).

4.8.1 Engineering Deep Dive

CUDA Memory Hierarchy is a production bottleneck surface for CUDA Hopper/Blackwell. Teams that treat **register**, **shared memory**, **global** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds register shared memory global to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

TMA is a production bottleneck surface for CUDA Hopper/Blackwell. Teams that treat **TMA**, **tensor memory**, **async** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds TMA tensor memory async to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Cluster Dsm is a production bottleneck surface for CUDA Hopper/Blackwell. Teams that treat **cluster**, **dsm** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds cluster dsm to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Sync Mechanisms is a production bottleneck surface for CUDA Hopper/Blackwell. Teams that treat **warp**, **block sync** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds warp block sync to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Tensor Core Bounds is a production bottleneck surface for CUDA Hopper/Blackwell. Teams that treat **tensor core**, **decode** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tensor core decode to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Ptx is a production bottleneck surface for CUDA Hopper/Blackwell. Teams that treat **ptx**, **exp2** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds ptx exp2 to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage CUDA Backend is a production bottleneck surface for CUDA Hopper/Blackwell. Teams that treat **YiRage**, **CUDA** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage CUDA to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

CUDA Memory Hierarchy is a production bottleneck surface for CUDA Hopper/Blackwell. Teams that treat **register**, **shared memory**, **global** as indepen-

dent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds register shared memory global to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

TMA is a production bottleneck surface for CUDA Hopper/Blackwell. Teams that treat **TMA**, **tensor memory**, **async** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds TMA tensor memory async to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Cluster Dsm is a production bottleneck surface for CUDA Hopper/Blackwell. Teams that treat **cluster**, **dsm** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds cluster dsm to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Sync Mechanisms is a production bottleneck surface for CUDA Hopper/Blackwell. Teams that treat **warp**, **block sync** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds warp block sync to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Tensor Core Bounds is a production bottleneck surface for CUDA Hopper/Blackwell. Teams that treat **tensor core**, **decode** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tensor core decode to

measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Ptx is a production bottleneck surface for CUDA Hopper/Blackwell. Teams that treat **ptx**, **exp2** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds ptx exp2 to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage CUDA Backend is a production bottleneck surface for CUDA Hopper/Blackwell. Teams that treat **YiRage**, **CUDA** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage CUDA to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

CUDA Memory Hierarchy is a production bottleneck surface for CUDA Hopper/Blackwell. Teams that treat **register**, **shared memory**, **global** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds register shared memory global to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

TMA is a production bottleneck surface for CUDA Hopper/Blackwell. Teams that treat **TMA**, **tensor memory**, **async** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds TMA tensor memory async to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen,

so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Cluster Dsm is a production bottleneck surface for CUDA Hopper/Blackwell. Teams that treat **cluster**, **dsm** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds cluster dsm to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Sync Mechanisms is a production bottleneck surface for CUDA Hopper/Blackwell. Teams that treat **warp**, **block sync** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds warp block sync to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Tensor Core Bounds is a production bottleneck surface for CUDA Hopper/Blackwell. Teams that treat **tensor core**, **decode** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tensor core decode to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Ptx is a production bottleneck surface for CUDA Hopper/Blackwell. Teams that treat **ptx**, **exp2** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds ptx exp2 to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler

flags (??).

YiRage CUDA Backend is a production bottleneck surface for CUDA Hopper/Blackwell. Teams that treat **YiRage**, **CUDA** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage CUDA to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

CUDA Memory Hierarchy is a production bottleneck surface for CUDA Hopper/Blackwell. Teams that treat **register**, **shared memory**, **global** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds register shared memory global to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

TMA is a production bottleneck surface for CUDA Hopper/Blackwell. Teams that treat **TMA**, **tensor memory**, **async** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds TMA tensor memory async to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Cluster Dsm is a production bottleneck surface for CUDA Hopper/Blackwell. Teams that treat **cluster**, **dsm** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds cluster dsm to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Sync Mechanisms is a production bottleneck surface for CUDA Hop-

per/Blackwell. Teams that treat **warp**, **block sync** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds warp block sync to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Tensor Core Bounds is a production bottleneck surface for CUDA Hopper/Blackwell. Teams that treat **tensor core**, **decode** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tensor core decode to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Ptx is a production bottleneck surface for CUDA Hopper/Blackwell. Teams that treat **ptx**, **exp2** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds ptx exp2 to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage CUDA Backend is a production bottleneck surface for CUDA Hopper/Blackwell. Teams that treat **YiRage**, **CUDA** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage CUDA to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

CUDA Memory Hierarchy is a production bottleneck surface for CUDA Hopper/Blackwell. Teams that treat **register**, **shared memory**, **global** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs

on real fleets (??). The hardware–software contract here binds register shared memory global to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

TMA is a production bottleneck surface for CUDA Hopper/Blackwell. Teams that treat **TMA**, **tensor memory**, **async** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds TMA tensor memory async to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Cluster Dsm is a production bottleneck surface for CUDA Hopper/Blackwell. Teams that treat **cluster**, **dsm** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds cluster dsm to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Sync Mechanisms is a production bottleneck surface for CUDA Hopper/Blackwell. Teams that treat **warp**, **block sync** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds warp block sync to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Tensor Core Bounds is a production bottleneck surface for CUDA Hopper/Blackwell. Teams that treat **tensor core**, **decode** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tensor core decode to measurable bytes/token and launches/token outcomes across GPU, CPU, and

NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Ptx is a production bottleneck surface for CUDA Hopper/Blackwell. Teams that treat **ptx**, **exp2** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds ptx exp2 to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage CUDA Backend is a production bottleneck surface for CUDA Hopper/Blackwell. Teams that treat **YiRage**, **CUDA** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage CUDA to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Chapter 5

AMD XDNA/AIE Dataflow Architecture

Production decode teams rarely fail for lack of framework features—they fail because compiler and kernel schedules violate residency, launch, and bandwidth contracts on the SKU they actually ship (??). This chapter continues the book’s problem-first thread: name the contradiction, trace it to hardware limits, then show how passes and megakernels recover SLO on GPU, CPU, and NPU fleets (??).

5.1 XDNA Tiles

XDNA Tiles is a production bottleneck surface for AMD XDNA/AIE Dataflow Architecture. Teams that treat **XDNA**, **AIE**, **tile** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds XDNA AIE tile to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XDNA Tiles is a production bottleneck surface for AMD XDNA/AIE Dataflow Architecture. Teams that treat **XDNA**, **AIE**, **tile** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds XDNA AIE tile to measurable



Figure 5.1: XDNA Tiles optimization pipeline in AMD XDNA/AIE Dataflow Architecture.



Figure 5.2: Static Dataflow Rules block diagram for AMD XDNA/AIE Dataflow Architecture.

bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Static Dataflow Rules is a production bottleneck surface for AMD XDNA/AIE Dataflow Architecture. Teams that treat **static**, **dataflow** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds static dataflow to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

CUDA XDNA Mapping is a production bottleneck surface for AMD XDNA/AIE Dataflow Architecture. Teams that treat **buffer descriptor**, **TMA** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds buffer descriptor TMA to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

5.2 Static Dataflow Rules

Static Dataflow Rules is a production bottleneck surface for AMD XDNA/AIE Dataflow Architecture. Teams that treat **static**, **dataflow** as independent knobs

routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds static dataflow to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XDNA Tiles is a production bottleneck surface for AMD XDNA/AIE Dataflow Architecture. Teams that treat **XDNA**, **AIE**, **tile** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds XDNA AIE tile to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Static Dataflow Rules is a production bottleneck surface for AMD XDNA/AIE Dataflow Architecture. Teams that treat **static**, **dataflow** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds static dataflow to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

CUDA XDNA Mapping is a production bottleneck surface for AMD XDNA/AIE Dataflow Architecture. Teams that treat **buffer descriptor**, **TMA** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds buffer descriptor TMA to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

5.3 CUDA XDNA Mapping

CUDA XDNA Mapping is a production bottleneck surface for AMD XDNA/AIE Dataflow Architecture. Teams that treat **buffer descriptor**, **TMA** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds buffer descriptor TMA to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XDNA Tiles is a production bottleneck surface for AMD XDNA/AIE Dataflow Architecture. Teams that treat **XDNA**, **AIE**, **tile** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds XDNA AIE tile to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Static Dataflow Rules is a production bottleneck surface for AMD XDNA/AIE Dataflow Architecture. Teams that treat **static**, **dataflow** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds static dataflow to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

CUDA XDNA Mapping is a production bottleneck surface for AMD XDNA/AIE Dataflow Architecture. Teams that treat **buffer descriptor**, **TMA** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds buffer descriptor TMA to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

5.4 Online Softmax Hw

Online Softmax Hw is a production bottleneck surface for AMD XDNA/AIE Dataflow Architecture. Teams that treat **softmax**, **carrier** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds softmax carrier to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XDNA Tiles is a production bottleneck surface for AMD XDNA/AIE Dataflow Architecture. Teams that treat **XDNA**, **AIE**, **tile** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds XDNA AIE tile to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Static Dataflow Rules is a production bottleneck surface for AMD XDNA/AIE Dataflow Architecture. Teams that treat **static**, **dataflow** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds static dataflow to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

CUDA XDNA Mapping is a production bottleneck surface for AMD XDNA/AIE Dataflow Architecture. Teams that treat **buffer descriptor**, **TMA** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds buffer descriptor TMA to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

5.5 Architecture Tradeoffs

Architecture Tradeoffs is a production bottleneck surface for AMD XDNA/AIE Dataflow Architecture. Teams that treat **CUDA**, **XDNA** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds CUDA XDNA to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XDNA Tiles is a production bottleneck surface for AMD XDNA/AIE Dataflow Architecture. Teams that treat **XDNA**, **AIE**, **tile** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds XDNA AIE tile to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Static Dataflow Rules is a production bottleneck surface for AMD XDNA/AIE Dataflow Architecture. Teams that treat **static**, **dataflow** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds static dataflow to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

CUDA XDNA Mapping is a production bottleneck surface for AMD XDNA/AIE Dataflow Architecture. Teams that treat **buffer descriptor**, **TMA** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds buffer descriptor TMA to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

5.6 Dataflow Conclusion

Dataflow Conclusion is a production bottleneck surface for AMD XDNA/AIE Dataflow Architecture. Teams that treat **dataflow**, **inference** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds dataflow inference to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XDNA Tiles is a production bottleneck surface for AMD XDNA/AIE Dataflow Architecture. Teams that treat **XDNA**, **AIE**, **tile** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds XDNA AIE tile to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Static Dataflow Rules is a production bottleneck surface for AMD XDNA/AIE Dataflow Architecture. Teams that treat **static**, **dataflow** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds static dataflow to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

CUDA XDNA Mapping is a production bottleneck surface for AMD XDNA/AIE Dataflow Architecture. Teams that treat **buffer descriptor**, **TMA** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds buffer descriptor TMA to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

5.7 YiRage XDNA Backend

YiRage XDNA Backend is a production bottleneck surface for AMD XDNA/AIE Dataflow Architecture. Teams that treat **YiRage**, **XDNA** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage XDNA to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XDNA Tiles is a production bottleneck surface for AMD XDNA/AIE Dataflow Architecture. Teams that treat **XDNA**, **AIE**, **tile** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds XDNA AIE tile to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Static Dataflow Rules is a production bottleneck surface for AMD XDNA/AIE Dataflow Architecture. Teams that treat **static**, **dataflow** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds static dataflow to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

CUDA XDNA Mapping is a production bottleneck surface for AMD XDNA/AIE Dataflow Architecture. Teams that treat **buffer descriptor**, **TMA** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds buffer descriptor TMA to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

5.8 Chapter Summary

The through-line for ch05 is unchanged: hardware constraints dictate compiler passes, fusion boundaries, and deployment metrics (??). Carry bytes/token and launches/token spreadsheets forward; later chapters assume those baselines exist (??).

5.8.1 Engineering Deep Dive

XDNA Tiles is a production bottleneck surface for AMD XDNA/AIE Dataflow Architecture. Teams that treat **XDNA**, **AIE**, **tile** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds XDNA AIE tile to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Static Dataflow Rules is a production bottleneck surface for AMD XDNA/AIE Dataflow Architecture. Teams that treat **static**, **dataflow** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds static dataflow to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

CUDA XDNA Mapping is a production bottleneck surface for AMD XDNA/AIE Dataflow Architecture. Teams that treat **buffer descriptor**, **TMA** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds buffer descriptor TMA to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Online Softmax Hw is a production bottleneck surface for AMD XDNA/AIE Dataflow Architecture. Teams that treat **softmax**, **carrier** as independent knobs

routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds softmax carrier to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Architecture Tradeoffs is a production bottleneck surface for AMD XDNA/AIE Dataflow Architecture. Teams that treat **CUDA**, **XDNA** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds CUDA XDNA to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Dataflow Conclusion is a production bottleneck surface for AMD XDNA/AIE Dataflow Architecture. Teams that treat **dataflow**, **inference** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds dataflow inference to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage XDNA Backend is a production bottleneck surface for AMD XDNA/AIE Dataflow Architecture. Teams that treat **YiRage**, **XDNA** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage XDNA to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XDNA Tiles is a production bottleneck surface for AMD XDNA/AIE Dataflow Architecture. Teams that treat **XDNA**, **AIE**, **tile** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds XDNA AIE tile to measurable

bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Static Dataflow Rules is a production bottleneck surface for AMD XDNA/AIE Dataflow Architecture. Teams that treat **static**, **dataflow** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds static dataflow to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

CUDA XDNA Mapping is a production bottleneck surface for AMD XDNA/AIE Dataflow Architecture. Teams that treat **buffer descriptor**, **TMA** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds buffer descriptor TMA to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Online Softmax Hw is a production bottleneck surface for AMD XDNA/AIE Dataflow Architecture. Teams that treat **softmax**, **carrier** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds softmax carrier to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Architecture Tradeoffs is a production bottleneck surface for AMD XDNA/AIE Dataflow Architecture. Teams that treat **CUDA**, **XDNA** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds CUDA XDNA to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen,

so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Dataflow Conclusion is a production bottleneck surface for AMD XDNA/AIE Dataflow Architecture. Teams that treat **dataflow**, **inference** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds dataflow inference to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage XDNA Backend is a production bottleneck surface for AMD XDNA/AIE Dataflow Architecture. Teams that treat **YiRage**, **XDNA** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage XDNA to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XDNA Tiles is a production bottleneck surface for AMD XDNA/AIE Dataflow Architecture. Teams that treat **XDNA**, **AIE**, **tile** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds XDNA AIE tile to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Static Dataflow Rules is a production bottleneck surface for AMD XDNA/AIE Dataflow Architecture. Teams that treat **static**, **dataflow** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds static dataflow to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record

fleet measurements alongside compiler flags (??).

CUDA XDNA Mapping is a production bottleneck surface for AMD XDNA/AIE Dataflow Architecture. Teams that treat **buffer descriptor**, **TMA** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds buffer descriptor TMA to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Online Softmax Hw is a production bottleneck surface for AMD XDNA/AIE Dataflow Architecture. Teams that treat **softmax**, **carrier** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds softmax carrier to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Architecture Tradeoffs is a production bottleneck surface for AMD XDNA/AIE Dataflow Architecture. Teams that treat **CUDA**, **XDNA** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds CUDA XDNA to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Dataflow Conclusion is a production bottleneck surface for AMD XDNA/AIE Dataflow Architecture. Teams that treat **dataflow**, **inference** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds dataflow inference to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage XDNA Backend is a production bottleneck surface for AMD

XDNA/AIE Dataflow Architecture. Teams that treat **YiRage**, **XDNA** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage XDNA to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XDNA Tiles is a production bottleneck surface for AMD XDNA/AIE Dataflow Architecture. Teams that treat **XDNA**, **AIE**, **tile** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds XDNA AIE tile to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Static Dataflow Rules is a production bottleneck surface for AMD XDNA/AIE Dataflow Architecture. Teams that treat **static**, **dataflow** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds static dataflow to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

CUDA XDNA Mapping is a production bottleneck surface for AMD XDNA/AIE Dataflow Architecture. Teams that treat **buffer descriptor**, **TMA** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds buffer descriptor TMA to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Online Softmax Hw is a production bottleneck surface for AMD XDNA/AIE Dataflow Architecture. Teams that treat **softmax**, **carrier** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets

(??). The hardware–software contract here binds softmax carrier to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Chapter 6

Data Residency and On-Chip Memory Design

Production decode teams rarely fail for lack of framework features—they fail because compiler and kernel schedules violate residency, launch, and bandwidth contracts on the SKU they actually ship (??). This chapter continues the book’s problem-first thread: name the contradiction, trace it to hardware limits, then show how passes and megakernels recover SLO on GPU, CPU, and NPU fleets (??).

6.1 Lifetime Planning

Lifetime Planning is a production bottleneck surface for Data Residency and On-Chip Memory Design. Teams that treat **lifetime**, **residen** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds lifetime residen to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Lifetime Planning is a production bottleneck surface for Data Residency and On-Chip Memory Design. Teams that treat **lifetime**, **residen** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds lifetime residen

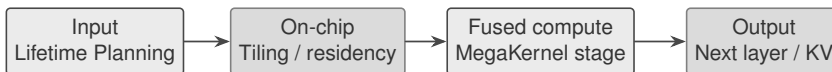


Figure 6.1: Lifetime Planning optimization pipeline in Data Residency and On-Chip Memory Design.



Figure 6.2: GPU Residency block diagram for Data Residency and On-Chip Memory Design.

to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

GPU Residency is a production bottleneck surface for Data Residency and On-Chip Memory Design. Teams that treat **shared**, **register** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds shared register to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

CPU Residency is a production bottleneck surface for Data Residency and On-Chip Memory Design. Teams that treat **cache**, **NUMA** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds cache NUMA to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

6.2 GPU Residency

GPU Residency is a production bottleneck surface for Data Residency and On-Chip Memory Design. Teams that treat **shared**, **register** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds shared register to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Lifetime Planning is a production bottleneck surface for Data Residency and On-Chip Memory Design. Teams that treat **lifetime**, **residen** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds lifetime residen to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

GPU Residency is a production bottleneck surface for Data Residency and On-Chip Memory Design. Teams that treat **shared**, **register** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds shared register to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

CPU Residency is a production bottleneck surface for Data Residency and On-Chip Memory Design. Teams that treat **cache**, **NUMA** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds cache NUMA to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

6.3 CPU Residency

CPU Residency is a production bottleneck surface for Data Residency and On-Chip Memory Design. Teams that treat **cache**, **NUMA** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds cache NUMA to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Lifetime Planning is a production bottleneck surface for Data Residency and On-Chip Memory Design. Teams that treat **lifetime**, **residen** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds lifetime residen to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

GPU Residency is a production bottleneck surface for Data Residency and On-Chip Memory Design. Teams that treat **shared**, **register** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds shared register to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

CPU Residency is a production bottleneck surface for Data Residency and On-Chip Memory Design. Teams that treat **cache**, **NUMA** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds cache NUMA to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

6.4 NPU Residency

NPU Residency is a production bottleneck surface for Data Residency and On-Chip Memory Design. Teams that treat **NPU**, **sram** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds NPU sram to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Lifetime Planning is a production bottleneck surface for Data Residency and On-Chip Memory Design. Teams that treat **lifetime**, **residen** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds lifetime residen to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

GPU Residency is a production bottleneck surface for Data Residency and On-Chip Memory Design. Teams that treat **shared**, **register** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds shared register to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

CPU Residency is a production bottleneck surface for Data Residency and On-Chip Memory Design. Teams that treat **cache**, **NUMA** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds cache NUMA to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

6.5 YiRage Memory Pass

YiRage Memory Pass is a production bottleneck surface for Data Residency and On-Chip Memory Design. Teams that treat **YiRage**, **memory** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage memory to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Lifetime Planning is a production bottleneck surface for Data Residency and On-Chip Memory Design. Teams that treat **lifetime**, **residen** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds lifetime residen to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

GPU Residency is a production bottleneck surface for Data Residency and On-Chip Memory Design. Teams that treat **shared**, **register** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds shared register to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

CPU Residency is a production bottleneck surface for Data Residency and On-Chip Memory Design. Teams that treat **cache**, **NUMA** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds cache NUMA to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

6.6 Chapter Summary

The through-line for ch06 is unchanged: hardware constraints dictate compiler passes, fusion boundaries, and deployment metrics (??). Carry bytes/token and launches/token spreadsheets forward; later chapters assume those baselines exist (??).

6.6.1 Engineering Deep Dive

Lifetime Planning is a production bottleneck surface for Data Residency and On-Chip Memory Design. Teams that treat **lifetime**, **residen** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds lifetime residen to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

GPU Residency is a production bottleneck surface for Data Residency and On-Chip Memory Design. Teams that treat **shared**, **register** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds shared register to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

CPU Residency is a production bottleneck surface for Data Residency and On-Chip Memory Design. Teams that treat **cache**, **NUMA** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds cache NUMA to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

NPU Residency is a production bottleneck surface for Data Residency and On-Chip Memory Design. Teams that treat **NPU**, **srnm** as independent knobs

routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds NPU sram to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Memory Pass is a production bottleneck surface for Data Residency and On-Chip Memory Design. Teams that treat **YiRage, memory** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage memory to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Lifetime Planning is a production bottleneck surface for Data Residency and On-Chip Memory Design. Teams that treat **lifetime, residen** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds lifetime residen to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

GPU Residency is a production bottleneck surface for Data Residency and On-Chip Memory Design. Teams that treat **shared, register** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds shared register to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

CPU Residency is a production bottleneck surface for Data Residency and On-Chip Memory Design. Teams that treat **cache, NUMA** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds cache NUMA to measurable

bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

NPU Residency is a production bottleneck surface for Data Residency and On-Chip Memory Design. Teams that treat **NPU**, **sram** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds NPU sram to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Memory Pass is a production bottleneck surface for Data Residency and On-Chip Memory Design. Teams that treat **YiRage**, **memory** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage memory to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Lifetime Planning is a production bottleneck surface for Data Residency and On-Chip Memory Design. Teams that treat **lifetime**, **residen** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds lifetime residen to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

GPU Residency is a production bottleneck surface for Data Residency and On-Chip Memory Design. Teams that treat **shared**, **register** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds shared register to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks

before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

CPU Residency is a production bottleneck surface for Data Residency and On-Chip Memory Design. Teams that treat **cache**, **NUMA** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds cache NUMA to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

NPU Residency is a production bottleneck surface for Data Residency and On-Chip Memory Design. Teams that treat **NPU**, **sram** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds NPU sram to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Memory Pass is a production bottleneck surface for Data Residency and On-Chip Memory Design. Teams that treat **YiRage**, **memory** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage memory to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Lifetime Planning is a production bottleneck surface for Data Residency and On-Chip Memory Design. Teams that treat **lifetime**, **residen** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds lifetime residen to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint

matrix and record fleet measurements alongside compiler flags (??).

GPU Residency is a production bottleneck surface for Data Residency and On-Chip Memory Design. Teams that treat **shared**, **register** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds shared register to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

CPU Residency is a production bottleneck surface for Data Residency and On-Chip Memory Design. Teams that treat **cache**, **NUMA** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds cache NUMA to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

NPU Residency is a production bottleneck surface for Data Residency and On-Chip Memory Design. Teams that treat **NPU**, **sram** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds NPU sram to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Memory Pass is a production bottleneck surface for Data Residency and On-Chip Memory Design. Teams that treat **YiRage**, **memory** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage memory to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Lifetime Planning is a production bottleneck surface for Data Residency and

On-Chip Memory Design. Teams that treat **lifetime**, **residen** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds lifetime residen to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

GPU Residency is a production bottleneck surface for Data Residency and On-Chip Memory Design. Teams that treat **shared**, **register** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds shared register to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

CPU Residency is a production bottleneck surface for Data Residency and On-Chip Memory Design. Teams that treat **cache**, **NUMA** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds cache NUMA to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

NPU Residency is a production bottleneck surface for Data Residency and On-Chip Memory Design. Teams that treat **NPU**, **sram** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds NPU sram to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Memory Pass is a production bottleneck surface for Data Residency and On-Chip Memory Design. Teams that treat **YiRage**, **memory** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs

on real fleets (??). The hardware–software contract here binds YiRage memory to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Lifetime Planning is a production bottleneck surface for Data Residency and On-Chip Memory Design. Teams that treat **lifetime**, **residen** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds lifetime residen to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

GPU Residency is a production bottleneck surface for Data Residency and On-Chip Memory Design. Teams that treat **shared**, **register** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds shared register to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

CPU Residency is a production bottleneck surface for Data Residency and On-Chip Memory Design. Teams that treat **cache**, **NUMA** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds cache NUMA to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

NPU Residency is a production bottleneck surface for Data Residency and On-Chip Memory Design. Teams that treat **NPU**, **sram** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds NPU sram to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets

(??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Memory Pass is a production bottleneck surface for Data Residency and On-Chip Memory Design. Teams that treat **YiRage**, **memory** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage memory to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Chapter 7

Static Role Assignment and Schedule-Free Kernels

Production decode teams rarely fail for lack of framework features—they fail because compiler and kernel schedules violate residency, launch, and bandwidth contracts on the SKU they actually ship (??). This chapter continues the book’s problem-first thread: name the contradiction, trace it to hardware limits, then show how passes and megakernels recover SLO on GPU, CPU, and NPU fleets (??).

7.1 Static Roles

Static Roles is a production bottleneck surface for Static Role Assignment and Schedule-Free Kernels. Teams that treat **static**, **role** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds static role to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Static Roles is a production bottleneck surface for Static Role Assignment and Schedule-Free Kernels. Teams that treat **static**, **role** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds static role to measurable



Figure 7.1: Static Roles optimization pipeline in Static Role Assignment and Schedule-Free Kernels.



Figure 7.2: Thread Mapping block diagram for Static Role Assignment and Schedule-Free Kernels.

bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Thread Mapping is a production bottleneck surface for Static Role Assignment and Schedule-Free Kernels. Teams that treat **thread**, **warp** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds thread warp to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

No Runtime Sched is a production bottleneck surface for Static Role Assignment and Schedule-Free Kernels. Teams that treat **runtime**, **dispatch** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds runtime dispatch to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

7.2 Thread Mapping

Thread Mapping is a production bottleneck surface for Static Role Assignment and Schedule-Free Kernels. Teams that treat **thread**, **warp** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds thread warp to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Static Roles is a production bottleneck surface for Static Role Assignment and Schedule-Free Kernels. Teams that treat **static**, **role** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds static role to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Thread Mapping is a production bottleneck surface for Static Role Assignment and Schedule-Free Kernels. Teams that treat **thread**, **warp** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds thread warp to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

No Runtime Sched is a production bottleneck surface for Static Role Assignment and Schedule-Free Kernels. Teams that treat **runtime**, **dispatch** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds runtime dispatch to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

7.3 No Runtime Sched

No Runtime Sched is a production bottleneck surface for Static Role Assignment and Schedule-Free Kernels. Teams that treat **runtime**, **dispatch** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds runtime dispatch to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Static Roles is a production bottleneck surface for Static Role Assignment and Schedule-Free Kernels. Teams that treat **static**, **role** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds static role to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Thread Mapping is a production bottleneck surface for Static Role Assignment and Schedule-Free Kernels. Teams that treat **thread**, **warp** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds thread warp to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

No Runtime Sched is a production bottleneck surface for Static Role Assignment and Schedule-Free Kernels. Teams that treat **runtime**, **dispatch** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds runtime dispatch to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

7.4 YiRage Tiling

YiRage Tiling is a production bottleneck surface for Static Role Assignment and Schedule-Free Kernels. Teams that treat **YiRage**, **tiling** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage tiling to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Static Roles is a production bottleneck surface for Static Role Assignment and Schedule-Free Kernels. Teams that treat **static**, **role** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds static role to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Thread Mapping is a production bottleneck surface for Static Role Assignment and Schedule-Free Kernels. Teams that treat **thread**, **warp** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds thread warp to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

No Runtime Sched is a production bottleneck surface for Static Role Assignment and Schedule-Free Kernels. Teams that treat **runtime**, **dispatch** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds runtime dispatch to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

7.5 Chapter Summary

The through-line for ch07 is unchanged: hardware constraints dictate compiler passes, fusion boundaries, and deployment metrics (??). Carry bytes/token and launches/token spreadsheets forward; later chapters assume those baselines exist (??).

7.5.1 Engineering Deep Dive

Static Roles is a production bottleneck surface for Static Role Assignment and Schedule-Free Kernels. Teams that treat **static**, **role** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds static role to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Thread Mapping is a production bottleneck surface for Static Role Assignment and Schedule-Free Kernels. Teams that treat **thread**, **warp** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds thread warp to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

No Runtime Sched is a production bottleneck surface for Static Role Assignment and Schedule-Free Kernels. Teams that treat **runtime**, **dispatch** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds runtime dispatch to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Tiling is a production bottleneck surface for Static Role Assignment and Schedule-Free Kernels. Teams that treat **YiRage**, **tiling** as independent

knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage tiling to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Static Roles is a production bottleneck surface for Static Role Assignment and Schedule-Free Kernels. Teams that treat **static**, **role** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds static role to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Thread Mapping is a production bottleneck surface for Static Role Assignment and Schedule-Free Kernels. Teams that treat **thread**, **warp** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds thread warp to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

No Runtime Sched is a production bottleneck surface for Static Role Assignment and Schedule-Free Kernels. Teams that treat **runtime**, **dispatch** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds runtime dispatch to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Tiling is a production bottleneck surface for Static Role Assignment and Schedule-Free Kernels. Teams that treat **YiRage**, **tiling** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage tiling to

measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Static Roles is a production bottleneck surface for Static Role Assignment and Schedule-Free Kernels. Teams that treat **static**, **role** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds static role to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Thread Mapping is a production bottleneck surface for Static Role Assignment and Schedule-Free Kernels. Teams that treat **thread**, **warp** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds thread warp to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

No Runtime Sched is a production bottleneck surface for Static Role Assignment and Schedule-Free Kernels. Teams that treat **runtime**, **dispatch** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds runtime dispatch to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Tiling is a production bottleneck surface for Static Role Assignment and Schedule-Free Kernels. Teams that treat **YiRage**, **tiling** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage tiling to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks

before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Static Roles is a production bottleneck surface for Static Role Assignment and Schedule-Free Kernels. Teams that treat **static**, **role** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds static role to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Thread Mapping is a production bottleneck surface for Static Role Assignment and Schedule-Free Kernels. Teams that treat **thread**, **warp** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds thread warp to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

No Runtime Sched is a production bottleneck surface for Static Role Assignment and Schedule-Free Kernels. Teams that treat **runtime**, **dispatch** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds runtime dispatch to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Tiling is a production bottleneck surface for Static Role Assignment and Schedule-Free Kernels. Teams that treat **YiRage**, **tiling** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage tiling to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint

matrix and record fleet measurements alongside compiler flags (??).

Static Roles is a production bottleneck surface for Static Role Assignment and Schedule-Free Kernels. Teams that treat **static**, **role** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds static role to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Thread Mapping is a production bottleneck surface for Static Role Assignment and Schedule-Free Kernels. Teams that treat **thread**, **warp** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds thread warp to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

No Runtime Sched is a production bottleneck surface for Static Role Assignment and Schedule-Free Kernels. Teams that treat **runtime**, **dispatch** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds runtime dispatch to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Tiling is a production bottleneck surface for Static Role Assignment and Schedule-Free Kernels. Teams that treat **YiRage**, **tiling** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage tiling to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Static Roles is a production bottleneck surface for Static Role Assignment

and Schedule-Free Kernels. Teams that treat **static**, **role** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds static role to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Thread Mapping is a production bottleneck surface for Static Role Assignment and Schedule-Free Kernels. Teams that treat **thread**, **warp** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds thread warp to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

No Runtime Sched is a production bottleneck surface for Static Role Assignment and Schedule-Free Kernels. Teams that treat **runtime**, **dispatch** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds runtime dispatch to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Tiling is a production bottleneck surface for Static Role Assignment and Schedule-Free Kernels. Teams that treat **YiRage**, **tiling** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage tiling to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Static Roles is a production bottleneck surface for Static Role Assignment and Schedule-Free Kernels. Teams that treat **static**, **role** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real

fleets (??). The hardware–software contract here binds static role to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Thread Mapping is a production bottleneck surface for Static Role Assignment and Schedule-Free Kernels. Teams that treat **thread**, **warp** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds thread warp to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

No Runtime Sched is a production bottleneck surface for Static Role Assignment and Schedule-Free Kernels. Teams that treat **runtime**, **dispatch** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds runtime dispatch to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Tiling is a production bottleneck surface for Static Role Assignment and Schedule-Free Kernels. Teams that treat **YiRage**, **tiling** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage tiling to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Static Roles is a production bottleneck surface for Static Role Assignment and Schedule-Free Kernels. Teams that treat **static**, **role** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds static role to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets

(??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Thread Mapping is a production bottleneck surface for Static Role Assignment and Schedule-Free Kernels. Teams that treat **thread**, **warp** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds thread warp to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

No Runtime Sched is a production bottleneck surface for Static Role Assignment and Schedule-Free Kernels. Teams that treat **runtime**, **dispatch** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds runtime dispatch to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Tiling is a production bottleneck surface for Static Role Assignment and Schedule-Free Kernels. Teams that treat **YiRage**, **tiling** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage tiling to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Static Roles is a production bottleneck surface for Static Role Assignment and Schedule-Free Kernels. Teams that treat **static**, **role** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds static role to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial

readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Thread Mapping is a production bottleneck surface for Static Role Assignment and Schedule-Free Kernels. Teams that treat **thread**, **warp** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds thread warp to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

No Runtime Sched is a production bottleneck surface for Static Role Assignment and Schedule-Free Kernels. Teams that treat **runtime**, **dispatch** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds runtime dispatch to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Tiling is a production bottleneck surface for Static Role Assignment and Schedule-Free Kernels. Teams that treat **YiRage**, **tiling** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage tiling to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Chapter 8

TMA Double-Buffer Pipelines and Async Movement

Production decode teams rarely fail for lack of framework features—they fail because compiler and kernel schedules violate residency, launch, and bandwidth contracts on the SKU they actually ship (??). This chapter continues the book’s problem-first thread: name the contradiction, trace it to hardware limits, then show how passes and megakernels recover SLO on GPU, CPU, and NPU fleets (??).

8.1 Pipeline Theory

Pipeline Theory is a production bottleneck surface for TMA Double-Buffer Pipelines and Async Movement. Teams that treat **pipeline**, **double-buffer** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pipeline double-buffer to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Pipeline Theory is a production bottleneck surface for TMA Double-Buffer Pipelines and Async Movement. Teams that treat **pipeline**, **double-buffer** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds



Figure 8.1: Pipeline Theory optimization pipeline in TMA Double-Buffer Pipelines and Async Movement.



Figure 8.2: GPU TMA block diagram for TMA Double-Buffer Pipelines and Async Movement.

pipeline double-buffer to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

GPU TMA is a production bottleneck surface for TMA Double-Buffer Pipelines and Async Movement. Teams that treat **TMA**, **Hopper** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds TMA Hopper to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

CPU Dma is a production bottleneck surface for TMA Double-Buffer Pipelines and Async Movement. Teams that treat **dma**, **CPU** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds dma CPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

8.2 GPU TMA

GPU TMA is a production bottleneck surface for TMA Double-Buffer Pipelines and Async Movement. Teams that treat **TMA**, **Hopper** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds TMA Hopper to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Pipeline Theory is a production bottleneck surface for TMA Double-Buffer Pipelines and Async Movement. Teams that treat **pipeline**, **double-buffer** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pipeline double-buffer to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

GPU TMA is a production bottleneck surface for TMA Double-Buffer Pipelines and Async Movement. Teams that treat **TMA**, **Hopper** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds TMA Hopper to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

CPU Dma is a production bottleneck surface for TMA Double-Buffer Pipelines and Async Movement. Teams that treat **dma**, **CPU** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds dma CPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record

fleet measurements alongside compiler flags (??).

8.3 CPU Dma

CPU Dma is a production bottleneck surface for TMA Double-Buffer Pipelines and Async Movement. Teams that treat **dma**, **CPU** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds dma CPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Pipeline Theory is a production bottleneck surface for TMA Double-Buffer Pipelines and Async Movement. Teams that treat **pipeline**, **double-buffer** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pipeline double-buffer to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

GPU TMA is a production bottleneck surface for TMA Double-Buffer Pipelines and Async Movement. Teams that treat **TMA**, **Hopper** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds TMA Hopper to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

CPU Dma is a production bottleneck surface for TMA Double-Buffer Pipelines and Async Movement. Teams that treat **dma**, **CPU** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds dma CPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen,

so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

8.4 NPU Pipeline

NPU Pipeline is a production bottleneck surface for TMA Double-Buffer Pipelines and Async Movement. Teams that treat **NPU**, **dma** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds NPU dma to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Pipeline Theory is a production bottleneck surface for TMA Double-Buffer Pipelines and Async Movement. Teams that treat **pipeline**, **double-buffer** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pipeline double-buffer to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

GPU TMA is a production bottleneck surface for TMA Double-Buffer Pipelines and Async Movement. Teams that treat **TMA**, **Hopper** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds TMA Hopper to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

CPU Dma is a production bottleneck surface for TMA Double-Buffer Pipelines and Async Movement. Teams that treat **dma**, **CPU** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds dma CPU to measurable

bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

8.5 YiRage Pipeline Gen

YiRage Pipeline Gen is a production bottleneck surface for TMA Double-Buffer Pipelines and Async Movement. Teams that treat **YiRage, pipeline** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage pipeline to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Pipeline Theory is a production bottleneck surface for TMA Double-Buffer Pipelines and Async Movement. Teams that treat **pipeline, double-buffer** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pipeline double-buffer to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

GPU TMA is a production bottleneck surface for TMA Double-Buffer Pipelines and Async Movement. Teams that treat **TMA, Hopper** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds TMA Hopper to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

CPU Dma is a production bottleneck surface for TMA Double-Buffer Pipelines and Async Movement. Teams that treat **dma, CPU** as independent knobs

routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds dma CPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

8.6 Chapter Summary

The through-line for ch08 is unchanged: hardware constraints dictate compiler passes, fusion boundaries, and deployment metrics (??). Carry bytes/token and launches/token spreadsheets forward; later chapters assume those baselines exist (??).

8.6.1 Engineering Deep Dive

Pipeline Theory is a production bottleneck surface for TMA Double-Buffer Pipelines and Async Movement. Teams that treat **pipeline**, **double-buffer** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pipeline double-buffer to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

GPU TMA is a production bottleneck surface for TMA Double-Buffer Pipelines and Async Movement. Teams that treat **TMA**, **Hopper** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds TMA Hopper to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

CPU Dma is a production bottleneck surface for TMA Double-Buffer Pipelines and Async Movement. Teams that treat **dma**, **CPU** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real

fleets (??). The hardware–software contract here binds dma CPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

NPU Pipeline is a production bottleneck surface for TMA Double-Buffer Pipelines and Async Movement. Teams that treat **NPU**, **dma** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds NPU dma to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Pipeline Gen is a production bottleneck surface for TMA Double-Buffer Pipelines and Async Movement. Teams that treat **YiRage**, **pipeline** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage pipeline to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Pipeline Theory is a production bottleneck surface for TMA Double-Buffer Pipelines and Async Movement. Teams that treat **pipeline**, **double-buffer** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pipeline double-buffer to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

GPU TMA is a production bottleneck surface for TMA Double-Buffer Pipelines and Async Movement. Teams that treat **TMA**, **Hopper** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds TMA Hopper to measurable

bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

CPU Dma is a production bottleneck surface for TMA Double-Buffer Pipelines and Async Movement. Teams that treat **dma**, **CPU** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds dma CPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

NPU Pipeline is a production bottleneck surface for TMA Double-Buffer Pipelines and Async Movement. Teams that treat **NPU**, **dma** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds NPU dma to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Pipeline Gen is a production bottleneck surface for TMA Double-Buffer Pipelines and Async Movement. Teams that treat **YiRage**, **pipeline** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage pipeline to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Pipeline Theory is a production bottleneck surface for TMA Double-Buffer Pipelines and Async Movement. Teams that treat **pipeline**, **double-buffer** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pipeline double-buffer to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints

as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

GPU TMA is a production bottleneck surface for TMA Double-Buffer Pipelines and Async Movement. Teams that treat **TMA**, **Hopper** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds TMA Hopper to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

CPU Dma is a production bottleneck surface for TMA Double-Buffer Pipelines and Async Movement. Teams that treat **dma**, **CPU** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds dma CPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

NPU Pipeline is a production bottleneck surface for TMA Double-Buffer Pipelines and Async Movement. Teams that treat **NPU**, **dma** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds NPU dma to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Pipeline Gen is a production bottleneck surface for TMA Double-Buffer Pipelines and Async Movement. Teams that treat **YiRage**, **pipeline** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage pipeline to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer

dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Pipeline Theory is a production bottleneck surface for TMA Double-Buffer Pipelines and Async Movement. Teams that treat **pipeline**, **double-buffer** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pipeline double-buffer to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

GPU TMA is a production bottleneck surface for TMA Double-Buffer Pipelines and Async Movement. Teams that treat **TMA**, **Hopper** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds TMA Hopper to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

CPU Dma is a production bottleneck surface for TMA Double-Buffer Pipelines and Async Movement. Teams that treat **dma**, **CPU** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds dma CPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

NPU Pipeline is a production bottleneck surface for TMA Double-Buffer Pipelines and Async Movement. Teams that treat **NPU**, **dma** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds NPU dma to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint

matrix and record fleet measurements alongside compiler flags (??).

YiRage Pipeline Gen is a production bottleneck surface for TMA Double-Buffer Pipelines and Async Movement. Teams that treat **YiRage**, **pipeline** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage pipeline to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Pipeline Theory is a production bottleneck surface for TMA Double-Buffer Pipelines and Async Movement. Teams that treat **pipeline**, **double-buffer** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pipeline double-buffer to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

GPU TMA is a production bottleneck surface for TMA Double-Buffer Pipelines and Async Movement. Teams that treat **TMA**, **Hopper** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds TMA Hopper to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

CPU Dma is a production bottleneck surface for TMA Double-Buffer Pipelines and Async Movement. Teams that treat **dma**, **CPU** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds dma CPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

NPU Pipeline is a production bottleneck surface for TMA Double-Buffer Pipelines and Async Movement. Teams that treat **NPU**, **dma** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds NPU dma to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Pipeline Gen is a production bottleneck surface for TMA Double-Buffer Pipelines and Async Movement. Teams that treat **YiRage**, **pipeline** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage pipeline to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Pipeline Theory is a production bottleneck surface for TMA Double-Buffer Pipelines and Async Movement. Teams that treat **pipeline**, **double-buffer** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pipeline double-buffer to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

GPU TMA is a production bottleneck surface for TMA Double-Buffer Pipelines and Async Movement. Teams that treat **TMA**, **Hopper** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds TMA Hopper to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

CPU Dma is a production bottleneck surface for TMA Double-Buffer Pipelines

and Async Movement. Teams that treat **dma**, **CPU** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds dma CPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

NPU Pipeline is a production bottleneck surface for TMA Double-Buffer Pipelines and Async Movement. Teams that treat **NPU**, **dma** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds NPU dma to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Pipeline Gen is a production bottleneck surface for TMA Double-Buffer Pipelines and Async Movement. Teams that treat **YiRage**, **pipeline** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage pipeline to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Chapter 9

Hierarchical Synchronization and Communication

Production decode teams rarely fail for lack of framework features—they fail because compiler and kernel schedules violate residency, launch, and bandwidth contracts on the SKU they actually ship (??). This chapter continues the book’s problem-first thread: name the contradiction, trace it to hardware limits, then show how passes and megakernels recover SLO on GPU, CPU, and NPU fleets (??).

9.1 Sync Hierarchy

Sync Hierarchy is a production bottleneck surface for Hierarchical Synchronization and Communication. Teams that treat **sync**, **barrier** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds sync barrier to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Sync Hierarchy is a production bottleneck surface for Hierarchical Synchronization and Communication. Teams that treat **sync**, **barrier** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds sync barrier to measurable



Figure 9.1: Sync Hierarchy optimization pipeline in Hierarchical Synchronization and Communication.



Figure 9.2: Hardware Sync Pick block diagram for Hierarchical Synchronization and Communication.

bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hardware Sync Pick is a production bottleneck surface for Hierarchical Synchronization and Communication. Teams that treat **GPU**, **CPU**, **NPU** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds GPU CPU NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Deadlock Detection is a production bottleneck surface for Hierarchical Synchronization and Communication. Teams that treat **deadlock** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds deadlock to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

9.2 Hardware Sync Pick

Hardware Sync Pick is a production bottleneck surface for Hierarchical Synchronization and Communication. Teams that treat **GPU**, **CPU**, **NPU** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds GPU CPU NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Sync Hierarchy is a production bottleneck surface for Hierarchical Synchronization and Communication. Teams that treat **sync**, **barrier** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds sync barrier to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hardware Sync Pick is a production bottleneck surface for Hierarchical Synchronization and Communication. Teams that treat **GPU**, **CPU**, **NPU** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds GPU CPU NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Deadlock Detection is a production bottleneck surface for Hierarchical Synchronization and Communication. Teams that treat **deadlock** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds deadlock to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

9.3 Deadlock Detection

Deadlock Detection is a production bottleneck surface for Hierarchical Synchronization and Communication. Teams that treat **deadlock** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds deadlock to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Sync Hierarchy is a production bottleneck surface for Hierarchical Synchronization and Communication. Teams that treat **sync**, **barrier** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds sync barrier to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hardware Sync Pick is a production bottleneck surface for Hierarchical Synchronization and Communication. Teams that treat **GPU**, **CPU**, **NPU** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds GPU CPU NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Deadlock Detection is a production bottleneck surface for Hierarchical Synchronization and Communication. Teams that treat **deadlock** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds deadlock to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

9.4 YiRage Sync Pass

YiRage Sync Pass is a production bottleneck surface for Hierarchical Synchronization and Communication. Teams that treat **YiRage**, **sync** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage sync to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Sync Hierarchy is a production bottleneck surface for Hierarchical Synchronization and Communication. Teams that treat **sync**, **barrier** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds sync barrier to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hardware Sync Pick is a production bottleneck surface for Hierarchical Synchronization and Communication. Teams that treat **GPU**, **CPU**, **NPU** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds GPU CPU NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Deadlock Detection is a production bottleneck surface for Hierarchical Synchronization and Communication. Teams that treat **deadlock** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds deadlock to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

9.5 Chapter Summary

The through-line for ch09 is unchanged: hardware constraints dictate compiler passes, fusion boundaries, and deployment metrics (??). Carry bytes/token and launches/token spreadsheets forward; later chapters assume those baselines exist (??).

9.5.1 Engineering Deep Dive

Sync Hierarchy is a production bottleneck surface for Hierarchical Synchronization and Communication. Teams that treat **sync**, **barrier** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds sync barrier to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hardware Sync Pick is a production bottleneck surface for Hierarchical Synchronization and Communication. Teams that treat **GPU**, **CPU**, **NPU** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds GPU CPU NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Deadlock Detection is a production bottleneck surface for Hierarchical Synchronization and Communication. Teams that treat **deadlock** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds deadlock to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Sync Pass is a production bottleneck surface for Hierarchical Synchronization and Communication. Teams that treat **YiRage**, **sync** as independent

knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage sync to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Sync Hierarchy is a production bottleneck surface for Hierarchical Synchronization and Communication. Teams that treat **sync**, **barrier** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds sync barrier to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hardware Sync Pick is a production bottleneck surface for Hierarchical Synchronization and Communication. Teams that treat **GPU**, **CPU**, **NPU** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds GPU CPU NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Deadlock Detection is a production bottleneck surface for Hierarchical Synchronization and Communication. Teams that treat **deadlock** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds deadlock to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Sync Pass is a production bottleneck surface for Hierarchical Synchronization and Communication. Teams that treat **YiRage**, **sync** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage sync to

measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Sync Hierarchy is a production bottleneck surface for Hierarchical Synchronization and Communication. Teams that treat **sync**, **barrier** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds sync barrier to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hardware Sync Pick is a production bottleneck surface for Hierarchical Synchronization and Communication. Teams that treat **GPU**, **CPU**, **NPU** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds GPU CPU NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Deadlock Detection is a production bottleneck surface for Hierarchical Synchronization and Communication. Teams that treat **deadlock** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds deadlock to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Sync Pass is a production bottleneck surface for Hierarchical Synchronization and Communication. Teams that treat **YiRage**, **sync** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage sync to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks

before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Sync Hierarchy is a production bottleneck surface for Hierarchical Synchronization and Communication. Teams that treat **sync**, **barrier** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds sync barrier to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hardware Sync Pick is a production bottleneck surface for Hierarchical Synchronization and Communication. Teams that treat **GPU**, **CPU**, **NPU** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds GPU CPU NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Deadlock Detection is a production bottleneck surface for Hierarchical Synchronization and Communication. Teams that treat **deadlock** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds deadlock to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Sync Pass is a production bottleneck surface for Hierarchical Synchronization and Communication. Teams that treat **YiRage**, **sync** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage sync to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint

matrix and record fleet measurements alongside compiler flags (??).

Sync Hierarchy is a production bottleneck surface for Hierarchical Synchronization and Communication. Teams that treat **sync**, **barrier** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds sync barrier to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hardware Sync Pick is a production bottleneck surface for Hierarchical Synchronization and Communication. Teams that treat **GPU**, **CPU**, **NPU** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds GPU CPU NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Deadlock Detection is a production bottleneck surface for Hierarchical Synchronization and Communication. Teams that treat **deadlock** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds deadlock to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Sync Pass is a production bottleneck surface for Hierarchical Synchronization and Communication. Teams that treat **YiRage**, **sync** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage sync to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Sync Hierarchy is a production bottleneck surface for Hierarchical Synchroniza-

tion and Communication. Teams that treat **sync**, **barrier** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds sync barrier to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hardware Sync Pick is a production bottleneck surface for Hierarchical Synchronization and Communication. Teams that treat **GPU**, **CPU**, **NPU** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds GPU CPU NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Deadlock Detection is a production bottleneck surface for Hierarchical Synchronization and Communication. Teams that treat **deadlock** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds deadlock to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Sync Pass is a production bottleneck surface for Hierarchical Synchronization and Communication. Teams that treat **YiRage**, **sync** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage sync to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Sync Hierarchy is a production bottleneck surface for Hierarchical Synchronization and Communication. Teams that treat **sync**, **barrier** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real

fleets (??). The hardware–software contract here binds sync barrier to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hardware Sync Pick is a production bottleneck surface for Hierarchical Synchronization and Communication. Teams that treat **GPU**, **CPU**, **NPU** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds GPU CPU NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Deadlock Detection is a production bottleneck surface for Hierarchical Synchronization and Communication. Teams that treat **deadlock** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds deadlock to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Sync Pass is a production bottleneck surface for Hierarchical Synchronization and Communication. Teams that treat **YiRage**, **sync** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage sync to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Sync Hierarchy is a production bottleneck surface for Hierarchical Synchronization and Communication. Teams that treat **sync**, **barrier** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds sync barrier to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets

(??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hardware Sync Pick is a production bottleneck surface for Hierarchical Synchronization and Communication. Teams that treat **GPU**, **CPU**, **NPU** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds GPU CPU NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Deadlock Detection is a production bottleneck surface for Hierarchical Synchronization and Communication. Teams that treat **deadlock** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds deadlock to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Sync Pass is a production bottleneck surface for Hierarchical Synchronization and Communication. Teams that treat **YiRage**, **sync** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage sync to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Sync Hierarchy is a production bottleneck surface for Hierarchical Synchronization and Communication. Teams that treat **sync**, **barrier** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds sync barrier to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial

readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hardware Sync Pick is a production bottleneck surface for Hierarchical Synchronization and Communication. Teams that treat **GPU**, **CPU**, **NPU** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds GPU CPU NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Deadlock Detection is a production bottleneck surface for Hierarchical Synchronization and Communication. Teams that treat **deadlock** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds deadlock to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Sync Pass is a production bottleneck surface for Hierarchical Synchronization and Communication. Teams that treat **YiRage**, **sync** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage sync to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Sync Hierarchy is a production bottleneck surface for Hierarchical Synchronization and Communication. Teams that treat **sync**, **barrier** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds sync barrier to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Chapter 10

Attention Online Softmax Optimization

Production decode teams rarely fail for lack of framework features—they fail because compiler and kernel schedules violate residency, launch, and bandwidth contracts on the SKU they actually ship (??). This chapter continues the book’s problem-first thread: name the contradiction, trace it to hardware limits, then show how passes and megakernels recover SLO on GPU, CPU, and NPU fleets (??).

10.1 Online Softmax

Online Softmax is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **online**, **softmax** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds online softmax to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Online Softmax is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **online**, **softmax** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds online softmax to mea-

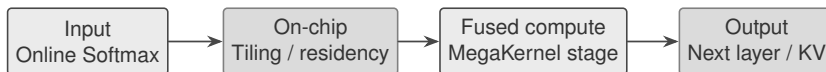


Figure 10.1: Online Softmax optimization pipeline in Attention Online Softmax Optimization.



Figure 10.2: Numerical Stability block diagram for Attention Online Softmax Optimization.

surable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Numerical Stability is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **stable**, **exp** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds stable exp to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Cross Hardware is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **GPU**, **XDNA** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds GPU XDNA to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

10.2 Numerical Stability

Numerical Stability is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **stable**, **exp** as independent knobs routinely ship

kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds stable exp to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Online Softmax is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **online**, **softmax** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds online softmax to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Numerical Stability is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **stable**, **exp** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds stable exp to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Cross Hardware is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **GPU**, **XDNA** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds GPU XDNA to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

10.3 Cross Hardware

Cross Hardware is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **GPU**, **XDNA** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds GPU XDNA to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Online Softmax is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **online**, **softmax** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds online softmax to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Numerical Stability is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **stable**, **exp** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds stable exp to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Cross Hardware is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **GPU**, **XDNA** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds GPU XDNA to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

10.4 Fusion Boundary

Fusion Boundary is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **fuse**, **MegaKernel** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds fuse MegaKernel to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Online Softmax is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **online**, **softmax** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds online softmax to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Numerical Stability is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **stable**, **exp** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds stable exp to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Cross Hardware is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **GPU**, **XDNA** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds GPU XDNA to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

10.5 Chapter Summary

The through-line for ch10 is unchanged: hardware constraints dictate compiler passes, fusion boundaries, and deployment metrics (??). Carry bytes/token and launches/token spreadsheets forward; later chapters assume those baselines exist (??).

10.5.1 Engineering Deep Dive

Online Softmax is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **online**, **softmax** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds online softmax to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Numerical Stability is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **stable**, **exp** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds stable exp to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Cross Hardware is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **GPU**, **XDNA** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds GPU XDNA to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Fusion Boundary is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **fuse**, **MegaKernel** as independent

knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds fuse MegaKernel to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Online Softmax is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **online**, **softmax** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds online softmax to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Numerical Stability is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **stable**, **exp** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds stable exp to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Cross Hardware is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **GPU**, **XDNA** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds GPU XDNA to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Fusion Boundary is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **fuse**, **MegaKernel** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds fuse MegaKernel

to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Online Softmax is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **online**, **softmax** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds online softmax to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Numerical Stability is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **stable**, **exp** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds stable exp to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Cross Hardware is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **GPU**, **XDNA** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds GPU XDNA to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Fusion Boundary is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **fuse**, **MegaKernel** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds fuse MegaKernel to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks

before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Online Softmax is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **online**, **softmax** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds online softmax to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Numerical Stability is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **stable**, **exp** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds stable exp to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Cross Hardware is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **GPU**, **XDNA** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds GPU XDNA to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Fusion Boundary is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **fuse**, **MegaKernel** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds fuse MegaKernel to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint

matrix and record fleet measurements alongside compiler flags (??).

Online Softmax is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **online**, **softmax** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds online softmax to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Numerical Stability is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **stable**, **exp** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds stable exp to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Cross Hardware is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **GPU**, **XDNA** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds GPU XDNA to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Fusion Boundary is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **fuse**, **MegaKernel** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds fuse MegaKernel to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Online Softmax is a production bottleneck surface for Attention Online Soft-

max Optimization. Teams that treat **online**, **softmax** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds online softmax to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Numerical Stability is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **stable**, **exp** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds stable exp to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Cross Hardware is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **GPU**, **XDNA** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds GPU XDNA to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Fusion Boundary is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **fuse**, **MegaKernel** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds fuse MegaKernel to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Online Softmax is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **online**, **softmax** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real

fleets (??). The hardware–software contract here binds online softmax to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Numerical Stability is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **stable**, **exp** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds stable exp to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Cross Hardware is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **GPU**, **XDNA** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds GPU XDNA to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Fusion Boundary is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **fuse**, **MegaKernel** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds fuse MegaKernel to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Online Softmax is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **online**, **softmax** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds online softmax to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU

targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Numerical Stability is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **stable**, **exp** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds stable exp to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Cross Hardware is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **GPU**, **XDNA** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds GPU XDNA to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Fusion Boundary is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **fuse**, **MegaKernel** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds fuse MegaKernel to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Online Softmax is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **online**, **softmax** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds online softmax to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??).

Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Numerical Stability is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **stable**, **exp** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds stable exp to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Cross Hardware is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **GPU**, **XDNA** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds GPU XDNA to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Fusion Boundary is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **fuse**, **MegaKernel** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds fuse MegaKernel to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Online Softmax is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **online**, **softmax** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds online softmax to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Numerical Stability is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **stable**, **exp** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds stable exp to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Cross Hardware is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **GPU**, **XDNA** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds GPU XDNA to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Fusion Boundary is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **fuse**, **MegaKernel** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds fuse MegaKernel to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Online Softmax is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **online**, **softmax** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds online softmax to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Numerical Stability is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **stable**, **exp** as independent knobs

routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds stable exp to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Cross Hardware is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **GPU**, **XDNA** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds GPU XDNA to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Fusion Boundary is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **fuse**, **MegaKernel** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds fuse MegaKernel to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Online Softmax is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **online**, **softmax** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds online softmax to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Numerical Stability is a production bottleneck surface for Attention Online Softmax Optimization. Teams that treat **stable**, **exp** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds stable exp to measurable

bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Chapter 11

Full Decoder MegaKernel Implementation

Production decode teams rarely fail for lack of framework features—they fail because compiler and kernel schedules violate residency, launch, and bandwidth contracts on the SKU they actually ship (??). This chapter continues the book’s problem-first thread: name the contradiction, trace it to hardware limits, then show how passes and megakernels recover SLO on GPU, CPU, and NPU fleets (??).

11.1 MegaKernel Structure

MegaKernel Structure is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **MegaKernel**, **decoder** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds MegaKernel decoder to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MegaKernel Structure is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **MegaKernel**, **decoder** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds MegaKernel

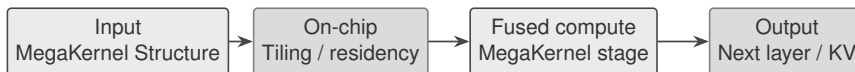


Figure 11.1: MegaKernel Structure optimization pipeline in Full Decoder MegaKernel Implementation.



Figure 11.2: Qkv Attn Mlp block diagram for Full Decoder MegaKernel Implementation.

decoder to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Qkv Attn Mlp is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **qkv**, **attention** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds qkv attention to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Source Walkthrough is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **source**, **implement** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds source implement to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

11.2 Qkv Attn Mlp

Qkv Attn Mlp is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **qkv**, **attention** as independent knobs routinely

ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds qkv attention to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MegaKernel Structure is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **MegaKernel**, **decoder** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds MegaKernel decoder to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Qkv Attn Mlp is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **qkv**, **attention** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds qkv attention to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Source Walkthrough is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **source**, **implement** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds source implement to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

11.3 Source Walkthrough

Source Walkthrough is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **source**, **implement** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds source implement to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MegaKernel Structure is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **MegaKernel**, **decoder** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds MegaKernel decoder to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Qkv Attn Mlp is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **qkv**, **attention** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds qkv attention to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Source Walkthrough is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **source**, **implement** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds source implement to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

11.4 Benchmark Results

Benchmark Results is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **benchmark**, **speedup** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds benchmark speedup to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MegaKernel Structure is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **MegaKernel**, **decoder** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds MegaKernel decoder to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Qkv Attn Mlp is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **qkv**, **attention** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds qkv attention to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Source Walkthrough is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **source**, **implement** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds source implement to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

11.5 Chapter Summary

The through-line for ch11 is unchanged: hardware constraints dictate compiler passes, fusion boundaries, and deployment metrics (??). Carry bytes/token and launches/token spreadsheets forward; later chapters assume those baselines exist (??).

11.5.1 Engineering Deep Dive

MegaKernel Structure is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **MegaKernel**, **decoder** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds MegaKernel decoder to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Qkv Attn Mlp is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **qkv**, **attention** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds qkv attention to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Source Walkthrough is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **source**, **implement** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds source implement to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Benchmark Results is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **benchmark**, **speedup** as independent

knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds benchmark speedup to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MegaKernel Structure is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **MegaKernel**, **decoder** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds MegaKernel decoder to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Qkv Attn Mlp is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **qkv**, **attention** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds qkv attention to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Source Walkthrough is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **source**, **implement** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds source implement to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Benchmark Results is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **benchmark**, **speedup** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds benchmark speedup

to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MegaKernel Structure is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **MegaKernel**, **decoder** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds MegaKernel decoder to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Qkv Attn Mlp is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **qkv**, **attention** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds qkv attention to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Source Walkthrough is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **source**, **implement** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds source implement to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Benchmark Results is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **benchmark**, **speedup** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds benchmark speedup to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks

before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MegaKernel Structure is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **MegaKernel**, **decoder** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds MegaKernel decoder to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Qkv Attn Mlp is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **qkv**, **attention** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds qkv attention to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Source Walkthrough is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **source**, **implement** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds source implement to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Benchmark Results is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **benchmark**, **speedup** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds benchmark speedup to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint

matrix and record fleet measurements alongside compiler flags (??).

MegaKernel Structure is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **MegaKernel**, **decoder** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds MegaKernel decoder to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Qkv Attn Mlp is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **qkv**, **attention** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds qkv attention to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Source Walkthrough is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **source**, **implement** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds source implement to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Benchmark Results is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **benchmark**, **speedup** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds benchmark speedup to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MegaKernel Structure is a production bottleneck surface for Full Decoder

MegaKernel Implementation. Teams that treat **MegaKernel**, **decoder** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds MegaKernel decoder to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Qkv Attn Mlp is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **qkv**, **attention** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds qkv attention to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Source Walkthrough is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **source**, **implement** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds source implement to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Benchmark Results is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **benchmark**, **speedup** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds benchmark speedup to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MegaKernel Structure is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **MegaKernel**, **decoder** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode

SLOs on real fleets (??). The hardware–software contract here binds MegaKernel decoder to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Qkv Attn Mlp is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **qkv**, **attention** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds qkv attention to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Source Walkthrough is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **source**, **implement** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds source implement to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Benchmark Results is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **benchmark**, **speedup** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds benchmark speedup to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MegaKernel Structure is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **MegaKernel**, **decoder** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds MegaKernel decoder to measurable bytes/token and launches/token outcomes across GPU,

CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Qkv Attn Mlp is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **qkv**, **attention** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds qkv attention to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Source Walkthrough is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **source**, **implement** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds source implement to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Benchmark Results is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **benchmark**, **speedup** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds benchmark speedup to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MegaKernel Structure is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **MegaKernel**, **decoder** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds MegaKernel decoder to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer

dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Qkv Attn Mlp is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **qkv**, **attention** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds qkv attention to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Source Walkthrough is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **source**, **implement** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds source implement to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Benchmark Results is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **benchmark**, **speedup** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds benchmark speedup to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MegaKernel Structure is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **MegaKernel**, **decoder** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds MegaKernel decoder to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Qkv Attn Mlp is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **qkv**, **attention** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds qkv attention to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Source Walkthrough is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **source**, **implement** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds source implement to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Benchmark Results is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **benchmark**, **speedup** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds benchmark speedup to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MegaKernel Structure is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **MegaKernel**, **decoder** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds MegaKernel decoder to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Qkv Attn Mlp is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **qkv**, **attention** as independent knobs routinely

ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds qkv attention to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Source Walkthrough is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **source**, **implement** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds source implement to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Benchmark Results is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **benchmark**, **speedup** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds benchmark speedup to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MegaKernel Structure is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **MegaKernel**, **decoder** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds MegaKernel decoder to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Qkv Attn Mlp is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **qkv**, **attention** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds qkv attention to measurable bytes/token

and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Source Walkthrough is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **source**, **implement** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds source implement to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Benchmark Results is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **benchmark**, **speedup** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds benchmark speedup to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MegaKernel Structure is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **MegaKernel**, **decoder** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds MegaKernel decoder to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Qkv Attn Mlp is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **qkv**, **attention** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds qkv attention to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal

fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Source Walkthrough is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **source**, **implement** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds source implement to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Benchmark Results is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **benchmark**, **speedup** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds benchmark speedup to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MegaKernel Structure is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **MegaKernel**, **decoder** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds MegaKernel decoder to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Qkv Attn Mlp is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **qkv**, **attention** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds qkv attention to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet

measurements alongside compiler flags (??).

Source Walkthrough is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **source**, **implement** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds source implement to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Benchmark Results is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **benchmark**, **speedup** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds benchmark speedup to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MegaKernel Structure is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **MegaKernel**, **decoder** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds MegaKernel decoder to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Qkv Attn Mlp is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **qkv**, **attention** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds qkv attention to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Source Walkthrough is a production bottleneck surface for Full Decoder MegaK-

ernel Implementation. Teams that treat **source**, **implement** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds source implement to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Benchmark Results is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **benchmark**, **speedup** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds benchmark speedup to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MegaKernel Structure is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **MegaKernel**, **decoder** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds MegaKernel decoder to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Qkv Attn Mlp is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **qkv**, **attention** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds qkv attention to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Source Walkthrough is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **source**, **implement** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real

fleets (??). The hardware–software contract here binds source implement to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Benchmark Results is a production bottleneck surface for Full Decoder MegaKernel Implementation. Teams that treat **benchmark**, **speedup** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds benchmark speedup to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Chapter 12

Three Kernel Execution Modes

Production decode teams rarely fail for lack of framework features—they fail because compiler and kernel schedules violate residency, launch, and bandwidth contracts on the SKU they actually ship (??). This chapter continues the book’s problem-first thread: name the contradiction, trace it to hardware limits, then show how passes and megakernels recover SLO on GPU, CPU, and NPU fleets (??).

12.1 Eager Mode

Eager Mode is a production bottleneck surface for Three Kernel Execution Modes. Teams that treat **eager**, **PyTorch** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds eager PyTorch to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Eager Mode is a production bottleneck surface for Three Kernel Execution Modes. Teams that treat **eager**, **PyTorch** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds eager PyTorch to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal



Figure 12.1: Eager Mode optimization pipeline in Three Kernel Execution Modes.



Figure 12.2: Graph Mode block diagram for Three Kernel Execution Modes.

fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Graph Mode is a production bottleneck surface for Three Kernel Execution Modes. Teams that treat **CUDA Graphs** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds CUDA Graphs to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MegaKernel Mode is a production bottleneck surface for Three Kernel Execution Modes. Teams that treat **MegaKernel** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds MegaKernel to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

12.2 Graph Mode

Graph Mode is a production bottleneck surface for Three Kernel Execution Modes. Teams that treat **CUDA Graphs** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds CUDA Graphs to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage

lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Eager Mode is a production bottleneck surface for Three Kernel Execution Modes. Teams that treat **eager**, **PyTorch** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds eager PyTorch to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Graph Mode is a production bottleneck surface for Three Kernel Execution Modes. Teams that treat **CUDA Graphs** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds CUDA Graphs to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MegaKernel Mode is a production bottleneck surface for Three Kernel Execution Modes. Teams that treat **MegaKernel** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds MegaKernel to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

12.3 MegaKernel Mode

MegaKernel Mode is a production bottleneck surface for Three Kernel Execution Modes. Teams that treat **MegaKernel** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds MegaKernel to measurable bytes/token and

launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Eager Mode is a production bottleneck surface for Three Kernel Execution Modes. Teams that treat **eager**, **PyTorch** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds eager PyTorch to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Graph Mode is a production bottleneck surface for Three Kernel Execution Modes. Teams that treat **CUDA Graphs** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds CUDA Graphs to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MegaKernel Mode is a production bottleneck surface for Three Kernel Execution Modes. Teams that treat **MegaKernel** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds MegaKernel to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

12.4 Mode Selection

Mode Selection is a production bottleneck surface for Three Kernel Execution Modes. Teams that treat **hardware**, **trade** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The

hardware–software contract here binds hardware trade to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Eager Mode is a production bottleneck surface for Three Kernel Execution Modes. Teams that treat **eager, PyTorch** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds eager PyTorch to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Graph Mode is a production bottleneck surface for Three Kernel Execution Modes. Teams that treat **CUDA Graphs** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds CUDA Graphs to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MegaKernel Mode is a production bottleneck surface for Three Kernel Execution Modes. Teams that treat **MegaKernel** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds MegaKernel to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

12.5 Chapter Summary

The through-line for ch12 is unchanged: hardware constraints dictate compiler passes, fusion boundaries, and deployment metrics (??). Carry bytes/token and

launches/token spreadsheets forward; later chapters assume those baselines exist (??).

12.5.1 Engineering Deep Dive

Eager Mode is a production bottleneck surface for Three Kernel Execution Modes. Teams that treat **eager**, **PyTorch** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds eager PyTorch to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Graph Mode is a production bottleneck surface for Three Kernel Execution Modes. Teams that treat **CUDA Graphs** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds CUDA Graphs to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MegaKernel Mode is a production bottleneck surface for Three Kernel Execution Modes. Teams that treat **MegaKernel** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds MegaKernel to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Mode Selection is a production bottleneck surface for Three Kernel Execution Modes. Teams that treat **hardware**, **trade** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware trade to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal

fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Eager Mode is a production bottleneck surface for Three Kernel Execution Modes. Teams that treat **eager**, **PyTorch** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds eager PyTorch to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Graph Mode is a production bottleneck surface for Three Kernel Execution Modes. Teams that treat **CUDA Graphs** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds CUDA Graphs to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MegaKernel Mode is a production bottleneck surface for Three Kernel Execution Modes. Teams that treat **MegaKernel** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds MegaKernel to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Mode Selection is a production bottleneck surface for Three Kernel Execution Modes. Teams that treat **hardware**, **trade** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware trade to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet

measurements alongside compiler flags (??).

Eager Mode is a production bottleneck surface for Three Kernel Execution Modes. Teams that treat **eager**, **PyTorch** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds eager PyTorch to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Graph Mode is a production bottleneck surface for Three Kernel Execution Modes. Teams that treat **CUDA Graphs** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds CUDA Graphs to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MegaKernel Mode is a production bottleneck surface for Three Kernel Execution Modes. Teams that treat **MegaKernel** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds MegaKernel to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Mode Selection is a production bottleneck surface for Three Kernel Execution Modes. Teams that treat **hardware**, **trade** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware trade to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Eager Mode is a production bottleneck surface for Three Kernel Execution

Modes. Teams that treat **eager**, **PyTorch** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds eager PyTorch to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Graph Mode is a production bottleneck surface for Three Kernel Execution Modes. Teams that treat **CUDA Graphs** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds CUDA Graphs to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MegaKernel Mode is a production bottleneck surface for Three Kernel Execution Modes. Teams that treat **MegaKernel** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds MegaKernel to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Mode Selection is a production bottleneck surface for Three Kernel Execution Modes. Teams that treat **hardware**, **trade** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware trade to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Eager Mode is a production bottleneck surface for Three Kernel Execution Modes. Teams that treat **eager**, **PyTorch** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The

hardware–software contract here binds eager PyTorch to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Graph Mode is a production bottleneck surface for Three Kernel Execution Modes. Teams that treat **CUDA Graphs** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds CUDA Graphs to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MegaKernel Mode is a production bottleneck surface for Three Kernel Execution Modes. Teams that treat **MegaKernel** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds MegaKernel to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Mode Selection is a production bottleneck surface for Three Kernel Execution Modes. Teams that treat **hardware, trade** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware trade to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Eager Mode is a production bottleneck surface for Three Kernel Execution Modes. Teams that treat **eager, PyTorch** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds eager PyTorch to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage

lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Graph Mode is a production bottleneck surface for Three Kernel Execution Modes. Teams that treat **CUDA Graphs** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds CUDA Graphs to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MegaKernel Mode is a production bottleneck surface for Three Kernel Execution Modes. Teams that treat **MegaKernel** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds MegaKernel to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Mode Selection is a production bottleneck surface for Three Kernel Execution Modes. Teams that treat **hardware, trade** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware trade to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Eager Mode is a production bottleneck surface for Three Kernel Execution Modes. Teams that treat **eager, PyTorch** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds eager PyTorch to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers

should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Graph Mode is a production bottleneck surface for Three Kernel Execution Modes. Teams that treat **CUDA Graphs** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds CUDA Graphs to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MegaKernel Mode is a production bottleneck surface for Three Kernel Execution Modes. Teams that treat **MegaKernel** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds MegaKernel to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Mode Selection is a production bottleneck surface for Three Kernel Execution Modes. Teams that treat **hardware, trade** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware trade to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Chapter 13

Core Theory: AI Compiler Optimization Principles

Production decode teams rarely fail for lack of framework features—they fail because compiler and kernel schedules violate residency, launch, and bandwidth contracts on the SKU they actually ship (??). This chapter continues the book’s problem-first thread: name the contradiction, trace it to hardware limits, then show how passes and megakernels recover SLO on GPU, CPU, and NPU fleets (??).

13.1 Tiling Theory

Tiling Theory is a production bottleneck surface for Core Theory. Teams that treat **tiling** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tiling to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Tiling Theory is a production bottleneck surface for Core Theory. Teams that treat **tiling** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tiling to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit



Figure 13.1: Tiling Theory optimization pipeline in Core Theory: AI Compiler Optimization Principles.



Figure 13.2: Fusion Theory block diagram for Core Theory: AI Compiler Optimization Principles.

legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Fusion Theory is a production bottleneck surface for Core Theory. Teams that treat **fusion** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds fusion to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Layout Theory is a production bottleneck surface for Core Theory. Teams that treat **layout** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds layout to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

13.2 Fusion Theory

Fusion Theory is a production bottleneck surface for Core Theory. Teams that treat **fusion** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds fusion to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit

legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Tiling Theory is a production bottleneck surface for Core Theory. Teams that treat **tiling** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tiling to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Fusion Theory is a production bottleneck surface for Core Theory. Teams that treat **fusion** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds fusion to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Layout Theory is a production bottleneck surface for Core Theory. Teams that treat **layout** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds layout to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

13.3 Layout Theory

Layout Theory is a production bottleneck surface for Core Theory. Teams that treat **layout** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds layout to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Tiling Theory is a production bottleneck surface for Core Theory. Teams that treat **tiling** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tiling to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Fusion Theory is a production bottleneck surface for Core Theory. Teams that treat **fusion** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds fusion to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Layout Theory is a production bottleneck surface for Core Theory. Teams that treat **layout** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds layout to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

13.4 Parallel Scheduling

Parallel Scheduling is a production bottleneck surface for Core Theory. Teams that treat **parallel** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds parallel to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Tiling Theory is a production bottleneck surface for Core Theory. Teams that treat **tiling** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here

binds tiling to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Fusion Theory is a production bottleneck surface for Core Theory. Teams that treat **fusion** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds fusion to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Layout Theory is a production bottleneck surface for Core Theory. Teams that treat **layout** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds layout to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

13.5 GPU CPU NPU Split

GPU CPU NPU Split is a production bottleneck surface for Core Theory. Teams that treat **GPU**, **CPU**, **NPU** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds GPU CPU NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Tiling Theory is a production bottleneck surface for Core Theory. Teams that treat **tiling** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tiling to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit

legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Fusion Theory is a production bottleneck surface for Core Theory. Teams that treat **fusion** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds fusion to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Layout Theory is a production bottleneck surface for Core Theory. Teams that treat **layout** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds layout to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

13.6 Chapter Summary

The through-line for ch13 is unchanged: hardware constraints dictate compiler passes, fusion boundaries, and deployment metrics (??). Carry bytes/token and launches/token spreadsheets forward; later chapters assume those baselines exist (??).

13.6.1 Engineering Deep Dive

Tiling Theory is a production bottleneck surface for Core Theory. Teams that treat **tiling** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tiling to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Fusion Theory is a production bottleneck surface for Core Theory. Teams that

treat **fusion** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds fusion to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Layout Theory is a production bottleneck surface for Core Theory. Teams that treat **layout** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds layout to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Parallel Scheduling is a production bottleneck surface for Core Theory. Teams that treat **parallel** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds parallel to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

GPU CPU NPU Split is a production bottleneck surface for Core Theory. Teams that treat **GPU**, **CPU**, **NPU** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds GPU CPU NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Tiling Theory is a production bottleneck surface for Core Theory. Teams that treat **tiling** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tiling to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit

legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Fusion Theory is a production bottleneck surface for Core Theory. Teams that treat **fusion** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds fusion to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Layout Theory is a production bottleneck surface for Core Theory. Teams that treat **layout** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds layout to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Parallel Scheduling is a production bottleneck surface for Core Theory. Teams that treat **parallel** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds parallel to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

GPU CPU NPU Split is a production bottleneck surface for Core Theory. Teams that treat **GPU**, **CPU**, **NPU** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds GPU CPU NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Tiling Theory is a production bottleneck surface for Core Theory. Teams that

treat **tiling** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tiling to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Fusion Theory is a production bottleneck surface for Core Theory. Teams that treat **fusion** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds fusion to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Layout Theory is a production bottleneck surface for Core Theory. Teams that treat **layout** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds layout to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Parallel Scheduling is a production bottleneck surface for Core Theory. Teams that treat **parallel** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds parallel to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

GPU CPU NPU Split is a production bottleneck surface for Core Theory. Teams that treat **GPU**, **CPU**, **NPU** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds GPU CPU NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion

fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Tiling Theory is a production bottleneck surface for Core Theory. Teams that treat **tiling** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tiling to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Fusion Theory is a production bottleneck surface for Core Theory. Teams that treat **fusion** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds fusion to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Layout Theory is a production bottleneck surface for Core Theory. Teams that treat **layout** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds layout to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Parallel Scheduling is a production bottleneck surface for Core Theory. Teams that treat **parallel** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds parallel to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

GPU CPU NPU Split is a production bottleneck surface for Core Theory. Teams that treat **GPU**, **CPU**, **NPU** as independent knobs routinely ship kernels

that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds GPU CPU NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Tiling Theory is a production bottleneck surface for Core Theory. Teams that treat **tiling** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tiling to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Fusion Theory is a production bottleneck surface for Core Theory. Teams that treat **fusion** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds fusion to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Layout Theory is a production bottleneck surface for Core Theory. Teams that treat **layout** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds layout to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Parallel Scheduling is a production bottleneck surface for Core Theory. Teams that treat **parallel** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds parallel to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to

the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

GPU CPU NPU Split is a production bottleneck surface for Core Theory. Teams that treat **GPU**, **CPU**, **NPU** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds GPU CPU NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Tiling Theory is a production bottleneck surface for Core Theory. Teams that treat **tiling** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tiling to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Fusion Theory is a production bottleneck surface for Core Theory. Teams that treat **fusion** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds fusion to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Layout Theory is a production bottleneck surface for Core Theory. Teams that treat **layout** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds layout to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Parallel Scheduling is a production bottleneck surface for Core Theory. Teams that treat **parallel** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software

contract here binds parallel to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

GPU CPU NPU Split is a production bottleneck surface for Core Theory. Teams that treat **GPU**, **CPU**, **NPU** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds GPU CPU NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Tiling Theory is a production bottleneck surface for Core Theory. Teams that treat **tiling** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tiling to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Fusion Theory is a production bottleneck surface for Core Theory. Teams that treat **fusion** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds fusion to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Layout Theory is a production bottleneck surface for Core Theory. Teams that treat **layout** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds layout to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3

constraint matrix and record fleet measurements alongside compiler flags (??).

Parallel Scheduling is a production bottleneck surface for Core Theory. Teams that treat **parallel** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds parallel to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

GPU CPU NPU Split is a production bottleneck surface for Core Theory. Teams that treat **GPU**, **CPU**, **NPU** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds GPU CPU NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Tiling Theory is a production bottleneck surface for Core Theory. Teams that treat **tiling** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tiling to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Fusion Theory is a production bottleneck surface for Core Theory. Teams that treat **fusion** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds fusion to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Layout Theory is a production bottleneck surface for Core Theory. Teams that treat **layout** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here

binds layout to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Parallel Scheduling is a production bottleneck surface for Core Theory. Teams that treat **parallel** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds parallel to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

GPU CPU NPU Split is a production bottleneck surface for Core Theory. Teams that treat **GPU, CPU, NPU** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds GPU CPU NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Tiling Theory is a production bottleneck surface for Core Theory. Teams that treat **tiling** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tiling to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Fusion Theory is a production bottleneck surface for Core Theory. Teams that treat **fusion** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds fusion to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3

constraint matrix and record fleet measurements alongside compiler flags (??).

Layout Theory is a production bottleneck surface for Core Theory. Teams that treat **layout** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds layout to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Parallel Scheduling is a production bottleneck surface for Core Theory. Teams that treat **parallel** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds parallel to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

GPU CPU NPU Split is a production bottleneck surface for Core Theory. Teams that treat **GPU**, **CPU**, **NPU** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds GPU CPU NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Tiling Theory is a production bottleneck surface for Core Theory. Teams that treat **tiling** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tiling to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Fusion Theory is a production bottleneck surface for Core Theory. Teams that treat **fusion** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here

binds fusion to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Layout Theory is a production bottleneck surface for Core Theory. Teams that treat **layout** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds layout to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Parallel Scheduling is a production bottleneck surface for Core Theory. Teams that treat **parallel** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds parallel to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

GPU CPU NPU Split is a production bottleneck surface for Core Theory. Teams that treat **GPU**, **CPU**, **NPU** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds GPU CPU NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Chapter 14

MLIR: Modern Compiler Infrastructure

Production decode teams rarely fail for lack of framework features—they fail because compiler and kernel schedules violate residency, launch, and bandwidth contracts on the SKU they actually ship (??). This chapter continues the book’s problem-first thread: name the contradiction, trace it to hardware limits, then show how passes and megakernels recover SLO on GPU, CPU, and NPU fleets (??).

14.1 MLIR Hw Matrix

MLIR Hw Matrix is a production bottleneck surface for MLIR. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MLIR Hw Matrix is a production bottleneck surface for MLIR. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable

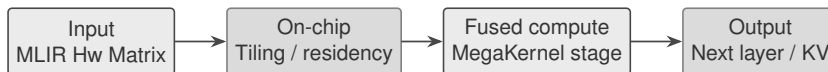


Figure 14.1: MLIR Hw Matrix optimization pipeline in MLIR: Modern Compiler Infrastructure.



Figure 14.2: MLIR Arch Passes block diagram for MLIR: Modern Compiler Infrastructure.

bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MLIR Arch Passes is a production bottleneck surface for MLIR. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MLIR Codegen is a production bottleneck surface for MLIR. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

14.2 MLIR Arch Passes

MLIR Arch Passes is a production bottleneck surface for MLIR. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win

microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MLIR Hw Matrix is a production bottleneck surface for MLIR. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MLIR Arch Passes is a production bottleneck surface for MLIR. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MLIR Codegen is a production bottleneck surface for MLIR. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

14.3 MLIR Codegen

MLIR Codegen is a production bottleneck surface for MLIR. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MLIR Hw Matrix is a production bottleneck surface for MLIR. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MLIR Arch Passes is a production bottleneck surface for MLIR. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MLIR Codegen is a production bottleneck surface for MLIR. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

14.4 MLIR Tuning

MLIR Tuning is a production bottleneck surface for MLIR. Teams that treat **tun**, **profile**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tun profile hardware pitfall to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MLIR Hw Matrix is a production bottleneck surface for MLIR. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MLIR Arch Passes is a production bottleneck surface for MLIR. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MLIR Codegen is a production bottleneck surface for MLIR. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

14.5 MLIR Case Study

MLIR Case Study is a production bottleneck surface for MLIR. Teams that treat **case**, **benchmark**, **cross-hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds case benchmark cross-hardware YiRage to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MLIR Hw Matrix is a production bottleneck surface for MLIR. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MLIR Arch Passes is a production bottleneck surface for MLIR. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MLIR Codegen is a production bottleneck surface for MLIR. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

14.6 Chapter Summary

The through-line for ch14 is unchanged: hardware constraints dictate compiler passes, fusion boundaries, and deployment metrics (??). Carry bytes/token and launches/token spreadsheets forward; later chapters assume those baselines exist (??).

14.6.1 Engineering Deep Dive

MLIR Hw Matrix is a production bottleneck surface for MLIR. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MLIR Arch Passes is a production bottleneck surface for MLIR. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MLIR Codegen is a production bottleneck surface for MLIR. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MLIR Tuning is a production bottleneck surface for MLIR. Teams that treat **tun**, **profile**, **hardware** as independent knobs routinely ship kernels that win

microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds `tun` profile hardware pitfall to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MLIR Case Study is a production bottleneck surface for MLIR. Teams that treat **case**, **benchmark**, **cross-hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds case benchmark cross-hardware YiRage to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MLIR Hw Matrix is a production bottleneck surface for MLIR. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MLIR Arch Passes is a production bottleneck surface for MLIR. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MLIR Codegen is a production bottleneck surface for MLIR. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and

launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MLIR Tuning is a production bottleneck surface for MLIR. Teams that treat **tun**, **profile**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tun profile hardware pitfall to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MLIR Case Study is a production bottleneck surface for MLIR. Teams that treat **case**, **benchmark**, **cross-hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds case benchmark cross-hardware YiRage to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MLIR Hw Matrix is a production bottleneck surface for MLIR. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MLIR Arch Passes is a production bottleneck surface for MLIR. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion

fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MLIR Codegen is a production bottleneck surface for MLIR. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MLIR Tuning is a production bottleneck surface for MLIR. Teams that treat **tun**, **profile**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tun profile hardware pitfall to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MLIR Case Study is a production bottleneck surface for MLIR. Teams that treat **case**, **benchmark**, **cross-hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds case benchmark cross-hardware YiRage to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MLIR Hw Matrix is a production bottleneck surface for MLIR. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record

fleet measurements alongside compiler flags (??).

MLIR Arch Passes is a production bottleneck surface for MLIR. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MLIR Codegen is a production bottleneck surface for MLIR. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MLIR Tuning is a production bottleneck surface for MLIR. Teams that treat **tun**, **profile**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tun profile hardware pitfall to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MLIR Case Study is a production bottleneck surface for MLIR. Teams that treat **case**, **benchmark**, **cross-hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds case benchmark cross-hardware YiRage to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MLIR Hw Matrix is a production bottleneck surface for MLIR. Teams that

treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MLIR Arch Passes is a production bottleneck surface for MLIR. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MLIR Codegen is a production bottleneck surface for MLIR. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MLIR Tuning is a production bottleneck surface for MLIR. Teams that treat **tun**, **profile**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tun profile hardware pitfall to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MLIR Case Study is a production bottleneck surface for MLIR. Teams that treat **case**, **benchmark**, **cross-hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The

hardware–software contract here binds case benchmark cross-hardware YiRage to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MLIR Hw Matrix is a production bottleneck surface for MLIR. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MLIR Arch Passes is a production bottleneck surface for MLIR. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MLIR Codegen is a production bottleneck surface for MLIR. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MLIR Tuning is a production bottleneck surface for MLIR. Teams that treat **tun**, **profile**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tun profile hardware pitfall to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers

the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MLIR Case Study is a production bottleneck surface for MLIR. Teams that treat **case**, **benchmark**, **cross-hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds case benchmark cross-hardware YiRage to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MLIR Hw Matrix is a production bottleneck surface for MLIR. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MLIR Arch Passes is a production bottleneck surface for MLIR. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MLIR Codegen is a production bottleneck surface for MLIR. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map

each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MLIR Tuning is a production bottleneck surface for MLIR. Teams that treat **tun, profile, hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tun profile hardware pitfall to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MLIR Case Study is a production bottleneck surface for MLIR. Teams that treat **case, benchmark, cross-hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds case benchmark cross-hardware YiRage to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MLIR Hw Matrix is a production bottleneck surface for MLIR. Teams that treat **hardware, target, matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MLIR Arch Passes is a production bottleneck surface for MLIR. Teams that treat **pass, lower, optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MLIR Codegen is a production bottleneck surface for MLIR. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

MLIR Tuning is a production bottleneck surface for MLIR. Teams that treat **tun**, **profile**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tun profile hardware pitfall to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Chapter 15

XLA: Production-Grade AI Compiler

Production decode teams rarely fail for lack of framework features—they fail because compiler and kernel schedules violate residency, launch, and bandwidth contracts on the SKU they actually ship (??). This chapter continues the book’s problem-first thread: name the contradiction, trace it to hardware limits, then show how passes and megakernels recover SLO on GPU, CPU, and NPU fleets (??).

15.1 XLA Hw Matrix

XLA Hw Matrix is a production bottleneck surface for XLA. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XLA Hw Matrix is a production bottleneck surface for XLA. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable

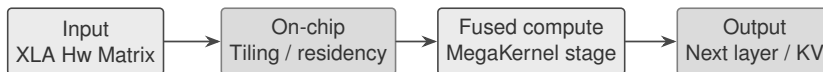


Figure 15.1: XLA Hw Matrix optimization pipeline in XLA: Production-Grade AI Compiler.



Figure 15.2: XLA Arch Passes block diagram for XLA: Production-Grade AI Compiler.

bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XLA Arch Passes is a production bottleneck surface for XLA. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XLA Codegen is a production bottleneck surface for XLA. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

15.2 XLA Arch Passes

XLA Arch Passes is a production bottleneck surface for XLA. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbench-

marks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XLA Hw Matrix is a production bottleneck surface for XLA. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XLA Arch Passes is a production bottleneck surface for XLA. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XLA Codegen is a production bottleneck surface for XLA. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

15.3 XLA Codegen

XLA Codegen is a production bottleneck surface for XLA. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XLA Hw Matrix is a production bottleneck surface for XLA. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XLA Arch Passes is a production bottleneck surface for XLA. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XLA Codegen is a production bottleneck surface for XLA. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

15.4 XLA Tuning

XLA Tuning is a production bottleneck surface for XLA. Teams that treat **tun**, **profile**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tun profile hardware pitfall to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XLA Hw Matrix is a production bottleneck surface for XLA. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XLA Arch Passes is a production bottleneck surface for XLA. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XLA Codegen is a production bottleneck surface for XLA. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

15.5 XLA Case Study

XLA Case Study is a production bottleneck surface for XLA. Teams that treat **case**, **benchmark**, **cross-hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds case benchmark cross-hardware YiRage to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XLA Hw Matrix is a production bottleneck surface for XLA. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XLA Arch Passes is a production bottleneck surface for XLA. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XLA Codegen is a production bottleneck surface for XLA. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

15.6 Chapter Summary

The through-line for ch15 is unchanged: hardware constraints dictate compiler passes, fusion boundaries, and deployment metrics (??). Carry bytes/token and launches/token spreadsheets forward; later chapters assume those baselines exist (??).

15.6.1 Engineering Deep Dive

XLA Hw Matrix is a production bottleneck surface for XLA. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XLA Arch Passes is a production bottleneck surface for XLA. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XLA Codegen is a production bottleneck surface for XLA. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XLA Tuning is a production bottleneck surface for XLA. Teams that treat **tun**, **profile**, **hardware** as independent knobs routinely ship kernels that win

microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tun profile hardware pitfall to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XLA Case Study is a production bottleneck surface for XLA. Teams that treat **case**, **benchmark**, **cross-hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds case benchmark cross-hardware YiRage to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XLA Hw Matrix is a production bottleneck surface for XLA. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XLA Arch Passes is a production bottleneck surface for XLA. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XLA Codegen is a production bottleneck surface for XLA. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and

launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XLA Tuning is a production bottleneck surface for XLA. Teams that treat **tun**, **profile**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tun profile hardware pitfall to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XLA Case Study is a production bottleneck surface for XLA. Teams that treat **case**, **benchmark**, **cross-hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds case benchmark cross-hardware YiRage to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XLA Hw Matrix is a production bottleneck surface for XLA. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XLA Arch Passes is a production bottleneck surface for XLA. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion

fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XLA Codegen is a production bottleneck surface for XLA. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XLA Tuning is a production bottleneck surface for XLA. Teams that treat **tun**, **profile**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tun profile hardware pitfall to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XLA Case Study is a production bottleneck surface for XLA. Teams that treat **case**, **benchmark**, **cross-hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds case benchmark cross-hardware YiRage to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XLA Hw Matrix is a production bottleneck surface for XLA. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record

fleet measurements alongside compiler flags (??).

XLA Arch Passes is a production bottleneck surface for XLA. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XLA Codegen is a production bottleneck surface for XLA. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XLA Tuning is a production bottleneck surface for XLA. Teams that treat **tun**, **profile**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tun profile hardware pitfall to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XLA Case Study is a production bottleneck surface for XLA. Teams that treat **case**, **benchmark**, **cross-hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds case benchmark cross-hardware YiRage to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XLA Hw Matrix is a production bottleneck surface for XLA. Teams that treat

hardware, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XLA Arch Passes is a production bottleneck surface for XLA. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XLA Codegen is a production bottleneck surface for XLA. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XLA Tuning is a production bottleneck surface for XLA. Teams that treat **tun**, **profile**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tun profile hardware pitfall to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XLA Case Study is a production bottleneck surface for XLA. Teams that treat **case**, **benchmark**, **cross-hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The

hardware–software contract here binds case benchmark cross-hardware YiRage to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XLA Hw Matrix is a production bottleneck surface for XLA. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XLA Arch Passes is a production bottleneck surface for XLA. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XLA Codegen is a production bottleneck surface for XLA. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XLA Tuning is a production bottleneck surface for XLA. Teams that treat **tun**, **profile**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tun profile hardware pitfall to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers

the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XLA Case Study is a production bottleneck surface for XLA. Teams that treat **case**, **benchmark**, **cross-hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds case benchmark cross-hardware YiRage to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XLA Hw Matrix is a production bottleneck surface for XLA. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XLA Arch Passes is a production bottleneck surface for XLA. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XLA Codegen is a production bottleneck surface for XLA. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map

each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XLA Tuning is a production bottleneck surface for XLA. Teams that treat **tun, profile, hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tun profile hardware pitfall to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XLA Case Study is a production bottleneck surface for XLA. Teams that treat **case, benchmark, cross-hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds case benchmark cross-hardware YiRage to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XLA Hw Matrix is a production bottleneck surface for XLA. Teams that treat **hardware, target, matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XLA Arch Passes is a production bottleneck surface for XLA. Teams that treat **pass, lower, optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XLA Codegen is a production bottleneck surface for XLA. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

XLA Tuning is a production bottleneck surface for XLA. Teams that treat **tun**, **profile**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tun profile hardware pitfall to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Chapter 16

TVM and Apache TVM Stack

Production decode teams rarely fail for lack of framework features—they fail because compiler and kernel schedules violate residency, launch, and bandwidth contracts on the SKU they actually ship (??). This chapter continues the book’s problem-first thread: name the contradiction, trace it to hardware limits, then show how passes and megakernels recover SLO on GPU, CPU, and NPU fleets (??).

16.1 TVM Hw Matrix

TVM Hw Matrix is a production bottleneck surface for TVM and Apache TVM Stack. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

TVM Hw Matrix is a production bottleneck surface for TVM and Apache TVM Stack. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit



Figure 16.1: TVM Hw Matrix optimization pipeline in TVM and Apache TVM Stack.



Figure 16.2: TVM Arch Passes block diagram for TVM and Apache TVM Stack.

legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

TVM Arch Passes is a production bottleneck surface for TVM and Apache TVM Stack. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

TVM Codegen is a production bottleneck surface for TVM and Apache TVM Stack. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

16.2 TVM Arch Passes

TVM Arch Passes is a production bottleneck surface for TVM and Apache TVM Stack. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets

(??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

TVM Hw Matrix is a production bottleneck surface for TVM and Apache TVM Stack. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

TVM Arch Passes is a production bottleneck surface for TVM and Apache TVM Stack. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

TVM Codegen is a production bottleneck surface for TVM and Apache TVM Stack. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

16.3 TVM Codegen

TVM Codegen is a production bottleneck surface for TVM and Apache TVM Stack. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to

measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

TVM Hw Matrix is a production bottleneck surface for TVM and Apache TVM Stack. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

TVM Arch Passes is a production bottleneck surface for TVM and Apache TVM Stack. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

TVM Codegen is a production bottleneck surface for TVM and Apache TVM Stack. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

16.4 TVM Tuning

TVM Tuning is a production bottleneck surface for TVM and Apache TVM Stack. Teams that treat **tun**, **profile**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The

hardware–software contract here binds tun profile hardware pitfall to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

TVM Hw Matrix is a production bottleneck surface for TVM and Apache TVM Stack. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

TVM Arch Passes is a production bottleneck surface for TVM and Apache TVM Stack. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

TVM Codegen is a production bottleneck surface for TVM and Apache TVM Stack. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

16.5 TVM Case Study

TVM Case Study is a production bottleneck surface for TVM and Apache TVM Stack. Teams that treat **case**, **benchmark**, **cross-hardware** as independent

knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds case benchmark cross-hardware YiRage to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

TVM Hw Matrix is a production bottleneck surface for TVM and Apache TVM Stack. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

TVM Arch Passes is a production bottleneck surface for TVM and Apache TVM Stack. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

TVM Codegen is a production bottleneck surface for TVM and Apache TVM Stack. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

16.6 Chapter Summary

The through-line for ch16 is unchanged: hardware constraints dictate compiler passes, fusion boundaries, and deployment metrics (??). Carry bytes/token and launches/token spreadsheets forward; later chapters assume those baselines exist (??).

16.6.1 Engineering Deep Dive

TVM Hw Matrix is a production bottleneck surface for TVM and Apache TVM Stack. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

TVM Arch Passes is a production bottleneck surface for TVM and Apache TVM Stack. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

TVM Codegen is a production bottleneck surface for TVM and Apache TVM Stack. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

TVM Tuning is a production bottleneck surface for TVM and Apache TVM Stack. Teams that treat **tun**, **profile**, **hardware** as independent knobs routinely

ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tun profile hardware pitfall to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

TVM Case Study is a production bottleneck surface for TVM and Apache TVM Stack. Teams that treat **case**, **benchmark**, **cross-hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds case benchmark cross-hardware YiRage to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

TVM Hw Matrix is a production bottleneck surface for TVM and Apache TVM Stack. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

TVM Arch Passes is a production bottleneck surface for TVM and Apache TVM Stack. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

TVM Codegen is a production bottleneck surface for TVM and Apache TVM Stack. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets

(??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

TVM Tuning is a production bottleneck surface for TVM and Apache TVM Stack. Teams that treat **tun**, **profile**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tun profile hardware pitfall to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

TVM Case Study is a production bottleneck surface for TVM and Apache TVM Stack. Teams that treat **case**, **benchmark**, **cross-hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds case benchmark cross-hardware YiRage to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

TVM Hw Matrix is a production bottleneck surface for TVM and Apache TVM Stack. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

TVM Arch Passes is a production bottleneck surface for TVM and Apache TVM Stack. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable

bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

TVM Codegen is a production bottleneck surface for TVM and Apache TVM Stack. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

TVM Tuning is a production bottleneck surface for TVM and Apache TVM Stack. Teams that treat **tun**, **profile**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tun profile hardware pitfall to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

TVM Case Study is a production bottleneck surface for TVM and Apache TVM Stack. Teams that treat **case**, **benchmark**, **cross-hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds case benchmark cross-hardware YiRage to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

TVM Hw Matrix is a production bottleneck surface for TVM and Apache TVM Stack. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across

GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

TVM Arch Passes is a production bottleneck surface for TVM and Apache TVM Stack. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

TVM Codegen is a production bottleneck surface for TVM and Apache TVM Stack. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

TVM Tuning is a production bottleneck surface for TVM and Apache TVM Stack. Teams that treat **tun**, **profile**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tun profile hardware pitfall to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

TVM Case Study is a production bottleneck surface for TVM and Apache TVM Stack. Teams that treat **case**, **benchmark**, **cross-hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds case benchmark cross-hardware YiRage to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than

in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

TVM Hw Matrix is a production bottleneck surface for TVM and Apache TVM Stack. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

TVM Arch Passes is a production bottleneck surface for TVM and Apache TVM Stack. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

TVM Codegen is a production bottleneck surface for TVM and Apache TVM Stack. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

TVM Tuning is a production bottleneck surface for TVM and Apache TVM Stack. Teams that treat **tun**, **profile**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tun profile hardware pitfall to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix

and record fleet measurements alongside compiler flags (??).

TVM Case Study is a production bottleneck surface for TVM and Apache TVM Stack. Teams that treat **case**, **benchmark**, **cross-hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds case benchmark cross-hardware YiRage to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

TVM Hw Matrix is a production bottleneck surface for TVM and Apache TVM Stack. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

TVM Arch Passes is a production bottleneck surface for TVM and Apache TVM Stack. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

TVM Codegen is a production bottleneck surface for TVM and Apache TVM Stack. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

TVM Tuning is a production bottleneck surface for TVM and Apache TVM Stack. Teams that treat **tun**, **profile**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tun profile hardware pitfall to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

TVM Case Study is a production bottleneck surface for TVM and Apache TVM Stack. Teams that treat **case**, **benchmark**, **cross-hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds case benchmark cross-hardware YiRage to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

TVM Hw Matrix is a production bottleneck surface for TVM and Apache TVM Stack. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

TVM Arch Passes is a production bottleneck surface for TVM and Apache TVM Stack. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

TVM Codegen is a production bottleneck surface for TVM and Apache TVM

Stack. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

TVM Tuning is a production bottleneck surface for TVM and Apache TVM Stack. Teams that treat **tun**, **profile**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tun profile hardware pitfall to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

TVM Case Study is a production bottleneck surface for TVM and Apache TVM Stack. Teams that treat **case**, **benchmark**, **cross-hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds case benchmark cross-hardware YiRage to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

TVM Hw Matrix is a production bottleneck surface for TVM and Apache TVM Stack. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

TVM Arch Passes is a production bottleneck surface for TVM and Apache TVM Stack. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely

ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

TVM Codegen is a production bottleneck surface for TVM and Apache TVM Stack. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Chapter 17

OpenAI Triton: Pythonic GPU Kernel Compiler

Production decode teams rarely fail for lack of framework features—they fail because compiler and kernel schedules violate residency, launch, and bandwidth contracts on the SKU they actually ship (??). This chapter continues the book’s problem-first thread: name the contradiction, trace it to hardware limits, then show how passes and megakernels recover SLO on GPU, CPU, and NPU fleets (??).

17.1 Triton Hw Matrix

Triton Hw Matrix is a production bottleneck surface for OpenAI Triton. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Triton Hw Matrix is a production bottleneck surface for OpenAI Triton. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable

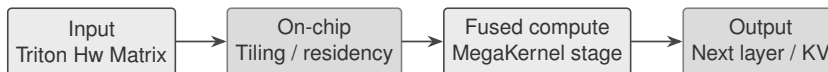


Figure 17.1: Triton Hw Matrix optimization pipeline in OpenAI Triton: Pythonic GPU Kernel Compiler.



Figure 17.2: Triton Arch Passes block diagram for OpenAI Triton: Pythonic GPU Kernel Compiler.

bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Triton Arch Passes is a production bottleneck surface for OpenAI Triton. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Triton Codegen is a production bottleneck surface for OpenAI Triton. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

17.2 Triton Arch Passes

Triton Arch Passes is a production bottleneck surface for OpenAI Triton. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Triton Hw Matrix is a production bottleneck surface for OpenAI Triton. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Triton Arch Passes is a production bottleneck surface for OpenAI Triton. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Triton Codegen is a production bottleneck surface for OpenAI Triton. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

17.3 Triton Codegen

Triton Codegen is a production bottleneck surface for OpenAI Triton. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Triton Hw Matrix is a production bottleneck surface for OpenAI Triton. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Triton Arch Passes is a production bottleneck surface for OpenAI Triton. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Triton Codegen is a production bottleneck surface for OpenAI Triton. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

17.4 Triton Tuning

Triton Tuning is a production bottleneck surface for OpenAI Triton. Teams that treat **tun**, **profile**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tun profile hardware pitfall to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Triton Hw Matrix is a production bottleneck surface for OpenAI Triton. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Triton Arch Passes is a production bottleneck surface for OpenAI Triton. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Triton Codegen is a production bottleneck surface for OpenAI Triton. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

17.5 Triton Case Study

Triton Case Study is a production bottleneck surface for OpenAI Triton. Teams that treat **case**, **benchmark**, **cross-hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds case benchmark cross-hardware YiRage to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Triton Hw Matrix is a production bottleneck surface for OpenAI Triton. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Triton Arch Passes is a production bottleneck surface for OpenAI Triton. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Triton Codegen is a production bottleneck surface for OpenAI Triton. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

17.6 Chapter Summary

The through-line for ch17 is unchanged: hardware constraints dictate compiler passes, fusion boundaries, and deployment metrics (??). Carry bytes/token and launches/token spreadsheets forward; later chapters assume those baselines exist (??).

17.6.1 Engineering Deep Dive

Triton Hw Matrix is a production bottleneck surface for OpenAI Triton. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Triton Arch Passes is a production bottleneck surface for OpenAI Triton. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Triton Codegen is a production bottleneck surface for OpenAI Triton. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Triton Tuning is a production bottleneck surface for OpenAI Triton. Teams that treat **tun**, **profile**, **hardware** as independent knobs routinely ship kernels

that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tun profile hardware pitfall to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Triton Case Study is a production bottleneck surface for OpenAI Triton. Teams that treat **case**, **benchmark**, **cross-hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds case benchmark cross-hardware YiRage to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Triton Hw Matrix is a production bottleneck surface for OpenAI Triton. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Triton Arch Passes is a production bottleneck surface for OpenAI Triton. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Triton Codegen is a production bottleneck surface for OpenAI Triton. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to

measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Triton Tuning is a production bottleneck surface for OpenAI Triton. Teams that treat **tun**, **profile**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tun profile hardware pitfall to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Triton Case Study is a production bottleneck surface for OpenAI Triton. Teams that treat **case**, **benchmark**, **cross-hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds case benchmark cross-hardware YiRage to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Triton Hw Matrix is a production bottleneck surface for OpenAI Triton. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Triton Arch Passes is a production bottleneck surface for OpenAI Triton. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal

fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Triton Codegen is a production bottleneck surface for OpenAI Triton. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Triton Tuning is a production bottleneck surface for OpenAI Triton. Teams that treat **tun**, **profile**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tun profile hardware pitfall to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Triton Case Study is a production bottleneck surface for OpenAI Triton. Teams that treat **case**, **benchmark**, **cross-hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds case benchmark cross-hardware YiRage to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Triton Hw Matrix is a production bottleneck surface for OpenAI Triton. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record

fleet measurements alongside compiler flags (??).

Triton Arch Passes is a production bottleneck surface for OpenAI Triton. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Triton Codegen is a production bottleneck surface for OpenAI Triton. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Triton Tuning is a production bottleneck surface for OpenAI Triton. Teams that treat **tun**, **profile**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tun profile hardware pitfall to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Triton Case Study is a production bottleneck surface for OpenAI Triton. Teams that treat **case**, **benchmark**, **cross-hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds case benchmark cross-hardware YiRage to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Triton Hw Matrix is a production bottleneck surface for OpenAI Triton. Teams

that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Triton Arch Passes is a production bottleneck surface for OpenAI Triton. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Triton Codegen is a production bottleneck surface for OpenAI Triton. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Triton Tuning is a production bottleneck surface for OpenAI Triton. Teams that treat **tun**, **profile**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tun profile hardware pitfall to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Triton Case Study is a production bottleneck surface for OpenAI Triton. Teams that treat **case**, **benchmark**, **cross-hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??).

The hardware–software contract here binds case benchmark cross-hardware YiRage to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Triton Hw Matrix is a production bottleneck surface for OpenAI Triton. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Triton Arch Passes is a production bottleneck surface for OpenAI Triton. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Triton Codegen is a production bottleneck surface for OpenAI Triton. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Triton Tuning is a production bottleneck surface for OpenAI Triton. Teams that treat **tun**, **profile**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tun profile hardware pitfall to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage

lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Triton Case Study is a production bottleneck surface for OpenAI Triton. Teams that treat **case**, **benchmark**, **cross-hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds case benchmark cross-hardware YiRage to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Triton Hw Matrix is a production bottleneck surface for OpenAI Triton. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Triton Arch Passes is a production bottleneck surface for OpenAI Triton. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Triton Codegen is a production bottleneck surface for OpenAI Triton. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards

(??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Triton Tuning is a production bottleneck surface for OpenAI Triton. Teams that treat **tun**, **profile**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tun profile hardware pitfall to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Triton Case Study is a production bottleneck surface for OpenAI Triton. Teams that treat **case**, **benchmark**, **cross-hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds case benchmark cross-hardware YiRage to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Triton Hw Matrix is a production bottleneck surface for OpenAI Triton. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Triton Arch Passes is a production bottleneck surface for OpenAI Triton. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Triton Codegen is a production bottleneck surface for OpenAI Triton. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Triton Tuning is a production bottleneck surface for OpenAI Triton. Teams that treat **tun**, **profile**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tun profile hardware pitfall to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Chapter 18

IREE: MLIR-Native Runtime and Compilation

Production decode teams rarely fail for lack of framework features—they fail because compiler and kernel schedules violate residency, launch, and bandwidth contracts on the SKU they actually ship (??). This chapter continues the book’s problem-first thread: name the contradiction, trace it to hardware limits, then show how passes and megakernels recover SLO on GPU, CPU, and NPU fleets (??).

18.1 IREE Hw Matrix

IREE Hw Matrix is a production bottleneck surface for IREE. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

IREE Hw Matrix is a production bottleneck surface for IREE. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable

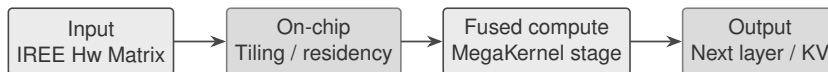


Figure 18.1: IREE Hw Matrix optimization pipeline in IREE: MLIR-Native Runtime and Compilation.



Figure 18.2: IREE Arch Passes block diagram for IREE: MLIR-Native Runtime and Compilation.

bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

IREE Arch Passes is a production bottleneck surface for IREE. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

IREE Codegen is a production bottleneck surface for IREE. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

18.2 IREE Arch Passes

IREE Arch Passes is a production bottleneck surface for IREE. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

IREE Hw Matrix is a production bottleneck surface for IREE. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

IREE Arch Passes is a production bottleneck surface for IREE. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

IREE Codegen is a production bottleneck surface for IREE. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

18.3 IREE Codegen

IREE Codegen is a production bottleneck surface for IREE. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

IREE Hw Matrix is a production bottleneck surface for IREE. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

IREE Arch Passes is a production bottleneck surface for IREE. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

IREE Codegen is a production bottleneck surface for IREE. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

18.4 IREE Tuning

IREE Tuning is a production bottleneck surface for IREE. Teams that treat **tun**, **profile**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tun profile hardware pitfall to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

IREE Hw Matrix is a production bottleneck surface for IREE. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

IREE Arch Passes is a production bottleneck surface for IREE. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

IREE Codegen is a production bottleneck surface for IREE. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

18.5 IREE Case Study

IREE Case Study is a production bottleneck surface for IREE. Teams that treat **case**, **benchmark**, **cross-hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds case benchmark cross-hardware YiRage to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

IREE Hw Matrix is a production bottleneck surface for IREE. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

IREE Arch Passes is a production bottleneck surface for IREE. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

IREE Codegen is a production bottleneck surface for IREE. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

18.6 Chapter Summary

The through-line for ch18 is unchanged: hardware constraints dictate compiler passes, fusion boundaries, and deployment metrics (??). Carry bytes/token and launches/token spreadsheets forward; later chapters assume those baselines exist (??).

18.6.1 Engineering Deep Dive

IREE Hw Matrix is a production bottleneck surface for IREE. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

IREE Arch Passes is a production bottleneck surface for IREE. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

IREE Codegen is a production bottleneck surface for IREE. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

IREE Tuning is a production bottleneck surface for IREE. Teams that treat **tun**, **profile**, **hardware** as independent knobs routinely ship kernels that win

microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds `tun` profile hardware pitfall to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

IREE Case Study is a production bottleneck surface for IREE. Teams that treat **case**, **benchmark**, **cross-hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds case benchmark cross-hardware YiRage to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

IREE Hw Matrix is a production bottleneck surface for IREE. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

IREE Arch Passes is a production bottleneck surface for IREE. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

IREE Codegen is a production bottleneck surface for IREE. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and

launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

IREE Tuning is a production bottleneck surface for IREE. Teams that treat **tun**, **profile**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tun profile hardware pitfall to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

IREE Case Study is a production bottleneck surface for IREE. Teams that treat **case**, **benchmark**, **cross-hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds case benchmark cross-hardware YiRage to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

IREE Hw Matrix is a production bottleneck surface for IREE. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

IREE Arch Passes is a production bottleneck surface for IREE. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion

fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

IREE Codegen is a production bottleneck surface for IREE. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

IREE Tuning is a production bottleneck surface for IREE. Teams that treat **tun**, **profile**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tun profile hardware pitfall to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

IREE Case Study is a production bottleneck surface for IREE. Teams that treat **case**, **benchmark**, **cross-hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds case benchmark cross-hardware YiRage to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

IREE Hw Matrix is a production bottleneck surface for IREE. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record

fleet measurements alongside compiler flags (??).

IREE Arch Passes is a production bottleneck surface for IREE. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

IREE Codegen is a production bottleneck surface for IREE. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

IREE Tuning is a production bottleneck surface for IREE. Teams that treat **tun**, **profile**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tun profile hardware pitfall to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

IREE Case Study is a production bottleneck surface for IREE. Teams that treat **case**, **benchmark**, **cross-hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds case benchmark cross-hardware YiRage to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

IREE Hw Matrix is a production bottleneck surface for IREE. Teams that

treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

IREE Arch Passes is a production bottleneck surface for IREE. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

IREE Codegen is a production bottleneck surface for IREE. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

IREE Tuning is a production bottleneck surface for IREE. Teams that treat **tun**, **profile**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tun profile hardware pitfall to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

IREE Case Study is a production bottleneck surface for IREE. Teams that treat **case**, **benchmark**, **cross-hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The

hardware–software contract here binds case benchmark cross-hardware YiRage to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

IREE Hw Matrix is a production bottleneck surface for IREE. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

IREE Arch Passes is a production bottleneck surface for IREE. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

IREE Codegen is a production bottleneck surface for IREE. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

IREE Tuning is a production bottleneck surface for IREE. Teams that treat **tun**, **profile**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tun profile hardware pitfall to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers

the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

IREE Case Study is a production bottleneck surface for IREE. Teams that treat **case**, **benchmark**, **cross-hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds case benchmark cross-hardware YiRage to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Chapter 19

Glow: Lightweight AI Compiler

Production decode teams rarely fail for lack of framework features—they fail because compiler and kernel schedules violate residency, launch, and bandwidth contracts on the SKU they actually ship (??). This chapter continues the book’s problem-first thread: name the contradiction, trace it to hardware limits, then show how passes and megakernels recover SLO on GPU, CPU, and NPU fleets (??).

19.1 Glow Hw Matrix

Glow Hw Matrix is a production bottleneck surface for Glow. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Glow Hw Matrix is a production bottleneck surface for Glow. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen,



Figure 19.1: Glow Hw Matrix optimization pipeline in Glow: Lightweight AI Compiler.



Figure 19.2: Glow Arch Passes block diagram for Glow: Lightweight AI Compiler.

so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Glow Arch Passes is a production bottleneck surface for Glow. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Glow Codegen is a production bottleneck surface for Glow. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

19.2 Glow Arch Passes

Glow Arch Passes is a production bottleneck surface for Glow. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers

the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Glow Hw Matrix is a production bottleneck surface for Glow. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Glow Arch Passes is a production bottleneck surface for Glow. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Glow Codegen is a production bottleneck surface for Glow. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

19.3 Glow Codegen

Glow Codegen is a production bottleneck surface for Glow. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and

launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Glow Hw Matrix is a production bottleneck surface for Glow. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Glow Arch Passes is a production bottleneck surface for Glow. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Glow Codegen is a production bottleneck surface for Glow. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

19.4 Glow Tuning

Glow Tuning is a production bottleneck surface for Glow. Teams that treat **tun**, **profile**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software

contract here binds tun profile hardware pitfall to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Glow Hw Matrix is a production bottleneck surface for Glow. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Glow Arch Passes is a production bottleneck surface for Glow. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Glow Codegen is a production bottleneck surface for Glow. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

19.5 Glow Case Study

Glow Case Study is a production bottleneck surface for Glow. Teams that treat **case**, **benchmark**, **cross-hardware** as independent knobs routinely ship kernels

that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds case benchmark cross-hardware YiRage to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Glow Hw Matrix is a production bottleneck surface for Glow. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Glow Arch Passes is a production bottleneck surface for Glow. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Glow Codegen is a production bottleneck surface for Glow. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

19.6 Chapter Summary

The through-line for ch19 is unchanged: hardware constraints dictate compiler passes, fusion boundaries, and deployment metrics (??). Carry bytes/token and launches/token spreadsheets forward; later chapters assume those baselines exist (??).

19.6.1 Engineering Deep Dive

Glow Hw Matrix is a production bottleneck surface for Glow. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Glow Arch Passes is a production bottleneck surface for Glow. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Glow Codegen is a production bottleneck surface for Glow. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Glow Tuning is a production bottleneck surface for Glow. Teams that treat **tun**, **profile**, **hardware** as independent knobs routinely ship kernels that win

microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds `tun` profile hardware pitfall to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Glow Case Study is a production bottleneck surface for Glow. Teams that treat **case**, **benchmark**, **cross-hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds case benchmark cross-hardware YiRage to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Glow Hw Matrix is a production bottleneck surface for Glow. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Glow Arch Passes is a production bottleneck surface for Glow. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Glow Codegen is a production bottleneck surface for Glow. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and

launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Glow Tuning is a production bottleneck surface for Glow. Teams that treat **tun**, **profile**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tun profile hardware pitfall to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Glow Case Study is a production bottleneck surface for Glow. Teams that treat **case**, **benchmark**, **cross-hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds case benchmark cross-hardware YiRage to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Glow Hw Matrix is a production bottleneck surface for Glow. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Glow Arch Passes is a production bottleneck surface for Glow. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion

fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Glow Codegen is a production bottleneck surface for Glow. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Glow Tuning is a production bottleneck surface for Glow. Teams that treat **tun**, **profile**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tun profile hardware pitfall to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Glow Case Study is a production bottleneck surface for Glow. Teams that treat **case**, **benchmark**, **cross-hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds case benchmark cross-hardware YiRage to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Glow Hw Matrix is a production bottleneck surface for Glow. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record

fleet measurements alongside compiler flags (??).

Glow Arch Passes is a production bottleneck surface for Glow. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Glow Codegen is a production bottleneck surface for Glow. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Glow Tuning is a production bottleneck surface for Glow. Teams that treat **tun**, **profile**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tun profile hardware pitfall to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Glow Case Study is a production bottleneck surface for Glow. Teams that treat **case**, **benchmark**, **cross-hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds case benchmark cross-hardware YiRage to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Glow Hw Matrix is a production bottleneck surface for Glow. Teams that

treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Glow Arch Passes is a production bottleneck surface for Glow. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Glow Codegen is a production bottleneck surface for Glow. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Glow Tuning is a production bottleneck surface for Glow. Teams that treat **tun**, **profile**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tun profile hardware pitfall to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Glow Case Study is a production bottleneck surface for Glow. Teams that treat **case**, **benchmark**, **cross-hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The

hardware–software contract here binds case benchmark cross-hardware YiRage to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Glow Hw Matrix is a production bottleneck surface for Glow. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Glow Arch Passes is a production bottleneck surface for Glow. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Glow Codegen is a production bottleneck surface for Glow. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Glow Tuning is a production bottleneck surface for Glow. Teams that treat **tun**, **profile**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tun profile hardware pitfall to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers

the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Glow Case Study is a production bottleneck surface for Glow. Teams that treat **case**, **benchmark**, **cross-hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds case benchmark cross-hardware YiRage to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Chapter 20

Mirage and Emerging AI Compilers

Production decode teams rarely fail for lack of framework features—they fail because compiler and kernel schedules violate residency, launch, and bandwidth contracts on the SKU they actually ship (??). This chapter continues the book’s problem-first thread: name the contradiction, trace it to hardware limits, then show how passes and megakernels recover SLO on GPU, CPU, and NPU fleets (??).

20.1 Mirage Hw Matrix

Mirage Hw Matrix is a production bottleneck surface for Mirage and Emerging AI Compilers. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Mirage Hw Matrix is a production bottleneck surface for Mirage and Emerging AI Compilers. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix

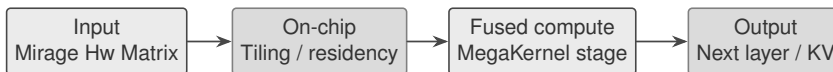


Figure 20.1: Mirage Hw Matrix optimization pipeline in Mirage and Emerging AI Compilers.



Figure 20.2: Mirage Arch Passes block diagram for Mirage and Emerging AI Compilers.

GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Mirage Arch Passes is a production bottleneck surface for Mirage and Emerging AI Compilers. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Mirage Codegen is a production bottleneck surface for Mirage and Emerging AI Compilers. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

20.2 Mirage Arch Passes

Mirage Arch Passes is a production bottleneck surface for Mirage and Emerging AI Compilers. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely

ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Mirage Hw Matrix is a production bottleneck surface for Mirage and Emerging AI Compilers. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Mirage Arch Passes is a production bottleneck surface for Mirage and Emerging AI Compilers. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Mirage Codegen is a production bottleneck surface for Mirage and Emerging AI Compilers. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

20.3 Mirage Codegen

Mirage Codegen is a production bottleneck surface for Mirage and Emerging AI Compilers. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Mirage Hw Matrix is a production bottleneck surface for Mirage and Emerging AI Compilers. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Mirage Arch Passes is a production bottleneck surface for Mirage and Emerging AI Compilers. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Mirage Codegen is a production bottleneck surface for Mirage and Emerging AI Compilers. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

20.4 Mirage Tuning

Mirage Tuning is a production bottleneck surface for Mirage and Emerging AI Compilers. Teams that treat **tun**, **profile**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tun profile hardware pitfall to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Mirage Hw Matrix is a production bottleneck surface for Mirage and Emerging AI Compilers. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Mirage Arch Passes is a production bottleneck surface for Mirage and Emerging AI Compilers. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Mirage Codegen is a production bottleneck surface for Mirage and Emerging AI Compilers. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

20.5 Mirage Case Study

Mirage Case Study is a production bottleneck surface for Mirage and Emerging AI Compilers. Teams that treat **case**, **benchmark**, **cross-hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds case benchmark cross-hardware YiRage to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Mirage Hw Matrix is a production bottleneck surface for Mirage and Emerging AI Compilers. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Mirage Arch Passes is a production bottleneck surface for Mirage and Emerging AI Compilers. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Mirage Codegen is a production bottleneck surface for Mirage and Emerging AI Compilers. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3

constraint matrix and record fleet measurements alongside compiler flags (??).

20.6 Chapter Summary

The through-line for ch20 is unchanged: hardware constraints dictate compiler passes, fusion boundaries, and deployment metrics (??). Carry bytes/token and launches/token spreadsheets forward; later chapters assume those baselines exist (??).

20.6.1 Engineering Deep Dive

Mirage Hw Matrix is a production bottleneck surface for Mirage and Emerging AI Compilers. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Mirage Arch Passes is a production bottleneck surface for Mirage and Emerging AI Compilers. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Mirage Codegen is a production bottleneck surface for Mirage and Emerging AI Compilers. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Mirage Tuning is a production bottleneck surface for Mirage and Emerging AI Compilers. Teams that treat **tun**, **profile**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tun profile hardware pitfall to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Mirage Case Study is a production bottleneck surface for Mirage and Emerging AI Compilers. Teams that treat **case**, **benchmark**, **cross-hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds case benchmark cross-hardware YiRage to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Mirage Hw Matrix is a production bottleneck surface for Mirage and Emerging AI Compilers. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Mirage Arch Passes is a production bottleneck surface for Mirage and Emerging AI Compilers. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Mirage Codegen is a production bottleneck surface for Mirage and Emerging

AI Compilers. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Mirage Tuning is a production bottleneck surface for Mirage and Emerging AI Compilers. Teams that treat **tun**, **profile**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tun profile hardware pitfall to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Mirage Case Study is a production bottleneck surface for Mirage and Emerging AI Compilers. Teams that treat **case**, **benchmark**, **cross-hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds case benchmark cross-hardware YiRage to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Mirage Hw Matrix is a production bottleneck surface for Mirage and Emerging AI Compilers. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Mirage Arch Passes is a production bottleneck surface for Mirage and Emerging AI Compilers. Teams that treat **pass**, **lower**, **optim** as independent knobs

routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Mirage Codegen is a production bottleneck surface for Mirage and Emerging AI Compilers. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Mirage Tuning is a production bottleneck surface for Mirage and Emerging AI Compilers. Teams that treat **tun**, **profile**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tun profile hardware pitfall to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Mirage Case Study is a production bottleneck surface for Mirage and Emerging AI Compilers. Teams that treat **case**, **benchmark**, **cross-hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds case benchmark cross-hardware YiRage to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Mirage Hw Matrix is a production bottleneck surface for Mirage and Emerging AI Compilers. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real

fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Mirage Arch Passes is a production bottleneck surface for Mirage and Emerging AI Compilers. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Mirage Codegen is a production bottleneck surface for Mirage and Emerging AI Compilers. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Mirage Tuning is a production bottleneck surface for Mirage and Emerging AI Compilers. Teams that treat **tun**, **profile**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tun profile hardware pitfall to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Mirage Case Study is a production bottleneck surface for Mirage and Emerging AI Compilers. Teams that treat **case**, **benchmark**, **cross-hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds case benchmark cross-hardware YiRage to measurable bytes/token and launches/token outcomes

across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Mirage Hw Matrix is a production bottleneck surface for Mirage and Emerging AI Compilers. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Mirage Arch Passes is a production bottleneck surface for Mirage and Emerging AI Compilers. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Mirage Codegen is a production bottleneck surface for Mirage and Emerging AI Compilers. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Mirage Tuning is a production bottleneck surface for Mirage and Emerging AI Compilers. Teams that treat **tun**, **profile**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tun profile hardware pitfall to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks

before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Mirage Case Study is a production bottleneck surface for Mirage and Emerging AI Compilers. Teams that treat **case**, **benchmark**, **cross-hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds case benchmark cross-hardware YiRage to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Mirage Hw Matrix is a production bottleneck surface for Mirage and Emerging AI Compilers. Teams that treat **hardware**, **target**, **matrix** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware target matrix GPU|CPU|NPU to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Mirage Arch Passes is a production bottleneck surface for Mirage and Emerging AI Compilers. Teams that treat **pass**, **lower**, **optim** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pass lower optim dialect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Mirage Codegen is a production bottleneck surface for Mirage and Emerging AI Compilers. Teams that treat **codegen**, **backend**, **platform** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds codegen backend platform emit to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer

dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Mirage Tuning is a production bottleneck surface for Mirage and Emerging AI Compilers. Teams that treat **tun**, **profile**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds tun profile hardware pitfall to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Chapter 21

Unified AI Compiler Analysis and Cross-Hardware Benchmark

Production decode teams rarely fail for lack of framework features—they fail because compiler and kernel schedules violate residency, launch, and bandwidth contracts on the SKU they actually ship (??). This chapter continues the book’s problem-first thread: name the contradiction, trace it to hardware limits, then show how passes and megakernels recover SLO on GPU, CPU, and NPU fleets (??).

21.1 Compiler Ratings

Compiler Ratings is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **compiler, rating** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds compiler rating to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Compiler Ratings is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **compiler, rating** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds

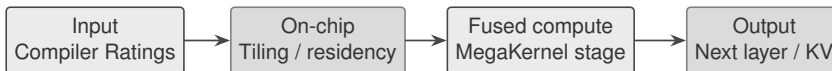


Figure 21.1: Compiler Ratings optimization pipeline in Unified AI Compiler Analysis and Cross-Hardware Benchmark.



Figure 21.2: Compile Overhead block diagram for Unified AI Compiler Analysis and Cross-Hardware Benchmark.

compiler rating to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Compile Overhead is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **compile**, **overhead** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds compile overhead to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hw Binding is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **hardware**, **binding** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware binding to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

21.2 Compile Overhead

Compile Overhead is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **compile**, **overhead** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds compile overhead to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Compiler Ratings is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **compiler**, **rating** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds compiler rating to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Compile Overhead is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **compile**, **overhead** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds compile overhead to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hw Binding is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **hardware**, **binding** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware binding to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

21.3 Hw Binding

Hw Binding is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **hardware, binding** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware binding to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Compiler Ratings is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **compiler, rating** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds compiler rating to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Compile Overhead is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **compile, overhead** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds compile overhead to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hw Binding is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **hardware, binding** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware binding to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

21.4 Search Space

Search Space is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **search**, **autotun** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds search autotun to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Compiler Ratings is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **compiler**, **rating** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds compiler rating to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Compile Overhead is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **compile**, **overhead** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds compile overhead to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hw Binding is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **hardware**, **binding** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware binding to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

21.5 YiRage Benchmark

YiRage Benchmark is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **YiRage, benchmark** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage benchmark to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Compiler Ratings is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **compiler, rating** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds compiler rating to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Compile Overhead is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **compile, overhead** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds compile overhead to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hw Binding is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **hardware, binding** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware binding to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

21.6 Chapter Summary

The through-line for ch21 is unchanged: hardware constraints dictate compiler passes, fusion boundaries, and deployment metrics (??). Carry bytes/token and launches/token spreadsheets forward; later chapters assume those baselines exist (??).

21.6.1 Engineering Deep Dive

Compiler Ratings is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **compiler**, **rating** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds compiler rating to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Compile Overhead is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **compile**, **overhead** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds compile overhead to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hw Binding is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **hardware**, **binding** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware binding to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Search Space is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **search**, **autotun** as

independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds search autotun to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Benchmark is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **YiRage, benchmark** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage benchmark to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Compiler Ratings is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **compiler, rating** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds compiler rating to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Compile Overhead is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **compile, overhead** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds compile overhead to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hw Binding is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **hardware, binding** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware

binding to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Search Space is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **search**, **autotun** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds search autotun to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Benchmark is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **YiRage**, **benchmark** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage benchmark to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Compiler Ratings is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **compiler**, **rating** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds compiler rating to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Compile Overhead is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **compile**, **overhead** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds compile overhead to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit

legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hw Binding is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **hardware**, **binding** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware binding to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Search Space is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **search**, **autotun** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds search autotun to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Benchmark is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **YiRage**, **benchmark** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage benchmark to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Compiler Ratings is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **compiler**, **rating** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds compiler rating to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3

constraint matrix and record fleet measurements alongside compiler flags (??).

Compile Overhead is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **compile**, **overhead** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds compile overhead to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hw Binding is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **hardware**, **binding** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware binding to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Search Space is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **search**, **autotun** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds search autotun to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Benchmark is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **YiRage**, **benchmark** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage benchmark to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Compiler Ratings is a production bottleneck surface for Unified AI Compiler

Analysis and Cross-Hardware Benchmark. Teams that treat **compiler, rating** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds compiler rating to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Compile Overhead is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **compile, overhead** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds compile overhead to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hw Binding is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **hardware, binding** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware binding to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Search Space is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **search, autotun** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds search autotun to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Benchmark is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **YiRage, benchmark** as independent knobs routinely ship kernels that win microbenchmarks yet miss

decode SLOs on real fleets (??). The hardware–software contract here binds YiRage benchmark to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Compiler Ratings is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **compiler, rating** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds compiler rating to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Compile Overhead is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **compile, overhead** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds compile overhead to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hw Binding is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **hardware, binding** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware binding to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Search Space is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **search, autotun** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds search autotun to measurable bytes/token and launches/token outcomes across GPU,

CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Benchmark is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **YiRage, benchmark** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage benchmark to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Compiler Ratings is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **compiler, rating** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds compiler rating to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Compile Overhead is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **compile, overhead** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds compile overhead to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hw Binding is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **hardware, binding** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware binding to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer

dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Search Space is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **search**, **autotun** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds search autotun to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Benchmark is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **YiRage**, **benchmark** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage benchmark to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Compiler Ratings is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **compiler**, **rating** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds compiler rating to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Compile Overhead is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **compile**, **overhead** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds compile overhead to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hw Binding is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **hardware, binding** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware binding to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Search Space is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **search, autotun** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds search autotun to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Benchmark is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **YiRage, benchmark** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage benchmark to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Compiler Ratings is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **compiler, rating** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds compiler rating to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Compile Overhead is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **compile, overhead**

as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds compile overhead to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hw Binding is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **hardware, binding** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware binding to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Search Space is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **search, autotun** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds search autotun to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Benchmark is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **YiRage, benchmark** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage benchmark to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Compiler Ratings is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **compiler, rating** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds

compiler rating to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Compile Overhead is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **compile**, **overhead** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds compile overhead to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hw Binding is a production bottleneck surface for Unified AI Compiler Analysis and Cross-Hardware Benchmark. Teams that treat **hardware**, **binding** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hardware binding to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Chapter 22

Compiler-Driven End-to-End Performance Tuning

Production decode teams rarely fail for lack of framework features—they fail because compiler and kernel schedules violate residency, launch, and bandwidth contracts on the SKU they actually ship (??). This chapter continues the book’s problem-first thread: name the contradiction, trace it to hardware limits, then show how passes and megakernels recover SLO on GPU, CPU, and NPU fleets (??).

22.1 Hw Aware Profiling

Hw Aware Profiling is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **profil**, **bandwidth** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds profil bandwidth to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hw Aware Profiling is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **profil**, **bandwidth** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds profil bandwidth

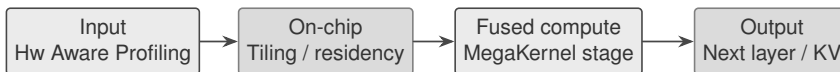


Figure 22.1: Hw Aware Profiling optimization pipeline in Compiler-Driven End-to-End Performance Tuning.



Figure 22.2: Bottleneck Locating block diagram for Compiler-Driven End-to-End Performance Tuning.

to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Bottleneck Locating is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **bottleneck** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds bottleneck to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Tuning Workflow is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **workflow**, **tun** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds workflow tun to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

22.2 Bottleneck Locating

Bottleneck Locating is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **bottleneck** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds bottleneck to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hw Aware Profiling is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **profil**, **bandwidth** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds profil bandwidth to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Bottleneck Locating is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **bottleneck** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds bottleneck to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Tuning Workflow is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **workflow**, **tun** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds workflow tun to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

22.3 Tuning Workflow

Tuning Workflow is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **workflow**, **tun** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds workflow tun to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hw Aware Profiling is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **profil**, **bandwidth** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds profil bandwidth to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Bottleneck Locating is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **bottleneck** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds bottleneck to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Tuning Workflow is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **workflow**, **tun** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds workflow tun to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

22.4 Regression Gates

Regression Gates is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **regression**, **gate** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds regression gate to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hw Aware Profiling is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **profil**, **bandwidth** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds profil bandwidth to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Bottleneck Locating is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **bottleneck** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds bottleneck to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Tuning Workflow is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **workflow**, **tun** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds workflow tun to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

22.5 Chapter Summary

The through-line for ch22 is unchanged: hardware constraints dictate compiler passes, fusion boundaries, and deployment metrics (??). Carry bytes/token and launches/token spreadsheets forward; later chapters assume those baselines exist (??).

22.5.1 Engineering Deep Dive

Hw Aware Profiling is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **profil**, **bandwidth** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds profil bandwidth to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Bottleneck Locating is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **bottleneck** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds bottleneck to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Tuning Workflow is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **workflow**, **tun** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds workflow tun to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Regression Gates is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **regression**, **gate** as independent

knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds regression gate to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hw Aware Profiling is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **profil**, **bandwidth** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds profil bandwidth to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Bottleneck Locating is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **bottleneck** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds bottleneck to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Tuning Workflow is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **workflow**, **tun** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds workflow tun to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Regression Gates is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **regression**, **gate** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds regression gate

to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hw Aware Profiling is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **profil**, **bandwidth** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds profil bandwidth to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Bottleneck Locating is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **bottleneck** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds bottleneck to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Tuning Workflow is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **workflow**, **tun** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds workflow tun to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Regression Gates is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **regression**, **gate** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds regression gate to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks

before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hw Aware Profiling is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **profil**, **bandwidth** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds profil bandwidth to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Bottleneck Locating is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **bottleneck** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds bottleneck to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Tuning Workflow is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **workflow**, **tun** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds workflow tun to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Regression Gates is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **regression**, **gate** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds regression gate to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint

matrix and record fleet measurements alongside compiler flags (??).

Hw Aware Profiling is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **profil**, **bandwidth** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds profil bandwidth to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Bottleneck Locating is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **bottleneck** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds bottleneck to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Tuning Workflow is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **workflow**, **tun** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds workflow tun to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Regression Gates is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **regression**, **gate** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds regression gate to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hw Aware Profiling is a production bottleneck surface for Compiler-Driven End-

to-End Performance Tuning. Teams that treat **profil**, **bandwidth** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds profil bandwidth to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Bottleneck Locating is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **bottleneck** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds bottleneck to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Tuning Workflow is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **workflow**, **tun** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds workflow tun to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Regression Gates is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **regression**, **gate** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds regression gate to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hw Aware Profiling is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **profil**, **bandwidth** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs

on real fleets (??). The hardware–software contract here binds profil bandwidth to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Bottleneck Locating is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **bottleneck** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds bottleneck to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Tuning Workflow is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **workflow**, **tun** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds workflow tun to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Regression Gates is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **regression**, **gate** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds regression gate to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hw Aware Profiling is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **profil**, **bandwidth** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds profil bandwidth to measurable bytes/token and launches/token outcomes across GPU, CPU, and

NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Bottleneck Locating is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **bottleneck** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds bottleneck to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Tuning Workflow is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **workflow**, **tun** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds workflow tun to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Regression Gates is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **regression**, **gate** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds regression gate to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hw Aware Profiling is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **profil**, **bandwidth** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds profil bandwidth to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards

(??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Bottleneck Locating is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **bottleneck** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds bottleneck to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Tuning Workflow is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **workflow**, **tun** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds workflow tun to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Regression Gates is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **regression**, **gate** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds regression gate to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hw Aware Profiling is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **profil**, **bandwidth** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds profil bandwidth to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Bottleneck Locating is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **bottleneck** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds bottleneck to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Tuning Workflow is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **workflow**, **tun** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds workflow tun to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Regression Gates is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **regression**, **gate** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds regression gate to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hw Aware Profiling is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **profil**, **bandwidth** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds profil bandwidth to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Bottleneck Locating is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **bottleneck** as independent

knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds bottleneck to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Tuning Workflow is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **workflow**, **tun** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds workflow tun to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Regression Gates is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **regression**, **gate** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds regression gate to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hw Aware Profiling is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **profil**, **bandwidth** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds profil bandwidth to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Bottleneck Locating is a production bottleneck surface for Compiler-Driven End-to-End Performance Tuning. Teams that treat **bottleneck** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds bottleneck to

measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Chapter 23

LLM and MoE Specialized Compilation

Production decode teams rarely fail for lack of framework features—they fail because compiler and kernel schedules violate residency, launch, and bandwidth contracts on the SKU they actually ship (??). This chapter continues the book’s problem-first thread: name the contradiction, trace it to hardware limits, then show how passes and megakernels recover SLO on GPU, CPU, and NPU fleets (??).

23.1 LLM Decode Compile

LLM Decode Compile is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **LLM**, **decode** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds LLM decode to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

LLM Decode Compile is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **LLM**, **decode** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds LLM decode to measurable

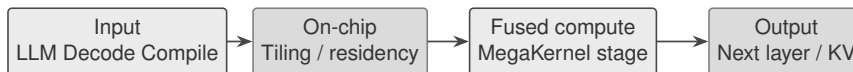


Figure 23.1: LLM Decode Compile optimization pipeline in LLM and MoE Specialized Compilation.



Figure 23.2: Moe Scheduling block diagram for LLM and MoE Specialized Compilation.

bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Moe Scheduling is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **moe**, **expert** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds moe expert to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Kv Hardware is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **kv**, **cache** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds kv cache to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

23.2 Moe Scheduling

Moe Scheduling is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **moe**, **expert** as independent knobs routinely ship

kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds moe expert to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

LLM Decode Compile is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **LLM**, **decode** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds LLM decode to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Moe Scheduling is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **moe**, **expert** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds moe expert to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Kv Hardware is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **kv**, **cache** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds kv cache to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

23.3 Kv Hardware

Kv Hardware is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **kv**, **cache** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds kv cache to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

LLM Decode Compile is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **LLM**, **decode** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds LLM decode to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Moe Scheduling is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **moe**, **expert** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds moe expert to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Kv Hardware is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **kv**, **cache** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds kv cache to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

23.4 Hetero Moe

Hetero Moe is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **hetero**, **load** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hetero load to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

LLM Decode Compile is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **LLM**, **decode** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds LLM decode to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Moe Scheduling is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **moe**, **expert** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds moe expert to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Kv Hardware is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **kv**, **cache** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds kv cache to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

23.5 Chapter Summary

The through-line for ch23 is unchanged: hardware constraints dictate compiler passes, fusion boundaries, and deployment metrics (??). Carry bytes/token and launches/token spreadsheets forward; later chapters assume those baselines exist (??).

23.5.1 Engineering Deep Dive

LLM Decode Compile is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **LLM**, **decode** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds LLM decode to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Moe Scheduling is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **moe**, **expert** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds moe expert to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Kv Hardware is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **kv**, **cache** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds kv cache to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hetero Moe is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **hetero**, **load** as independent knobs routinely ship

kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hetero load to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

LLM Decode Compile is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **LLM**, **decode** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds LLM decode to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Moe Scheduling is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **moe**, **expert** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds moe expert to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Kv Hardware is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **kv**, **cache** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds kv cache to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hetero Moe is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **hetero**, **load** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hetero load to measurable bytes/token and

launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

LLM Decode Compile is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **LLM**, **decode** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds LLM decode to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Moe Scheduling is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **moe**, **expert** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds moe expert to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Kv Hardware is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **kv**, **cache** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds kv cache to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hetero Moe is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **hetero**, **load** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hetero load to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion

fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

LLM Decode Compile is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **LLM**, **decode** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds LLM decode to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Moe Scheduling is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **moe**, **expert** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds moe expert to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Kv Hardware is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **kv**, **cache** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds kv cache to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hetero Moe is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **hetero**, **load** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hetero load to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements

alongside compiler flags (??).

LLM Decode Compile is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **LLM**, **decode** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds LLM decode to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Moe Scheduling is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **moe**, **expert** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds moe expert to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Kv Hardware is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **kv**, **cache** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds kv cache to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hetero Moe is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **hetero**, **load** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hetero load to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

LLM Decode Compile is a production bottleneck surface for LLM and MoE

Specialized Compilation. Teams that treat **LLM**, **decode** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds LLM decode to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Moe Scheduling is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **moe**, **expert** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds moe expert to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Kv Hardware is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **kv**, **cache** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds kv cache to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hetero Moe is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **hetero**, **load** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hetero load to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

LLM Decode Compile is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **LLM**, **decode** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real

fleets (??). The hardware–software contract here binds LLM decode to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Moe Scheduling is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **moe**, **expert** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds moe expert to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Kv Hardware is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **kv**, **cache** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds kv cache to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hetero Moe is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **hetero**, **load** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hetero load to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

LLM Decode Compile is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **LLM**, **decode** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds LLM decode to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets

(??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Moe Scheduling is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **moe**, **expert** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds moe expert to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Kv Hardware is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **kv**, **cache** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds kv cache to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hetero Moe is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **hetero**, **load** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hetero load to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

LLM Decode Compile is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **LLM**, **decode** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds LLM decode to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial

readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Moe Scheduling is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **moe**, **expert** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds moe expert to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Kv Hardware is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **kv**, **cache** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds kv cache to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hetero Moe is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **hetero**, **load** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hetero load to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

LLM Decode Compile is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **LLM**, **decode** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds LLM decode to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Moe Scheduling is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **moe**, **expert** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds moe expert to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Kv Hardware is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **kv**, **cache** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds kv cache to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hetero Moe is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **hetero**, **load** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hetero load to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

LLM Decode Compile is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **LLM**, **decode** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds LLM decode to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Moe Scheduling is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **moe**, **expert** as independent knobs routinely ship

kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds moe expert to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Kv Hardware is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **kv**, **cache** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds kv cache to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hetero Moe is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **hetero**, **load** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hetero load to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

LLM Decode Compile is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **LLM**, **decode** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds LLM decode to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Moe Scheduling is a production bottleneck surface for LLM and MoE Specialized Compilation. Teams that treat **moe**, **expert** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds moe expert to measurable bytes/token and

launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Chapter 24

Heterogeneous Compilation and Multi-Hardware Deployment

Production decode teams rarely fail for lack of framework features—they fail because compiler and kernel schedules violate residency, launch, and bandwidth contracts on the SKU they actually ship (??). This chapter continues the book’s problem-first thread: name the contradiction, trace it to hardware limits, then show how passes and megakernels recover SLO on GPU, CPU, and NPU fleets (??).

24.1 Hetero Cluster

Hetero Cluster is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **hetero**, **cluster** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hetero cluster to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hetero Cluster is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **hetero**, **cluster** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hetero cluster to

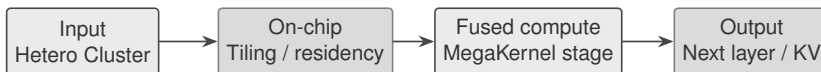


Figure 24.1: Hetero Cluster optimization pipeline in Heterogeneous Compilation and Multi-Hardware Deployment.



Figure 24.2: Unified Stack block diagram for Heterogeneous Compilation and Multi-Hardware Deployment.

measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Unified Stack is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **MLIR, unified** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds MLIR unified to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Runtime Adapt is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **runtime, detect** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds runtime detect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

24.2 Unified Stack

Unified Stack is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **MLIR**, **unified** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds MLIR unified to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hetero Cluster is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **hetero**, **cluster** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hetero cluster to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Unified Stack is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **MLIR**, **unified** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds MLIR unified to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Runtime Adapt is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **runtime**, **detect** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds runtime detect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

24.3 Runtime Adapt

Runtime Adapt is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **runtime**, **detect** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds runtime detect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hetero Cluster is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **hetero**, **cluster** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hetero cluster to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Unified Stack is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **MLIR**, **unified** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds MLIR unified to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Runtime Adapt is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **runtime**, **detect** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds runtime detect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

24.4 Version Regression

Version Regression is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **version**, **regression** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds version regression to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hetero Cluster is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **hetero**, **cluster** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hetero cluster to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Unified Stack is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **MLIR**, **unified** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds MLIR unified to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Runtime Adapt is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **runtime**, **detect** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds runtime detect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

24.5 Industrial Cases

Industrial Cases is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **case**, **production** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds case production to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hetero Cluster is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **hetero**, **cluster** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hetero cluster to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Unified Stack is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **MLIR**, **unified** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds MLIR unified to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Runtime Adapt is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **runtime**, **detect** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds runtime detect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

24.6 YiRage Cluster

YiRage Cluster is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **YiRage, cluster** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage cluster to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hetero Cluster is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **hetero, cluster** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hetero cluster to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Unified Stack is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **MLIR, unified** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds MLIR unified to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Runtime Adapt is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **runtime, detect** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds runtime detect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

24.7 Chapter Summary

The through-line for ch24 is unchanged: hardware constraints dictate compiler passes, fusion boundaries, and deployment metrics (??). Carry bytes/token and launches/token spreadsheets forward; later chapters assume those baselines exist (??).

24.7.1 Engineering Deep Dive

Hetero Cluster is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **hetero**, **cluster** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hetero cluster to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Unified Stack is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **MLIR**, **unified** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds MLIR unified to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Runtime Adapt is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **runtime**, **detect** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds runtime detect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Version Regression is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **version**, **regression**

as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds version regression to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Industrial Cases is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **case**, **production** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds case production to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Cluster is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **YiRage**, **cluster** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage cluster to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hetero Cluster is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **hetero**, **cluster** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hetero cluster to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Unified Stack is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **MLIR**, **unified** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds MLIR unified to

measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Runtime Adapt is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **runtime**, **detect** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds runtime detect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Version Regression is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **version**, **regression** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds version regression to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Industrial Cases is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **case**, **production** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds case production to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Cluster is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **YiRage**, **cluster** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage cluster to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit

legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hetero Cluster is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **hetero**, **cluster** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hetero cluster to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Unified Stack is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **MLIR**, **unified** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds MLIR unified to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Runtime Adapt is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **runtime**, **detect** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds runtime detect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Version Regression is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **version**, **regression** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds version regression to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3

constraint matrix and record fleet measurements alongside compiler flags (??).

Industrial Cases is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **case**, **production** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds case production to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Cluster is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **YiRage**, **cluster** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage cluster to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hetero Cluster is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **hetero**, **cluster** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hetero cluster to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Unified Stack is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **MLIR**, **unified** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds MLIR unified to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Runtime Adapt is a production bottleneck surface for Heterogeneous Com-

pilation and Multi-Hardware Deployment. Teams that treat **runtime**, **detect** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds runtime detect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Version Regression is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **version**, **regression** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds version regression to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Industrial Cases is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **case**, **production** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds case production to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Cluster is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **YiRage**, **cluster** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage cluster to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hetero Cluster is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **hetero**, **cluster** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs

on real fleets (??). The hardware–software contract here binds hetero cluster to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Unified Stack is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **MLIR, unified** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds MLIR unified to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Runtime Adapt is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **runtime, detect** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds runtime detect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Version Regression is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **version, regression** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds version regression to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Industrial Cases is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **case, production** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds case production to measurable bytes/token and launches/token outcomes across GPU,

CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Cluster is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **YiRage**, **cluster** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage cluster to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hetero Cluster is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **hetero**, **cluster** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hetero cluster to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Unified Stack is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **MLIR**, **unified** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds MLIR unified to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Runtime Adapt is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **runtime**, **detect** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds runtime detect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer

dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Version Regression is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **version**, **regression** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds version regression to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Industrial Cases is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **case**, **production** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds case production to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Cluster is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **YiRage**, **cluster** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage cluster to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hetero Cluster is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **hetero**, **cluster** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds hetero cluster to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Unified Stack is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **MLIR, unified** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds MLIR unified to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Runtime Adapt is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **runtime, detect** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds runtime detect to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Version Regression is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **version, regression** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds version regression to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Industrial Cases is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **case, production** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds case production to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Cluster is a production bottleneck surface for Heterogeneous Compilation and Multi-Hardware Deployment. Teams that treat **YiRage, cluster**

as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage cluster to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Chapter 25

Auto-Optimization and AI-Assisted Compilation

Production decode teams rarely fail for lack of framework features—they fail because compiler and kernel schedules violate residency, launch, and bandwidth contracts on the SKU they actually ship (??). This chapter continues the book’s problem-first thread: name the contradiction, trace it to hardware limits, then show how passes and megakernels recover SLO on GPU, CPU, and NPU fleets (??).

25.1 Rl Autotune

Rl Autotune is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **rl**, **autotun** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds rl autotun to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Rl Autotune is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **rl**, **autotun** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds rl autotun to measurable



Figure 25.1: RL Autotune optimization pipeline in Auto-Optimization and AI-Assisted Compilation.



Figure 25.2: Hw Reward block diagram for Auto-Optimization and AI-Assisted Compilation.

bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hw Reward is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **reward**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds reward hardware to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Auto is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **YiRage**, **auto** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage auto to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

25.2 Hw Reward

Hw Reward is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **reward**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds reward hardware to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Rl Autotune is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **rl**, **autotun** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds rl autotun to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hw Reward is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **reward**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds reward hardware to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Auto is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **YiRage**, **auto** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage auto to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

25.3 YiRage Auto

YiRage Auto is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **YiRage**, **auto** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage auto to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Rl Autotune is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **rl**, **autotun** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds rl autotun to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hw Reward is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **reward**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds reward hardware to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Auto is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **YiRage**, **auto** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage auto to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

25.4 Chapter Summary

The through-line for ch25 is unchanged: hardware constraints dictate compiler passes, fusion boundaries, and deployment metrics (??). Carry bytes/token and launches/token spreadsheets forward; later chapters assume those baselines exist (??).

25.4.1 Engineering Deep Dive

Rl Autotune is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **rl**, **autotun** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds rl autotun to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hw Reward is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **reward**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds reward hardware to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Auto is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **YiRage**, **auto** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage auto to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Rl Autotune is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **rl**, **autotun** as independent knobs

routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds `rl autotun` to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). `YiRage` lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

`Hw Reward` is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **reward**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds `reward hardware` to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). `YiRage` lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

`YiRage Auto` is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **YiRage**, **auto** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds `YiRage auto` to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). `YiRage` lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

`Rl Autotune` is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **rl**, **autotun** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds `rl autotun` to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). `YiRage` lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

`Hw Reward` is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **reward**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds `reward hardware`

to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Auto is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **YiRage**, **auto** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage auto to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Rl Autotune is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **rl**, **autotun** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds rl autotun to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hw Reward is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **reward**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds reward hardware to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Auto is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **YiRage**, **auto** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage auto to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen,

so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Rl Autotune is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **rl**, **autotun** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds rl autotun to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hw Reward is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **reward**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds reward hardware to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Auto is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **YiRage**, **auto** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage auto to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Rl Autotune is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **rl**, **autotun** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds rl autotun to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record

fleet measurements alongside compiler flags (??).

Hw Reward is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **reward**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds reward hardware to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Auto is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **YiRage**, **auto** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage auto to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Rl Autotune is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **rl**, **autotun** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds rl autotun to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hw Reward is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **reward**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds reward hardware to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Auto is a production bottleneck surface for Auto-Optimization and

AI-Assisted Compilation. Teams that treat **YiRage**, **auto** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage auto to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Rl Autotune is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **rl**, **autotun** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds rl autotun to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hw Reward is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **reward**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds reward hardware to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Auto is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **YiRage**, **auto** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage auto to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Rl Autotune is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **rl**, **autotun** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real

fleets (??). The hardware–software contract here binds `rl autotun` to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hw Reward is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **reward**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds reward hardware to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Auto is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **YiRage**, **auto** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage auto to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Rl Autotune is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **rl**, **autotun** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds `rl autotun` to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hw Reward is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **reward**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds reward hardware to measurable bytes/token and launches/token outcomes across GPU, CPU, and

NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Auto is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **YiRage**, **auto** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage auto to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Rl Autotune is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **rl**, **autotun** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds rl autotun to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hw Reward is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **reward**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds reward hardware to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Auto is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **YiRage**, **auto** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage auto to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial

readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Rl Autotune is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **rl**, **autotun** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds rl autotun to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hw Reward is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **reward**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds reward hardware to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Auto is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **YiRage**, **auto** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage auto to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Rl Autotune is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **rl**, **autotun** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds rl autotun to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hw Reward is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **reward**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds reward hardware to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Auto is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **YiRage**, **auto** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage auto to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Rl Autotune is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **rl**, **autotun** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds rl autotun to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hw Reward is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **reward**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds reward hardware to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Auto is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **YiRage**, **auto** as independent knobs

routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage auto to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Rl Autotune is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **rl**, **autotun** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds rl autotun to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hw Reward is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **reward**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds reward hardware to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Auto is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **YiRage**, **auto** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage auto to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Rl Autotune is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **rl**, **autotun** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds rl autotun to measurable

bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hw Reward is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **reward**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds reward hardware to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Auto is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **YiRage**, **auto** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage auto to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

RL Autotune is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **rl**, **autotun** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds rl autotun to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hw Reward is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **reward**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds reward hardware to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks

before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

YiRage Auto is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **YiRage**, **auto** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds YiRage auto to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Rl Autotune is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **rl**, **autotun** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds rl autotun to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Hw Reward is a production bottleneck surface for Auto-Optimization and AI-Assisted Compilation. Teams that treat **reward**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds reward hardware to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Chapter 26

Production Deployment and Engineering Best Practices

Production decode teams rarely fail for lack of framework features—they fail because compiler and kernel schedules violate residency, launch, and bandwidth contracts on the SKU they actually ship (??). This chapter continues the book’s problem-first thread: name the contradiction, trace it to hardware limits, then show how passes and megakernels recover SLO on GPU, CPU, and NPU fleets (??).

26.1 Env Packaging

Env Packaging is a production bottleneck surface for Production Deployment and Engineering Best Practices. Teams that treat **deploy**, **packag** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds deploy packag to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Env Packaging is a production bottleneck surface for Production Deployment and Engineering Best Practices. Teams that treat **deploy**, **packag** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds deploy packag to

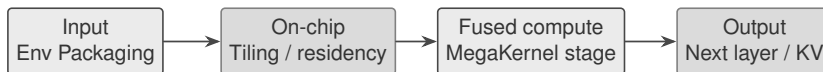


Figure 26.1: Env Packaging optimization pipeline in Production Deployment and Engineering Best Practices.



Figure 26.2: Multi Hw Ops block diagram for Production Deployment and Engineering Best Practices.

measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Multi Hw Ops is a production bottleneck surface for Production Deployment and Engineering Best Practices. Teams that treat **operation, production** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds operation production to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Pitfalls is a production bottleneck surface for Production Deployment and Engineering Best Practices. Teams that treat **pitfall, lesson** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pitfall lesson to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

26.2 Multi Hw Ops

Multi Hw Ops is a production bottleneck surface for Production Deployment and Engineering Best Practices. Teams that treat **operation**, **production** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds operation production to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Env Packaging is a production bottleneck surface for Production Deployment and Engineering Best Practices. Teams that treat **deploy**, **packag** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds deploy packag to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Multi Hw Ops is a production bottleneck surface for Production Deployment and Engineering Best Practices. Teams that treat **operation**, **production** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds operation production to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Pitfalls is a production bottleneck surface for Production Deployment and Engineering Best Practices. Teams that treat **pitfall**, **lesson** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pitfall lesson to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial

readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

26.3 Pitfalls

Pitfalls is a production bottleneck surface for Production Deployment and Engineering Best Practices. Teams that treat **pitfall**, **lesson** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pitfall lesson to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Env Packaging is a production bottleneck surface for Production Deployment and Engineering Best Practices. Teams that treat **deploy**, **packag** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds deploy packag to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Multi Hw Ops is a production bottleneck surface for Production Deployment and Engineering Best Practices. Teams that treat **operation**, **production** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds operation production to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Pitfalls is a production bottleneck surface for Production Deployment and Engineering Best Practices. Teams that treat **pitfall**, **lesson** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pitfall lesson to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets

(??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

26.4 Scheduling

Scheduling is a production bottleneck surface for Production Deployment and Engineering Best Practices. Teams that treat **schedul**, **fleet** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds schedul fleet to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Env Packaging is a production bottleneck surface for Production Deployment and Engineering Best Practices. Teams that treat **deploy**, **packag** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds deploy packag to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Multi Hw Ops is a production bottleneck surface for Production Deployment and Engineering Best Practices. Teams that treat **operation**, **production** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds operation production to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Pitfalls is a production bottleneck surface for Production Deployment and Engineering Best Practices. Teams that treat **pitfall**, **lesson** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real

fleets (??). The hardware–software contract here binds pitfall lesson to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

26.5 Chapter Summary

The through-line for ch26 is unchanged: hardware constraints dictate compiler passes, fusion boundaries, and deployment metrics (??). Carry bytes/token and launches/token spreadsheets forward; later chapters assume those baselines exist (??).

26.5.1 Engineering Deep Dive

Env Packaging is a production bottleneck surface for Production Deployment and Engineering Best Practices. Teams that treat **deploy**, **packag** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds deploy packag to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Multi Hw Ops is a production bottleneck surface for Production Deployment and Engineering Best Practices. Teams that treat **operation**, **production** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds operation production to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Pitfalls is a production bottleneck surface for Production Deployment and Engineering Best Practices. Teams that treat **pitfall**, **lesson** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real

fleets (??). The hardware–software contract here binds pitfall lesson to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Scheduling is a production bottleneck surface for Production Deployment and Engineering Best Practices. Teams that treat **schedul**, **fleet** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds schedul fleet to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Env Packaging is a production bottleneck surface for Production Deployment and Engineering Best Practices. Teams that treat **deploy**, **packag** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds deploy packag to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Multi Hw Ops is a production bottleneck surface for Production Deployment and Engineering Best Practices. Teams that treat **operation**, **production** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds operation production to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Pitfalls is a production bottleneck surface for Production Deployment and Engineering Best Practices. Teams that treat **pitfall**, **lesson** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pitfall lesson to measurable

bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Scheduling is a production bottleneck surface for Production Deployment and Engineering Best Practices. Teams that treat **schedul**, **fleet** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds schedul fleet to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Env Packaging is a production bottleneck surface for Production Deployment and Engineering Best Practices. Teams that treat **deploy**, **packag** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds deploy packag to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Multi Hw Ops is a production bottleneck surface for Production Deployment and Engineering Best Practices. Teams that treat **operation**, **production** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds operation production to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Pitfalls is a production bottleneck surface for Production Deployment and Engineering Best Practices. Teams that treat **pitfall**, **lesson** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pitfall lesson to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets

(??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Scheduling is a production bottleneck surface for Production Deployment and Engineering Best Practices. Teams that treat **schedul**, **fleet** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds schedul fleet to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Env Packaging is a production bottleneck surface for Production Deployment and Engineering Best Practices. Teams that treat **deploy**, **packag** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds deploy packag to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Multi Hw Ops is a production bottleneck surface for Production Deployment and Engineering Best Practices. Teams that treat **operation**, **production** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds operation production to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Pitfalls is a production bottleneck surface for Production Deployment and Engineering Best Practices. Teams that treat **pitfall**, **lesson** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pitfall lesson to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen,

so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Scheduling is a production bottleneck surface for Production Deployment and Engineering Best Practices. Teams that treat **schedul**, **fleet** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds schedul fleet to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Env Packaging is a production bottleneck surface for Production Deployment and Engineering Best Practices. Teams that treat **deploy**, **packag** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds deploy packag to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Multi Hw Ops is a production bottleneck surface for Production Deployment and Engineering Best Practices. Teams that treat **operation**, **production** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds operation production to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Pitfalls is a production bottleneck surface for Production Deployment and Engineering Best Practices. Teams that treat **pitfall**, **lesson** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pitfall lesson to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial

readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Scheduling is a production bottleneck surface for Production Deployment and Engineering Best Practices. Teams that treat **schedul**, **fleet** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds schedul fleet to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Env Packaging is a production bottleneck surface for Production Deployment and Engineering Best Practices. Teams that treat **deploy**, **packag** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds deploy packag to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Multi Hw Ops is a production bottleneck surface for Production Deployment and Engineering Best Practices. Teams that treat **operation**, **production** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds operation production to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Pitfalls is a production bottleneck surface for Production Deployment and Engineering Best Practices. Teams that treat **pitfall**, **lesson** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pitfall lesson to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record

fleet measurements alongside compiler flags (??).

Scheduling is a production bottleneck surface for Production Deployment and Engineering Best Practices. Teams that treat **schedul**, **fleet** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds schedul fleet to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Env Packaging is a production bottleneck surface for Production Deployment and Engineering Best Practices. Teams that treat **deploy**, **packag** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds deploy packag to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Multi Hw Ops is a production bottleneck surface for Production Deployment and Engineering Best Practices. Teams that treat **operation**, **production** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds operation production to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Pitfalls is a production bottleneck surface for Production Deployment and Engineering Best Practices. Teams that treat **pitfall**, **lesson** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pitfall lesson to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Scheduling is a production bottleneck surface for Production Deployment and Engineering Best Practices. Teams that treat **schedul**, **fleet** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds schedul fleet to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Env Packaging is a production bottleneck surface for Production Deployment and Engineering Best Practices. Teams that treat **deploy**, **packag** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds deploy packag to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Multi Hw Ops is a production bottleneck surface for Production Deployment and Engineering Best Practices. Teams that treat **operation**, **production** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds operation production to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Pitfalls is a production bottleneck surface for Production Deployment and Engineering Best Practices. Teams that treat **pitfall**, **lesson** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pitfall lesson to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Scheduling is a production bottleneck surface for Production Deployment and

Engineering Best Practices. Teams that treat **schedul**, **fleet** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds schedul fleet to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Env Packaging is a production bottleneck surface for Production Deployment and Engineering Best Practices. Teams that treat **deploy**, **packag** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds deploy packag to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Multi Hw Ops is a production bottleneck surface for Production Deployment and Engineering Best Practices. Teams that treat **operation**, **production** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds operation production to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Pitfalls is a production bottleneck surface for Production Deployment and Engineering Best Practices. Teams that treat **pitfall**, **lesson** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds pitfall lesson to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Scheduling is a production bottleneck surface for Production Deployment and Engineering Best Practices. Teams that treat **schedul**, **fleet** as independent knobs

routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds schedul fleet to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Env Packaging is a production bottleneck surface for Production Deployment and Engineering Best Practices. Teams that treat **deploy**, **packag** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds deploy packag to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Chapter 27

Future Trends in AI Compilers

Production decode teams rarely fail for lack of framework features—they fail because compiler and kernel schedules violate residency, launch, and bandwidth contracts on the SKU they actually ship (??). This chapter continues the book’s problem-first thread: name the contradiction, trace it to hardware limits, then show how passes and megakernels recover SLO on GPU, CPU, and NPU fleets (??).

27.1 Codesign

Codesign is a production bottleneck surface for Future Trends in AI Compilers. Teams that treat **co-design, hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds co-design hardware to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Codesign is a production bottleneck surface for Future Trends in AI Compilers. Teams that treat **co-design, hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds co-design hardware to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion



Figure 27.1: Codegen optimization pipeline in Future Trends in AI Compilers.



Figure 27.2: New Architectures block diagram for Future Trends in AI Compilers.

fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

New Architectures is a production bottleneck surface for Future Trends in AI Compilers. Teams that treat **architecture**, **trend** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds architecture trend to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Edge Cloud is a production bottleneck surface for Future Trends in AI Compilers. Teams that treat **edge**, **cloud** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds edge cloud to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

27.2 New Architectures

New Architectures is a production bottleneck surface for Future Trends in AI Compilers. Teams that treat **architecture**, **trend** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds architecture trend to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets

(??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Codesign is a production bottleneck surface for Future Trends in AI Compilers. Teams that treat **co-design, hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds co-design hardware to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

New Architectures is a production bottleneck surface for Future Trends in AI Compilers. Teams that treat **architecture, trend** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds architecture trend to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Edge Cloud is a production bottleneck surface for Future Trends in AI Compilers. Teams that treat **edge, cloud** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds edge cloud to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

27.3 Edge Cloud

Edge Cloud is a production bottleneck surface for Future Trends in AI Compilers. Teams that treat **edge, cloud** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds edge cloud to measurable bytes/token and launches/token

outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Codesign is a production bottleneck surface for Future Trends in AI Compilers. Teams that treat **co-design**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds co-design hardware to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

New Architectures is a production bottleneck surface for Future Trends in AI Compilers. Teams that treat **architecture**, **trend** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds architecture trend to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Edge Cloud is a production bottleneck surface for Future Trends in AI Compilers. Teams that treat **edge**, **cloud** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds edge cloud to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

27.4 Autonomous Kernels

Autonomous Kernels is a production bottleneck surface for Future Trends in AI Compilers. Teams that treat **autonomous**, **kernel** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??).

The hardware–software contract here binds autonomous kernel to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Codesign is a production bottleneck surface for Future Trends in AI Compilers. Teams that treat **co-design**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds co-design hardware to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

New Architectures is a production bottleneck surface for Future Trends in AI Compilers. Teams that treat **architecture**, **trend** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds architecture trend to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Edge Cloud is a production bottleneck surface for Future Trends in AI Compilers. Teams that treat **edge**, **cloud** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds edge cloud to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

27.5 Chapter Summary

The through-line for ch27 is unchanged: hardware constraints dictate compiler passes, fusion boundaries, and deployment metrics (??). Carry bytes/token and

launches/token spreadsheets forward; later chapters assume those baselines exist (??).

27.5.1 Engineering Deep Dive

Codesign is a production bottleneck surface for Future Trends in AI Compilers. Teams that treat **co-design**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds co-design hardware to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

New Architectures is a production bottleneck surface for Future Trends in AI Compilers. Teams that treat **architecture**, **trend** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds architecture trend to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Edge Cloud is a production bottleneck surface for Future Trends in AI Compilers. Teams that treat **edge**, **cloud** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds edge cloud to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Autonomous Kernels is a production bottleneck surface for Future Trends in AI Compilers. Teams that treat **autonomous**, **kernel** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds autonomous kernel to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks

before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Codesign is a production bottleneck surface for Future Trends in AI Compilers. Teams that treat **co-design**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds co-design hardware to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

New Architectures is a production bottleneck surface for Future Trends in AI Compilers. Teams that treat **architecture**, **trend** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds architecture trend to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Edge Cloud is a production bottleneck surface for Future Trends in AI Compilers. Teams that treat **edge**, **cloud** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds edge cloud to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Autonomous Kernels is a production bottleneck surface for Future Trends in AI Compilers. Teams that treat **autonomous**, **kernel** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds autonomous kernel to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint

matrix and record fleet measurements alongside compiler flags (??).

Codesign is a production bottleneck surface for Future Trends in AI Compilers. Teams that treat **co-design**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds co-design hardware to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

New Architectures is a production bottleneck surface for Future Trends in AI Compilers. Teams that treat **architecture**, **trend** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds architecture trend to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Edge Cloud is a production bottleneck surface for Future Trends in AI Compilers. Teams that treat **edge**, **cloud** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds edge cloud to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Autonomous Kernels is a production bottleneck surface for Future Trends in AI Compilers. Teams that treat **autonomous**, **kernel** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds autonomous kernel to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Codesign is a production bottleneck surface for Future Trends in AI Compilers.

Teams that treat **co-design, hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds co-design hardware to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

New Architectures is a production bottleneck surface for Future Trends in AI Compilers. Teams that treat **architecture, trend** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds architecture trend to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Edge Cloud is a production bottleneck surface for Future Trends in AI Compilers. Teams that treat **edge, cloud** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds edge cloud to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Autonomous Kernels is a production bottleneck surface for Future Trends in AI Compilers. Teams that treat **autonomous, kernel** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds autonomous kernel to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Codesign is a production bottleneck surface for Future Trends in AI Compilers. Teams that treat **co-design, hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–

software contract here binds co-design hardware to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

New Architectures is a production bottleneck surface for Future Trends in AI Compilers. Teams that treat **architecture**, **trend** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds architecture trend to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Edge Cloud is a production bottleneck surface for Future Trends in AI Compilers. Teams that treat **edge**, **cloud** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds edge cloud to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Autonomous Kernels is a production bottleneck surface for Future Trends in AI Compilers. Teams that treat **autonomous**, **kernel** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds autonomous kernel to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Codesign is a production bottleneck surface for Future Trends in AI Compilers. Teams that treat **co-design**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds co-design hardware to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers

the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

New Architectures is a production bottleneck surface for Future Trends in AI Compilers. Teams that treat **architecture**, **trend** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds architecture trend to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Edge Cloud is a production bottleneck surface for Future Trends in AI Compilers. Teams that treat **edge**, **cloud** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds edge cloud to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Autonomous Kernels is a production bottleneck surface for Future Trends in AI Compilers. Teams that treat **autonomous**, **kernel** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds autonomous kernel to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Codesign is a production bottleneck surface for Future Trends in AI Compilers. Teams that treat **co-design**, **hardware** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds co-design hardware to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map

each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

New Architectures is a production bottleneck surface for Future Trends in AI Compilers. Teams that treat **architecture**, **trend** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds architecture trend to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Edge Cloud is a production bottleneck surface for Future Trends in AI Compilers. Teams that treat **edge**, **cloud** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds edge cloud to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).

Autonomous Kernels is a production bottleneck surface for Future Trends in AI Compilers. Teams that treat **autonomous**, **kernel** as independent knobs routinely ship kernels that win microbenchmarks yet miss decode SLOs on real fleets (??). The hardware–software contract here binds autonomous kernel to measurable bytes/token and launches/token outcomes across GPU, CPU, and NPU targets (??). YiRage lowers the same constraints as explicit legality checks before codegen, so illegal fusion fails in CI rather than in customer dashboards (??). Industrial readers should map each paragraph to the Chapter 3 constraint matrix and record fleet measurements alongside compiler flags (??).